

day01

一、为什么要学习Vue

- 1.前端必备技能
- 2.岗位多，绝大互联网公司都在使用Vue
- 3.提高开发效率
- 4.高薪必备技能（Vue2+Vue3）

二、什么是Vue

概念：Vue (读音 /vju:/，类似于 view) 是一套 构建用户界面 的 渐进式 框架

Vue2官网：<https://v2.cn.vuejs.org/>

1.什么是构建用户界面

基于数据渲染出用户可以看到的界面



2.什么是渐进式

所谓渐进式就是循序渐进，不一定非得把Vue中的所有API都学完才能开发Vue，可以学一点开发一点

Vue的两种开发方式：

- 1. Vue核心包开发
场景：局部模块改造
- 2. Vue核心包&Vue插件&工程化
场景：整站开发

3.什么是框架

所谓框架：就是一套完整的解决方案

举个栗子

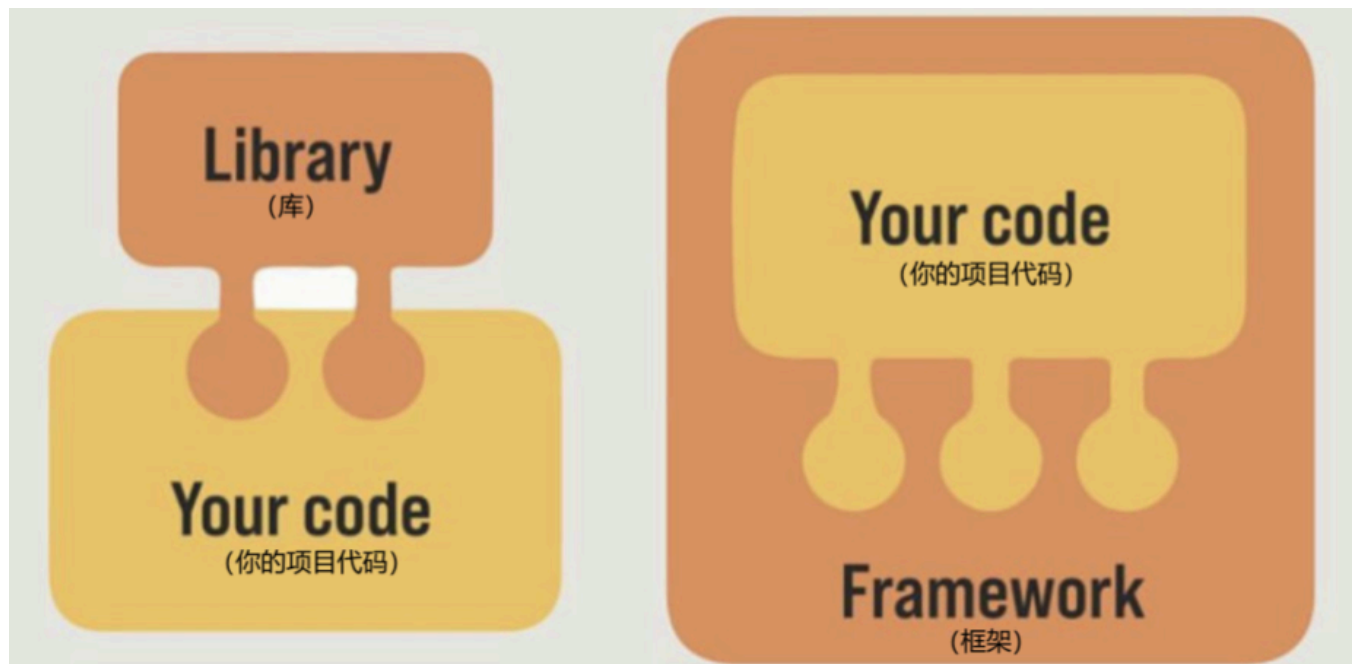
如果把一个完整的项目比喻为一个装修好的房子，那么框架就是一个毛坯房。

我们只需要在“毛坯房”的基础上，增加功能代码即可。

提到框架，不得不提一下库。

- 库，类似工具箱，是一堆方法的集合，比如 axios、lodash、echarts等
- 框架，是一套完整的解决方案，实现了大部分功能，我们只需要按照一定的规则去编码即可。

下图是 库 和 框架的对比。



框架的特点：有一套必须让开发者遵守的**规则**或者**约束**

咱们学框架就是学习的这些规则 [官网](#)

总结：什么是Vue？

Vue是什么：

什么是构建用户界面：

什么是渐进式：

什么是框架：

三、创建Vue实例

我们已经知道了Vue框架可以 基于数据帮助我们渲染出用户界面，那应该怎么做呢？



比如就上面这个数据，基于提供好的msg 怎么渲染后右侧可展示的数据呢？

核心步骤（4步）：

1. 准备容器
2. 引包（官网） — 开发版本/生产版本
3. 创建Vue实例 `new Vue()`
4. 指定配置项，渲染数据
 1. `el`:指定挂载点
 2. `data`提供数据

```
<div id="app">
  {{ msg }}
</div>

<script src="https://cdn.jsdelivr.net/npm/vue@2.7.10/dist/vue.js"></script>

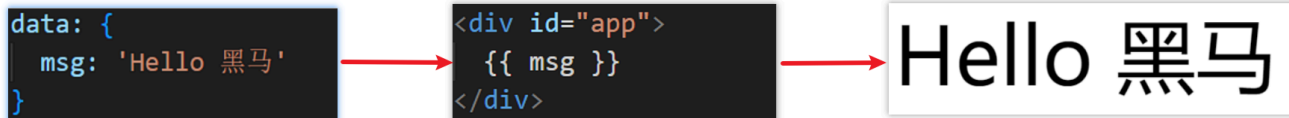
<script>
  const app = new Vue({
    el: '#app',
    data: {
      msg: 'Hello 黑马'
    }
  })
</script>
```

总结：创建Vue实例需要执行哪4步

四、插值表达式 {{}}

插值表达式是一种Vue的模板语法

我们可以用插值表达式渲染出Vue提供的数据



1.作用：利用表达式进行插值，渲染到页面中

表达式：是可以被求值的代码，JS引擎会讲其计算出一个结果

以下的情况都是表达式：

```
money + 100  
money - 100  
money * 10  
money / 10  
price >= 100 ? '真贵':'还行'  
obj.name  
arr[0]  
fn()  
obj.fn()
```

2.语法

插值表达式语法：{{ 表达式 }}

```
<h3>{{title}}</h3>  
  
<p>{{nickName.toUpperCase()}}</p>  
  
<p>{{age >= 18 ? '成年':'未成年'}}</p>  
  
<p>{{obj.name}}</p>  
  
<p>{{fn()}}</p>
```

3.错误用法

- 1.在插值表达式中使用的数据 必须在data中进行了提供
`<p>{{hobby}}</p>` //如果在data中不存在 则会报错
- 2.支持的是表达式，而非语句，比如：if for ...
`<p>{{if}}</p>`
- 3.不能在标签属性中使用 {{ }} 插值（插值表达式只能标签中间使用）
`<p title="{{username}}">我是P标签</p>`

4.总结

- 1.插值表达式的作用是什么
- 2.语法是什么
- 3.插值表达式的注意事项

五、响应式特性

1.什么是响应式？

简单理解就是数据变，视图对应变。

2.如何访问 和 修改 data中的数据（响应式演示）

data中的数据, 最终会被添加到实例上

- ① 访问数据: "实例.属性名"
- ② 修改数据: "实例.属性名" = "值"



3.总结

1. 什么是响应式
2. 如何访问和修改data中的数据呢

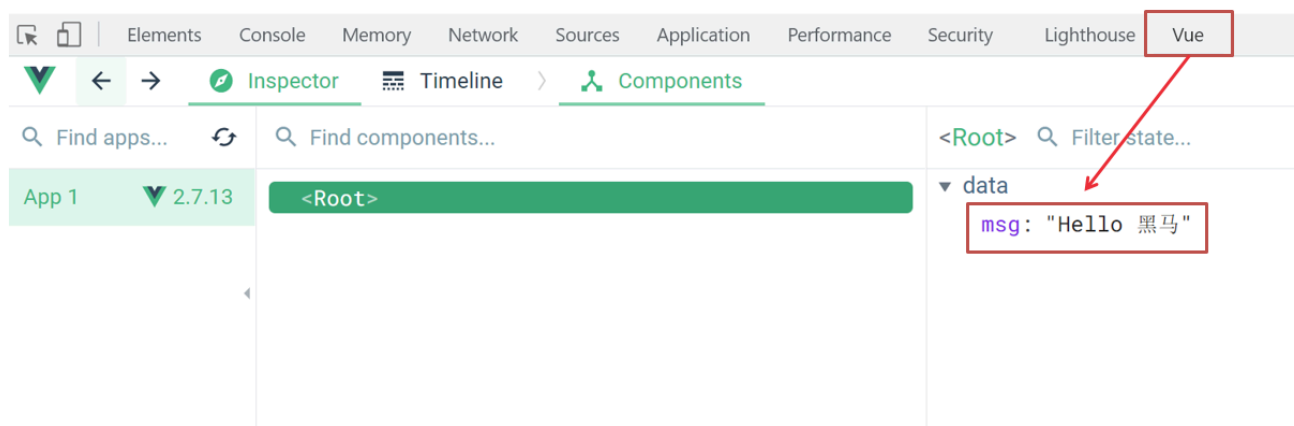
六、Vue开发者工具安装

1. 通过谷歌应用商店安装（国外网站）
2. 极简插件下载（推荐） <https://chrome.zzzmh.cn/index>

安装步骤：



安装之后可以F12后看到多一个Vue的调试面板



七、Vue中的常用指令

概念：指令（Directives）是 Vue 提供的带有 **v- 前缀** 的特殊 标签属性。

为啥要学：提高程序员操作 DOM 的效率。

vue 中的指令按照不同的用途可以分为如下 6 大类：

- 内容渲染指令（v-html、v-text）
- 条件渲染指令（v-show、v-if、v-else、v-else-if）
- 事件绑定指令（v-on）
- 属性绑定指令（v-bind）
- 双向绑定指令（v-model）
- 列表渲染指令（v-for）

指令是 vue 开发中最基础、最常用、最简单的知识点。

八、内容渲染指令

内容渲染指令用来辅助开发者渲染 DOM 元素的文本内容。常用的内容渲染指令有如下2个：

- v-text (类似innerText)
 - 使用语法： `<p v-text="uname">hello</p>`，意思是将 uame 值渲染到 p 标签中
 - 类似 innerText，使用该语法，会覆盖 p 标签原有内容
- v-html (类似 innerHTML)
 - 使用语法： `<p v-html="intro">hello</p>`，意思是将 intro 值渲染到 p 标签中
 - 类似 innerHTML，使用该语法，会覆盖 p 标签原有内容
 - 类似 innerHTML，使用该语法，能够将HTML标签的样式呈现出来。

代码演示：

```
<div id="app">
  <h2>个人信息</h2>
  // 既然指令是vue提供的特殊的html属性，所以咱们写的时候就当成属性来用即可
  <p v-text="uname">姓名: </p>
  <p v-html="intro">简介: </p>
</div>

<script>
  const app = new Vue({
    el: '#app',
    data: {
      uname: '张三',
      intro: '<h2>这是一个<strong>非常优秀</strong>的boy<h2>'
    }
  })
</script>
```

九、条件渲染指令

条件判断指令，用来辅助开发者按需控制 DOM 的显示与隐藏。条件渲染指令有如下两个，分别是：

1. v-show

1. 作用： 控制元素显示隐藏
2. 语法： `v-show = "表达式"` 表达式值为 true 显示， false 隐藏
3. 原理： 切换 display:none 控制显示隐藏
4. 场景： 频繁切换显示隐藏的场景



2. v-if

1. 作用： 控制元素显示隐藏（条件渲染）
2. 语法： `v-if="表达式"` 表达式值 true显示， false 隐藏
3. 原理： 基于条件判断，是否创建 或 移除元素节点
4. 场景： 要么显示，要么隐藏，不频繁切换的场景



示例代码：

```
<div id="app">
  <div class="box">我是v-show控制的盒子</div>
  <div class="box">我是v-if控制的盒子</div>
</div>

<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      flag: false
    }
  })
</script>
```

3. v-else 和 v-else-if

1. 作用： 辅助v-if进行判断渲染
2. 语法： `v-else v-else-if="表达式"`
3. 需要紧接着v-if使用

示例代码：

```
<div id="app">
  <p>性别：♂ 男</p>
  <p>性别：♀ 女</p>
</div>
```

```
<p>成绩评定A: 奖励电脑一台</p>
<p>成绩评定B: 奖励周末郊游</p>
<p>成绩评定C: 奖励零食礼包</p>
<p>成绩评定D: 惩罚一周不能玩手机</p>
</div>

<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>

  const app = new Vue({
    el: '#app',
    data: {
      gender: 2,
      score: 95
    }
  })
</script>
```

十、事件绑定指令

使用Vue时，如需为DOM注册事件，及其的简单，语法如下：

- <button v-on:事件名="内联语句">按钮
- <button v-on:事件名="处理函数">按钮
- <button v-on:事件名="处理函数(实参)">按钮
- **v-on**：简写为 @

1. 内联语句

```
<div id="app">
  <button @click="count--"></button>
  <span>{{ count }}</span>
  <button v-on:click="count++"></button>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      count: 100
    }
  })
</script>
```

2. 事件处理函数

注意：

- 事件处理函数应该写到一个跟data同级的配置项（methods）中
- methods中的函数内部的this都指向Vue实例

```

<div id="app">
  <button>切换显示隐藏</button>
  <h1 v-show="isShow">黑马程序员</h1>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      isShow: true
    }
  })
</script>

```

3.给事件处理函数传参

- 如果不传递任何参数，则方法无需加小括号；methods方法中可以直接使用 e 当做事件对象
- 如果传递了参数，则实参 `$event` 表示事件对象，固定用法。

```

<style>
  .box {
    border: 3px solid #000000;
    border-radius: 10px;
    padding: 20px;
    margin: 20px;
    width: 200px;
  }
  h3 {
    margin: 10px 0 20px 0;
  }
  p {
    margin: 20px;
  }
</style>

<div id="app">
  <div class="box">
    <h3>小黑自动售货机</h3>
    <button>可乐5元</button>
    <button>咖啡10元</button>
    <button>牛奶8元</button>
  </div>
  <p>银行卡余额: {{ money }}元</p>
</div>

<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      money: 100
    }
  })

```

```
</script>
```

十一、属性绑定指令

1. **作用：** 动态设置html的标签属性 比如： src、 url、 title
2. **语法：** v-bind:属性名=“表达式”
3. v-bind:可以简写成 => :

比如，有一个图片，它的 `src` 属性值，是一个图片地址。这个地址在数据 data 中存储。

则可以这样设置属性值：

- ``
- `` （v-bind可以省略）

```
<div id="app">
  
  
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      imgUrl: './imgs/10-02.png',
      msg: 'hello 波仔'
    }
  })
</script>
```

十二、小案例-波仔的学习之旅

需求：默认展示数组中的第一张图片，点击上一页下一页来回切换数组中的图片

实现思路：

- 1.数组存储图片路径 ['url1','url2','url3', ...]
- 2.可以准备个下标index 去数组中取图片地址。
- 3.通过v-bind给src绑定当前的图片地址
- 4.点击上一页下一页只需要修改下标的值即可
- 5.当展示第一张的时候，上一页按钮应该隐藏。展示最后一张的时候，下一页按钮应该隐藏

```
<div id="app">
```

```

<button>上一页</button>
<div>
  <img src alt="">
</div>
<button>下一页</button>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      list: [
        './imgs/11-00.gif',
        './imgs/11-01.gif',
        './imgs/11-02.gif',
        './imgs/11-03.gif',
        './imgs/11-04.png',
        './imgs/11-05.png',
      ]
    }
  })
</script>

```

十三、列表渲染指令

Vue 提供了 v-for 列表渲染指令，用来辅助开发者基于一个数组来循环渲染一个列表结构。

v-for 指令需要使用 `(item, index) in arr` 形式的特殊语法，其中：

- item 是数组中的每一项
- index 是每一项的索引，不需要可以省略
- arr 是被遍历的数组

此语法也可以遍历 **对象和数字**

```

//遍历对象
<div v-for="(value, key, index) in object">{{value}}</div>
value: 对象中的值
key: 对象中的键
index: 遍历索引从0开始

//遍历数字
<p v-for="item in 10">{{item}}</p>
item从1 开始

```


十四、小案例-小黑的书架

需求：

- 1.根据左侧数据渲染出右侧列表（v-for）
- 2.点击删除按钮时，应该把当前行从列表中删除（获取当前行的id，利用filter进行过滤）

```
booksList: [  
  { id: 1, name: '《红楼梦》', author: '曹雪芹' },  
  { id: 2, name: '《西游记》', author: '吴承恩' },  
  { id: 3, name: '《水浒传》', author: '施耐庵' },  
  { id: 4, name: '《三国演义》', author: '罗贯中' }  
]
```

小黑的书架

- 《红楼梦》 曹雪芹 删除
- 《西游记》 吴承恩 删除
- 《水浒传》 施耐庵 删除
- 《三国演义》 罗贯中 删除

准备代码：

```
<div id="app">  
  <h3>小黑的书架</h3>  
  <ul>  
    <li>  
      <span>《红楼梦》</span>  
      <span>曹雪芹</span>  
      <button>删除</button>  
    </li>  
  </ul>  
</div>  
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>  
<script>  
  const app = new Vue({  
    el: '#app',  
    data: {  
      booksList: [  
        { id: 1, name: '《红楼梦》', author: '曹雪芹' },  
        { id: 2, name: '《西游记》', author: '吴承恩' },  
        { id: 3, name: '《水浒传》', author: '施耐庵' },  
        { id: 4, name: '《三国演义》', author: '罗贯中' }  
      ]  
    }  
  })  
</script>
```

十五、v-for中的key

语法：key="唯一值"

作用：给列表项添加的**唯一标识**。便于Vue进行列表项的**正确排序复用**。

为什么加key：Vue 的默认行为会尝试原地修改元素（**就地复用**）

实例代码：

```
<ul>
  <li v-for="(item, index) in booksList" :key="item.id">
    <span>{{ item.name }}</span>
    <span>{{ item.author }}</span>
    <button @click="del(item.id)">删除</button>
  </li>
</ul>
```

注意：

1. key 的值只能是字符串 或 数字类型
2. key 的值必须具有唯一性
3. 推荐使用 id 作为 key（唯一），不推荐使用 index 作为 key（会变化，不对应）

十六、双向绑定指令

所谓双向绑定就是：

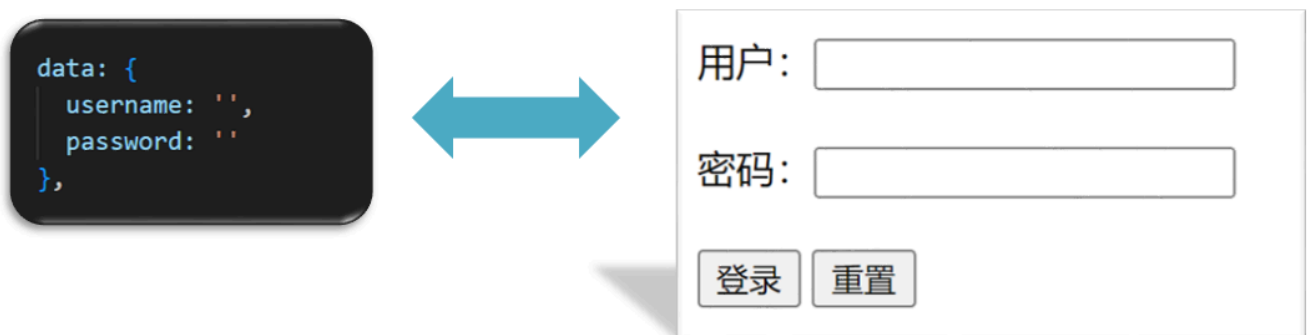
1. 数据改变后，呈现的页面结果会更新
2. 页面结果更新后，数据也会随之而变

作用：给**表单元素**（input、radio、select）使用，双向绑定数据，可以快速 **获取** 或 **设置** 表单元素内容

语法：v-model="变量"

需求：使用双向绑定实现以下需求

1. 点击登录按钮获取表单中的内容
2. 点击重置按钮清空表单中的内容



```
<div id="app">
  账户: <input type="text"> <br><br>
  密码: <input type="password"> <br><br>
```

```
<button>登录</button>
<button>重置</button>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      username: '',
      password: ''
    },
  })
</script>
```

十七、综合案例-小黑记事本

小黑记事本

添加任务

1. 跑步锻炼20分钟

×

2. 复习数组语法

合计: 2

清空任务

功能需求：

1. 列表渲染
2. 删除功能
3. 添加功能
4. 底部统计 和 清空

day02

一、今日学习目标

1.指令补充

1. 指令修饰符
2. v-bind对样式增强的操作
3. v-model应用于其他表单元素

2.computed计算属性

1. 基础语法
2. 计算属性vs方法
3. 计算属性的完整写法
4. 成绩案例

3.watch侦听器

1. 基础写法
2. 完整写法

4.综合案例（演示）

1. 渲染 / 删除 / 修改数量 / 全选 / 反选 / 统计总价 / 持久化

二、指令修饰符

1.什么是指令修饰符？

所谓指令修饰符就是通过“.”指明一些指令**后缀** 不同的**后缀**封装了不同的处理操作 → 简化代码

2.按键修饰符

- @keyup.enter →当点击enter键的时候才触发

代码演示：

```
<div id="app">
  <h3>@keyup.enter → 监听键盘回车事件</h3>
  <input v-model="username" type="text">
</div>
```

```

<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      username: ''
    },
    methods: {

    }
  })
</script>

```

3.v-model修饰符

- v-model.trim →去除首位空格
- v-model.number →转数字

4.事件修饰符

- @事件名.stop → 阻止冒泡
- @事件名.prevent →阻止默认行为
- @事件名.stop.prevent →可以连用 即阻止事件冒泡也阻止默认行为

```

<style>
  .father {
    width: 200px;
    height: 200px;
    background-color: pink;
    margin-top: 20px;
  }
  .son {
    width: 100px;
    height: 100px;
    background-color: skyblue;
  }
</style>

<div id="app">
  <h3>v-model修饰符 .trim .number</h3>
  姓名: <input v-model="username" type="text"><br>
  年纪: <input v-model="age" type="text"><br>

  <h3>@事件名.stop → 阻止冒泡</h3>
  <div @click="fatherFn" class="father">
    <div @click="sonFn" class="son">儿子</div>
  </div>

  <h3>@事件名.prevent → 阻止默认行为</h3>
  <a @click href="http://www.baidu.com">阻止默认行为</a>
</div>

```

```
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      username: '',
      age: '',
    },
    methods: {
      fatherFn () {
        alert('老父亲被点击了')
      },
      sonFn (e) {
        // e.stopPropagation()
        alert('儿子被点击了')
      }
    }
  })
</script>
```

三、v-bind对样式控制的增强-操作class

为了方便开发者进行样式控制，Vue 扩展了 v-bind 的语法，可以针对 **class 类名** 和 **style 行内样式** 进行控制。

1.语法：

```
<div> :class = "对象/数组">这是一个div</div>
```

2.对象语法

当class动态绑定的是**对象**时，**键就是类名，值就是布尔值**，如果值是true，就有这个类，否则没有这个类

```
<div class="box" :class="{ 类名1: 布尔值, 类名2: 布尔值 }"></div>
```

适用场景：一个类名，来回切换

3.数组语法

当class动态绑定的是**数组**时 → 数组中所有的类，都会添加到盒子上，本质就是一个 class 列表

```
<div class="box" :class="[ 类名1, 类名2, 类名3 ]"></div>
```

使用场景:批量添加或删除类

4.代码练习

```
<style>
  .box {
    width: 200px;
    height: 200px;
    border: 3px solid #000;
    font-size: 30px;
    margin-top: 10px;
  }
  .pink {
    background-color: pink;
  }
  .big {
    width: 300px;
    height: 300px;
  }
</style>

<div id="app">
  <!--绑定对象-->
  <div class="box">黑马程序员</div>
  <!--绑定数组-->
  <div class="box">黑马程序员</div>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {

    }
  })
</script>
```

四、京东秒杀-tab栏切换导航高亮

1.需求：

当我们点击哪个tab页签时，哪个tab页签就高亮

2.准备代码:

```
<style>
  * {
    margin: 0;
    padding: 0;
  }
```



```

ul {
  display: flex;
  border-bottom: 2px solid #e01222;
  padding: 0 10px;
}
li {
  width: 100px;
  height: 50px;
  line-height: 50px;
  list-style: none;
  text-align: center;
}
li a {
  display: block;
  text-decoration: none;
  font-weight: bold;
  color: #333333;
}
li a.active {
  background-color: #e01222;
  color: #fff;
}

</style>

<div id="app">
  <ul>
    <li><a class="active" href="#">京东秒杀</a></li>
    <li><a href="#">每日特价</a></li>
    <li><a href="#">品类秒杀</a></li>
  </ul>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      list: [
        { id: 1, name: '京东秒杀' },
        { id: 2, name: '每日特价' },
        { id: 3, name: '品类秒杀' }
      ]
    }
  })
</script>

```

3.思路：

- 1.基于数据，动态渲染tab（v-for）
- 2.准备一个下标 记录高亮的是哪一个 tab
- 3.基于下标动态切换class的类名

五、v-bind对有样式控制的增强-操作style

1.语法

```
<div class="box" :style="{ CSS属性名1: CSS属性值, CSS属性名2: CSS属性值 }"></div>
```

2.代码练习

```
<style>
  .box {
    width: 200px;
    height: 200px;
    background-color: rgb(187, 150, 156);
  }
</style>
<div id="app">
  <div class="box"></div>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {

    }
  })
</script>
```

3.进度条案例

```
<style>
  .progress {
    height: 25px;
    width: 400px;
    border-radius: 15px;
    background-color: #272425;
    border: 3px solid #272425;
    box-sizing: border-box;
    margin-bottom: 30px;
  }
  .inner {
    width: 50%;
    height: 20px;
    border-radius: 10px;
    text-align: right;
    position: relative;
    background-color: #409eff;
    background-size: 20px 20px;
  }
```

```

    box-sizing: border-box;
    transition: all 1s;
  }
  .inner span {
    position: absolute;
    right: -20px;
    bottom: -25px;
  }
</style>

<div id="app">
  <div class="progress">
    <div class="inner">
      <span>50%</span>
    </div>
  </div>
  <button>设置25%</button>
  <button>设置50%</button>
  <button>设置75%</button>
  <button>设置100%</button>
</div>

<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {

    }
  })
</script>

```

六、v-model在其他表单元素的使用

1.讲解内容：

常见的表单元素都可以用 v-model 绑定关联 → 快速 获取 或 设置 表单元素的值

它会根据 控件类型 自动选取 正确的方法 来更新元素

```

输入框  input:text    --> value
文本域  textarea     --> value
复选框  input:checkbox  --> checked
单选框  input:radio  --> checked
下拉菜单 select      --> value
...

```

2.代码准备

```
<style>
  textarea {
    display: block;
    width: 240px;
    height: 100px;
    margin: 10px 0;
  }
</style>
<div id="app">
  <h3>小黑学习网</h3>
  姓名:
    <input type="text">
    <br><br>
  是否单身:
    <input type="checkbox">
    <br><br>
  <!--
    前置理解:
      1. name: 给单选框加上 name 属性 可以分组 → 同一组互相会互斥
      2. value: 给单选框加上 value 属性, 用于提交给后台的数据
    结合 Vue 使用 → v-model
  -->
  性别:
    <input type="radio">男
    <input type="radio">女
    <br><br>
  <!--
    前置理解:
      1. option 需要设置 value 值, 提交给后台
      2. select 的 value 值, 关联了选中的 option 的 value 值
    结合 Vue 使用 → v-model
  -->
  所在城市:
    <select>
      <option>北京</option>
      <option>上海</option>
      <option>成都</option>
      <option>南京</option>
    </select>
    <br><br>
  自我描述:
    <textarea></textarea>
    <button>立即注册</button>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {

    }
  })
</script>
```

```
  })
</script>
```

七、computed计算属性

1.概念

基于**现有的数据**，计算出来的**新属性**。 **依赖**的数据变化，**自动**重新计算。

2.语法

1. 声明在 **computed 配置项**中，一个计算属性对应一个函数
2. 使用起来和普通属性一样使用 {{ 计算属性名 }}

3.注意

1. computed配置项和data配置项是**同级**的
2. computed中的计算属性**虽然是函数的写法**，但他**依然是个属性**
3. computed中的计算属性**不能**和data中的属性**同名**
4. 使用computed中的计算属性和使用data中的属性是一样的用法
5. computed中计算属性内部的**this**依然**指向的是Vue实例**

4.案例

比如我们可以使用计算属性实现下面这个业务场景



5.代码准备

```
<style>
  table {
    border: 1px solid #000;
    text-align: center;
    width: 240px;
  }
  th,td {
    border: 1px solid #000;
  }
</style>
```

```

    h3 {
      position: relative;
    }
  </style>

<div id="app">
  <h3>小黑的礼物清单</h3>
  <table>
    <tr>
      <th>名字</th>
      <th>数量</th>
    </tr>
    <tr v-for="(item, index) in list" :key="item.id">
      <td>{{ item.name }}</td>
      <td>{{ item.num }}个</td>
    </tr>
  </table>

  <!-- 目标：统计求和，求得礼物总数 -->
  <p>礼物总数： ? 个</p>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      // 现有的数据
      list: [
        { id: 1, name: '篮球', num: 1 },
        { id: 2, name: '玩具', num: 2 },
        { id: 3, name: '铅笔', num: 5 },
      ]
    }
  })
</script>

```

八、computed计算属性 VS methods方法

1.computed计算属性

作用：封装了一段对于**数据**的处理，求得一个**结果**

语法：

1. 写在computed配置项中
2. 作为属性，直接使用
 - js中使用计算属性：this.计算属性
 - 模板中使用计算属性：{{计算属性}}

2.methods计算属性

作用：给Vue实例提供一个**方法**，调用以**处理业务逻辑**。

语法：

1. 写在methods配置项中
2. 作为方法调用
 - js中调用：this.方法名()
 - 模板中调用 {{方法名()}} 或者 @事件名="方法名"

3.计算属性的优势

1. 缓存特性（提升性能）

计算属性会对计算出来的结果缓存，再次使用直接读取缓存，
依赖项变化了，会自动重新计算 → 并再次缓存
2. methods没有缓存特性
3. 通过代码比较

```
<style>
  table {
    border: 1px solid #000;
    text-align: center;
    width: 300px;
  }
  th,td {
    border: 1px solid #000;
  }
  h3 {
    position: relative;
  }
  span {
    position: absolute;
    left: 145px;
    top: -4px;
    width: 16px;
    height: 16px;
    color: white;
    font-size: 12px;
    text-align: center;
    border-radius: 50%;
    background-color: #e63f32;
  }
</style>

<div id="app">
  <h3>小黑的礼物清单 𐄂<span>?</span></h3>
```

```

<table>
  <tr>
    <th>名字</th>
    <th>数量</th>
  </tr>
  <tr v-for="(item, index) in list" :key="item.id">
    <td>{{ item.name }}</td>
    <td>{{ item.num }}个</td>
  </tr>
</table>

<p>礼物总数: {{ totalCount }} 个</p>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
const app = new Vue({
  el: '#app',
  data: {
    // 现有的数据
    list: [
      { id: 1, name: '篮球', num: 3 },
      { id: 2, name: '玩具', num: 2 },
      { id: 3, name: '铅笔', num: 5 },
    ]
  },
  computed: {
    totalCount () {
      let total = this.list.reduce((sum, item) => sum + item.num, 0)
      return total
    }
  }
})
</script>

```

4.总结

- 1.computed **有缓存特性**，methods **没有缓存**
- 2.当一个结果依赖其他多个值时，推荐使用计算属性
- 3.当处理业务逻辑时，推荐使用methods方法，比如事件的处理函数

九、计算属性的完整写法

既然计算属性也是属性，能访问，应该也能修改了？

1. 计算属性默认的简写，只能读取访问，不能 "修改"
2. 如果要 "修改" → 需要写计算属性的完整写法


```
computed: {  
  计算属性名 () {  
    一段代码逻辑 (计算逻辑)  
    return 结果  
  }  
}
```



```
computed: {  
  计算属性名: {  
    get() {  
      一段代码逻辑 (计算逻辑)  
      return 结果  
    },  
    set(修改的值) {  
      一段代码逻辑 (修改逻辑)  
    }  
  }  
}
```

完整写法代码演示

```
<div id="app">  
  姓: <input type="text" v-model="firstName"> +  
  名: <input type="text" v-model="lastName"> =  
  <span></span><br><br>  
  <button>改名卡</button>  
</div>  
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>  
<script>  
  const app = new Vue({  
    el: '#app',  
    data: {  
      firstName: '刘',  
      lastName: '备'  
    },  
    computed: {  
  
    },  
    methods: {  
  
    }  
  })  
</script>
```

十、综合案例-成绩案例

科目:

分数:

添加

v-model修饰符

编号

科目

成绩

操作

1

语文

46

删除

2

英语

80

删除

3

数学

100

删除

总分: 246

平均分: 82

v-bind动态控制样式

计算属性

功能描述:

- 1.渲染功能
- 2.删除功能
- 3.添加功能
- 4.统计总分，求平均分

思路分析:

- 1.渲染功能 v-for :key v-bind:动态绑定class的样式
- 2.删除功能 v-on绑定事件， 阻止a标签的默认行为
- 3.v-model的修饰符 .trim、 .number、 判断数据是否为空后 再添加、添加后清空文本框的数据
- 4.使用计算属性computed 计算总分和平均分的值

十一、watch侦听器（监视器）

1.作用:

监视数据变化，执行一些业务逻辑或异步操作

2.语法:

- 1. watch同样声明在跟data同级的配置项中
- 2. 简单写法： 简单类型数据直接监视
- 3. 完整写法： 添加额外配置项

```
data: {  
  words: '苹果',  
}
```

```

obj: {
  words: '苹果'
}
},

watch: {
  // 该方法会在数据变化时，触发执行
  数据属性名 (newValue, oldValue) {
    一些业务逻辑 或 异步操作。
  },
  '对象.属性名' (newValue, oldValue) {
    一些业务逻辑 或 异步操作。
  }
}
}

```

3.侦听器代码准备

```

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-size: 18px;
}
#app {
  padding: 10px 20px;
}
.query {
  margin: 10px 0;
}
.box {
  display: flex;
}
textarea {
  width: 300px;
  height: 160px;
  font-size: 18px;
  border: 1px solid #dedede;
  outline: none;
  resize: none;
  padding: 10px;
}
textarea:hover {
  border: 1px solid #1589f5;
}
.transbox {
  width: 300px;
  height: 160px;
  background-color: #f0f0f0;
  padding: 10px;
  border: none;
}
.tip-box {

```



```

// 请求方式: get
// 请求参数:
// (1) words: 需要被翻译的文本 (必传)
// (2) lang: 需要被翻译成的语言 (可选) 默认值-意大利
// -----

const app = new Vue({
  el: '#app',
  data: {
    words: ''
  },
  // 具体讲解: (1) watch语法 (2) 具体业务实现
})
</script>

```

十二、翻译案例-代码实现

```

<script>
// 接口地址: https://applet-base-api-t.itheima.net/api/translate
// 请求方式: get
// 请求参数:
// (1) words: 需要被翻译的文本 (必传)
// (2) lang: 需要被翻译成的语言 (可选) 默认值-意大利
// -----

const app = new Vue({
  el: '#app',
  data: {
    //words: ''
    obj: {
      words: ''
    },
    result: '', // 翻译结果
    // timer: null // 延时器id
  },
  // 具体讲解: (1) watch语法 (2) 具体业务实现
  watch: {
    // 该方法会在数据变化时调用执行
    // newValue新值, oldValue老值 (一般不用)
    // words (newValue) {
    //   console.log('变化了', newValue)
    // }

    'obj.words' (newValue) {
      // console.log('变化了', newValue)
      // 防抖: 延迟执行 → 干啥事先等一等, 延迟一会, 一段时间内没有再次触发, 才执行
      clearTimeout(this.timer)
      this.timer = setTimeout(async () => {
        const res = await axios({

```

```

        url: 'https://applet-base-api-t.itheima.net/api/translate',
        params: {
            words: newValue
        }
    })
    this.result = res.data.data
    console.log(res.data.data)
}, 300)
}
}
})
</script>

```

十三、watch侦听器

1.语法

完整写法 → 添加额外的配置项

1. deep:true 对复杂类型进行深度监听
2. immdiate:true 初始化 立刻执行一次

```

data: {
  obj: {
    words: '苹果',
    lang: 'italy'
  },
},

watch: { // watch 完整写法
  对象: {
    deep: true, // 深度监视
    immdiate:true, // 立即执行handler函数
    handler (newValue) {
      console.log(newValue)
    }
  }
}
}

```

2.需求



- 当文本框输入的时候 右侧翻译内容要时时变化
- 当下拉框中的语言发生变化的时候 右侧翻译的内容依然要时时变化
- 如果文本框中有默认值的话要立即翻译

3.代码实现

```
<script>
  const app = new Vue({
    el: '#app',
    data: {
      obj: {
        words: '小黑',
        lang: 'italy'
      },
      result: '', // 翻译结果
    },
    watch: {
      obj: {
        deep: true, // 深度监视
        immediate: true, // 立刻执行，一进入页面handler就立刻执行一次
        handler (newValue) {
          clearTimeout(this.timer)
          this.timer = setTimeout(async () => {
            const res = await axios({
              url: 'https://applet-base-api-t.itheima.net/api/translate',
              params: newValue
            })
            this.result = res.data.data
            console.log(res.data.data)
          }, 300)
        }
      }
    }
  })
</script>
```

4.总结

watch侦听器的写法有几种？

1.简单写法

```
watch: {  
  数据属性名 (newValue, oldValue) {  
    一些业务逻辑 或 异步操作。  
  },  
  '对象.属性名' (newValue, oldValue) {  
    一些业务逻辑 或 异步操作。  
  }  
}
```

2.完整写法

```
watch: { // watch 完整写法  
  数据属性名: {  
    deep: true, // 深度监视(针对复杂类型)  
    immediate: true, // 是否立刻执行一次handler  
    handler (newValue) {  
      console.log(newValue)  
    }  
  }  
}
```

十四、综合案例

购物车案例



🏠 / 购物车

选中	图片	单价	个数	小计	操作
☒		6	- 2 +	12	删除
☐		7	- 1 +	14	删除
☐ 全选				总价：¥ 24	结算(6)

需求说明：

1. 渲染功能
2. 删除功能
3. 修改个数
4. 全选反选
5. 统计 选中的 总价 和 总数量
6. 持久化到本地

实现思路：

1. 基本渲染：v-for遍历、:class动态绑定样式
2. 删除功能：v-on 绑定事件，获取当前行的id
3. 修改个数：v-on绑定事件，获取当前行的id，进行筛选出对应的项然后增加或减少
4. 全选反选
 1. 必须所有的小选框都选中，全选按钮才选中 → every
 2. 如果全选按钮选中，则所有小选框都选中

3. 如果全选取消，则所有小选框都取消选中

声明计算属性，判断数组中的每一个checked属性的值，看是否需要全部选

5.统计 选中的 总价 和 总数量：通过计算属性来计算**选中的**总价和总数量

6.持久化到本地： 在数据变化时都要更新下本地存储 watch

day03

一、今日目标

1.生命周期

1. 生命周期介绍
2. 生命周期的四个阶段
3. 生命周期钩子
4. 声明周期案例

2.综合案例-小黑记账清单

1. 列表渲染
2. 添加/删除
3. 饼图渲染

3.工程化开发入门

1. 工程化开发和脚手架
2. 项目运行流程
3. 组件化
4. 组件注册

4.综合案例-小兔仙首页

1. 拆分模块-局部注册
2. 结构样式完善
3. 拆分组件 – 全局注册

二、Vue生命周期

思考：什么时候可以发送初始化渲染请求？（越早越好）什么时候可以开始操作dom？（至少dom得渲染出来）

Vue生命周期：就是一个Vue实例从创建 到 销毁 的整个过程。

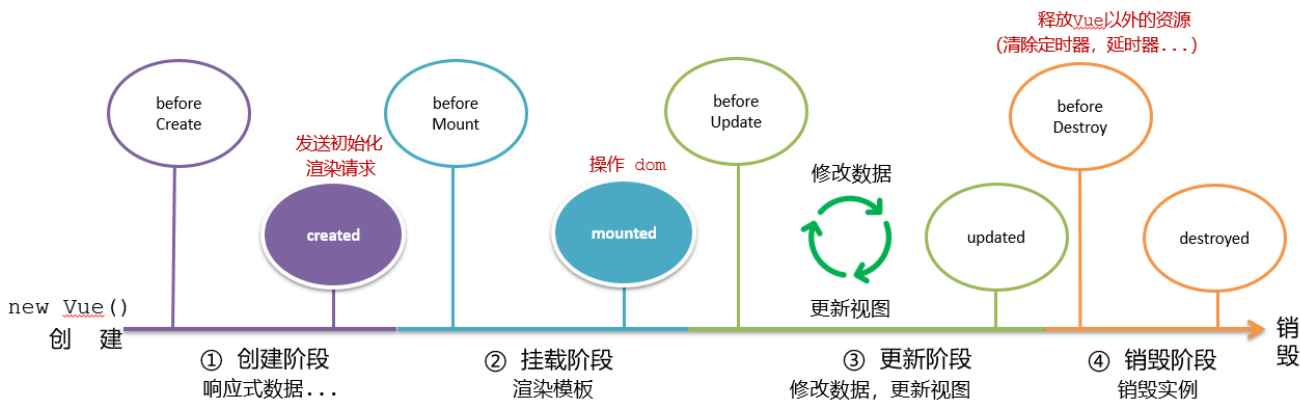
生命周期四个阶段：① 创建 ② 挂载 ③ 更新 ④ 销毁

- 1.创建阶段：创建响应式数据
- 2.挂载阶段：渲染模板
- 3.更新阶段：修改数据，更新视图
- 4.销毁阶段：销毁Vue实例



三、Vue生命周期钩子

Vue生命周期过程中，会自动运行一些函数，被称为【生命周期钩子】→ 让开发者可以在【特定阶段】运行自己的代码



```
<div id="app">
  <h3>{{ title }}</h3>
  <div>
    <button @click="count--"></button>
    <span>{{ count }}</span>
    <button @click="count++"></button>
  </div>
</div>
```

```
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      count: 100,
      title: '计数器'
    },
    // 1. 创建阶段（准备数据）

    // 2. 挂载阶段（渲染模板）

    // 3. 更新阶段（修改数据 → 更新视图）

    // 4. 卸载阶段

  })
</script>
```

四、生命周期钩子小案例

1.在created中发送数据

```
<style>
* {
  margin: 0;
  padding: 0;
  list-style: none;
}
.news {
  display: flex;
  height: 120px;
  width: 600px;
  margin: 0 auto;
  padding: 20px 0;
  cursor: pointer;
}
.news .left {
  flex: 1;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  padding-right: 10px;
}
.news .left .title {
  font-size: 20px;
```



```

    <div class="right">
      
    </div>
  </li>
</ul>
</div>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
<script>
  // 接口地址: http://hmajax.itheima.net/api/news
  // 请求方式: get
  const app = new Vue({
    el: '#app',
    data: {
      list: []
    }
  })
</script>

```

2.在mounted中获取焦点

```

<style>
  html,
  body {
    height: 100%;
  }
  .search-container {
    position: absolute;
    top: 30%;
    left: 50%;
    transform: translate(-50%, -50%);
    text-align: center;
  }
  .search-container .search-box {
    display: flex;
  }
  .search-container img {
    margin-bottom: 30px;
  }
  .search-container .search-box input {
    width: 512px;
    height: 16px;
    padding: 12px 16px;
    font-size: 16px;
    margin: 0;
    vertical-align: top;
    outline: 0;
    box-shadow: none;
    border-radius: 10px 0 0 10px;
    border: 2px solid #c4c7ce;
    background: #fff;
  }

```

```

    color: #222;
    overflow: hidden;
    box-sizing: content-box;
    -webkit-tap-highlight-color: transparent;
  }
  .search-container .search-box button {
    cursor: pointer;
    width: 112px;
    height: 44px;
    line-height: 41px;
    line-height: 42px;
    background-color: #ad2a27;
    border-radius: 0 10px 10px 0;
    font-size: 17px;
    box-shadow: none;
    font-weight: 400;
    border: 0;
    outline: 0;
    letter-spacing: normal;
    color: white;
  }
  body {
    background: no-repeat center /cover;
    background-color: #edf0f5;
  }
</style>

<div class="container" id="app">
  <div class="search-container">
    
    <div class="search-box">
      <input type="text" v-model="words" id="inp">
      <button>搜索一下</button>
    </div>
  </div>
</div>

<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script>
  const app = new Vue({
    el: '#app',
    data: {
      words: ''
    }
  })
</script>

```


五、案例-小黑记账清单

1.需求图示：



2.需求分析

- 1.基本渲染
- 2.添加功能
- 3.删除功能
- 4.饼图渲染

3.思路分析

- 1.基本渲染
 - 立刻发送请求获取数据 created
 - 拿到数据，存到data的响应式数据中
 - 结合数据，进行渲染 v-for
 - 消费统计 —> 计算属性
- 2.添加功能
 - 收集表单数据 v-model，使用指令修饰符处理数据
 - 给添加按钮注册点击事件，对输入的内容做非空判断，发送请求
 - 请求成功后，对文本框内容进行清空
 - 重新渲染列表
- 3.删除功能
 - 注册点击事件，获取当前行的id
 - 根据id发送删除请求
 - 需要重新渲染
- 4.饼图渲染
 - 初始化一个饼图 echarts.init(dom) mounted钩子中渲染
 - 根据数据试试更新饼图 echarts.setOptions({...})

4.代码准备

```
<!-- CSS only -->
<link
  rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
/>
<style>
  .red {
    color: red!important;
  }
  .search {
    width: 300px;
    margin: 20px 0;
  }
  .my-form {
    display: flex;
    margin: 20px 0;
  }
  .my-form input {
    flex: 1;
    margin-right: 20px;
  }
  .table > :not(:first-child) {
    border-top: none;
  }
  .contain {
    display: flex;
    padding: 10px;
  }
  .list-box {
    flex: 1;
    padding: 0 30px;
  }
  .list-box a {
    text-decoration: none;
  }
  .echarts-box {
    width: 600px;
    height: 400px;
    padding: 30px;
    margin: 0 auto;
    border: 1px solid #ccc;
  }
  tfoot {
    font-weight: bold;
  }
  @media screen and (max-width: 1000px) {
    .contain {
      flex-wrap: wrap;
    }
    .list-box {
      width: 100%;
    }
  }
</style>
```

```
    }  
    .echarts-box {  
        margin-top: 30px;  
    }  
}  
</style>
```

```
<div id="app">  
  <div class="contain">  
    <!-- 左侧列表 -->  
    <div class="list-box">  
  
      <!-- 添加资产 -->  
      <form class="my-form">  
        <input type="text" class="form-control" placeholder="消费名称" />  
        <input type="text" class="form-control" placeholder="消费价格" />  
        <button type="button" class="btn btn-primary">添加账单</button>  
      </form>  
  
      <table class="table table-hover">  
        <thead>  
          <tr>  
            <th>编号</th>  
            <th>消费名称</th>  
            <th>消费价格</th>  
            <th>操作</th>  
          </tr>  
        </thead>  
        <tbody>  
          <tr>  
            <td>1</td>  
            <td>帽子</td>  
            <td>99.00</td>  
            <td><a href="javascript:;">删除</a></td>  
          </tr>  
          <tr>  
            <td>2</td>  
            <td>大衣</td>  
            <td class="red">199.00</td>  
            <td><a href="javascript:;">删除</a></td>  
          </tr>  
        </tbody>  
        <tfoot>  
          <tr>  
            <td colspan="4">消费总计: 298.00</td>  
          </tr>  
        </tfoot>  
      </table>  
    </div>  
  
    <!-- 右侧图表 -->  
    <div class="echarts-box" id="main"></div>  
  </div>
```

```

</div>
<script src="https://cdn.jsdelivr.net/npm/echarts@5.4.0/dist/echarts.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
<script>
  /**
   * 接口文档地址:
   * https://www.apifox.cn/apidoc/shared-24459455-ebb1-4fdc-8df8-0aff8dc317a8/api-53371058
   *
   * 功能需求:
   * 1. 基本渲染
   * 2. 添加功能
   * 3. 删除功能
   * 4. 饼图渲染
   */
  const app = new Vue({
    el: '#app',
    data: {

    },
  })
</script>

```

六、工程化开发和脚手架

1. 开发Vue的两种方式

- 核心包传统开发模式：基于html / css / js 文件，直接引入核心包，开发 Vue。
- 工程化开发模式：基于构建工具（例如：webpack）的环境中开发Vue。



工程化开发模式优点：

提高编码效率，比如使用JS新语法、Less/Sass、Typescript等通过webpack都可以编译成浏览器识别的ES3/ES5/CSS等

工程化开发模式问题：

- webpack配置 **不简单**
- **雷同**的基础配置
- 缺乏**统一的标准**

为了解决以上问题，所以我们需要一个工具，生成标准化的配置

2.脚手架Vue CLI

基本介绍：

Vue CLI 是Vue官方提供的一个**全局命令工具**

可以帮助我们**快速创建**一个开发Vue项目的**标准化基础架子**。【集成了webpack配置】

好处：

1. 开箱即用，零配置
2. 内置babel等工具
3. 标准化的webpack配置

使用步骤：

1. 全局安装（只需安装一次即可） yarn global add @vue/cli 或者 npm i @vue/cli -g
2. 查看vue/cli版本： vue --version
3. 创建项目架子：**vue create project-name**(项目名不能使用中文)
4. 启动项目：**yarn serve** 或者 **npm run serve**(命令不固定，找package.json)

七、项目目录介绍和运行流程

1.项目目录介绍

```
| -node_modules 第三包文件夹
```

放html文件的地方

网站图标

index.html 模板文件 ③

源代码目录 → 以后写代码的文件夹

静态资源目录 → 存放图片、字体等

组件目录 → 存放通用组件

App根组件 → 项目运行看到的内容就在这里编写 ②

入口文件 → 打包或运行, 第一个执行的文件 ①

git忽视文件

babel配置文件

js配置文件

项目配置文件 → 包含项目名、版本号、scripts、依赖包等

项目说明文档

vue-cli配置文件

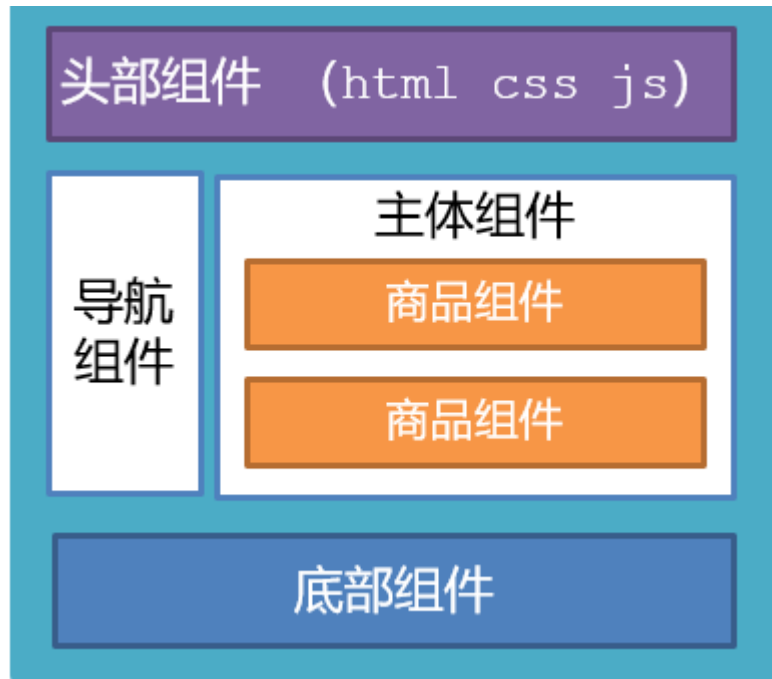
yarn锁文件，由yarn自动生成的，锁定安装版本

1. main.js 入口文件
2. App.vue App根组件
3. index.html 模板文件

main.js 核心代码

1. 导入 Vue
 - `import Vue from 'vue'`
2. 导入 App.vue
 - `import App from './App.vue'`
3. 实例化 Vue, 将 App.vue 渲染到 index.html 容器中
 - `new Vue({`
 - `render: h => h(App),`
 - `}).$mount('#app')`

比如：下面这个页面，可以把所有的代码都写在一个页面中，但是这样显得代码比较混乱，难易维护。咱们可以按模块进行组件划分



总结：

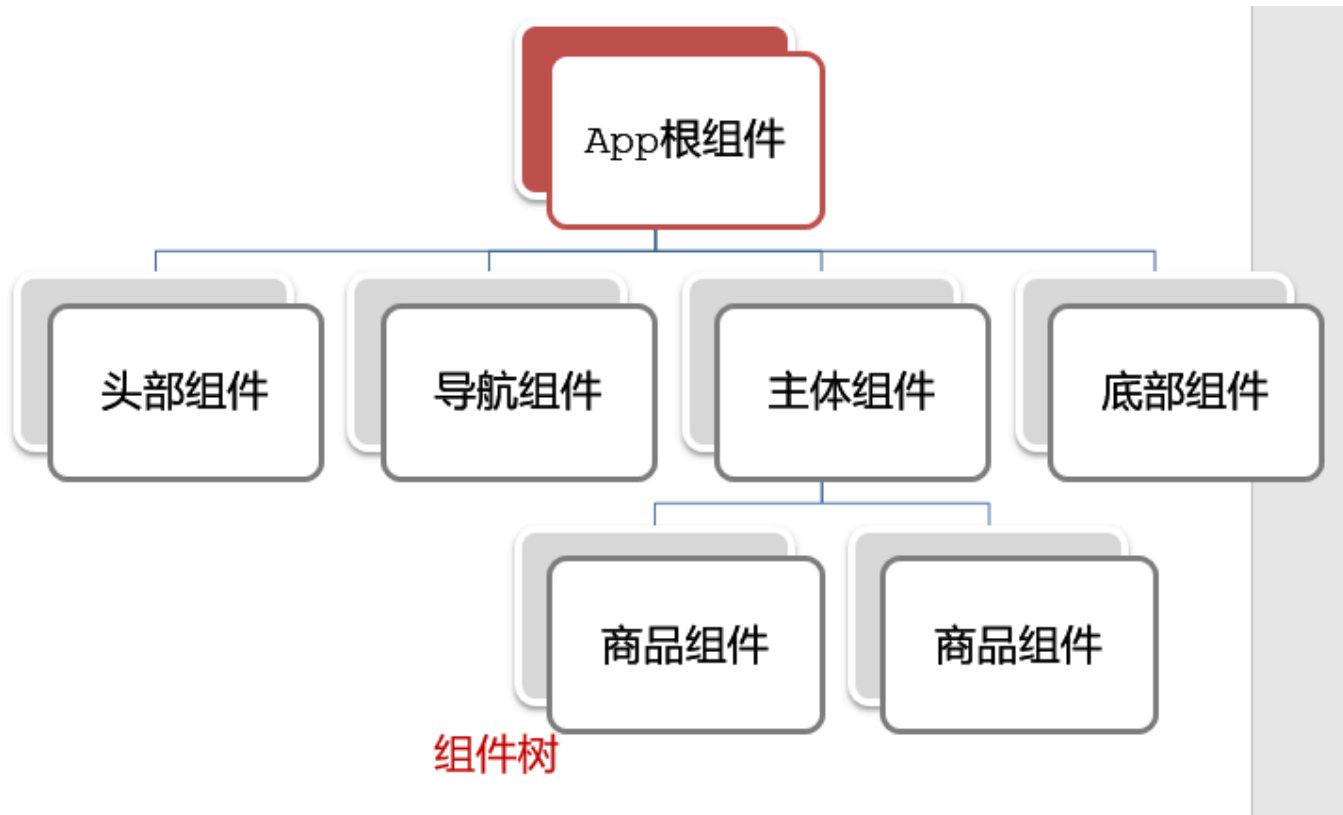
组件化的好处是什么？

组件的分类？

九、根组件 App.vue

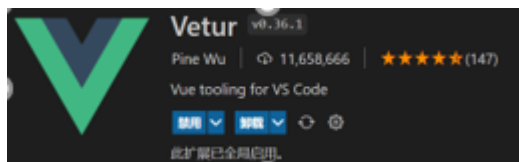
1.根组件介绍

整个应用最上层的组件，包裹所有普通小组件



2. 组件是由三部分构成

- 语法高亮插件



- 三部分构成
 - template: 结构 (有且只能一个根元素)
 - script: js逻辑
 - style: 样式 (可支持less, 需要装包)
- 让组件支持less
 - (1) style标签, lang="less" 开启less功能
 - (2) 装包: yarn add less less-loader -D 或者 npm i less less-loader -D

3. 总结

App组件包含哪三部分?

十、普通组件的注册使用-局部注册

1.特点：

只能在注册的组件内使用

2.步骤：

1. 创建.vue文件（三个组成部分）
2. 在使用的组件内先导入再注册，最后使用

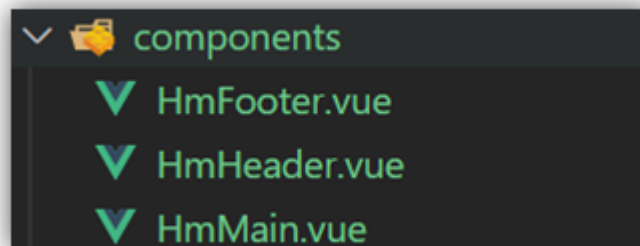
3.使用方式：

当成html标签使用即可 <组件名></组件名>

4.注意：

组件名规范 → 大驼峰命名法，如 HmHeader

5.语法：

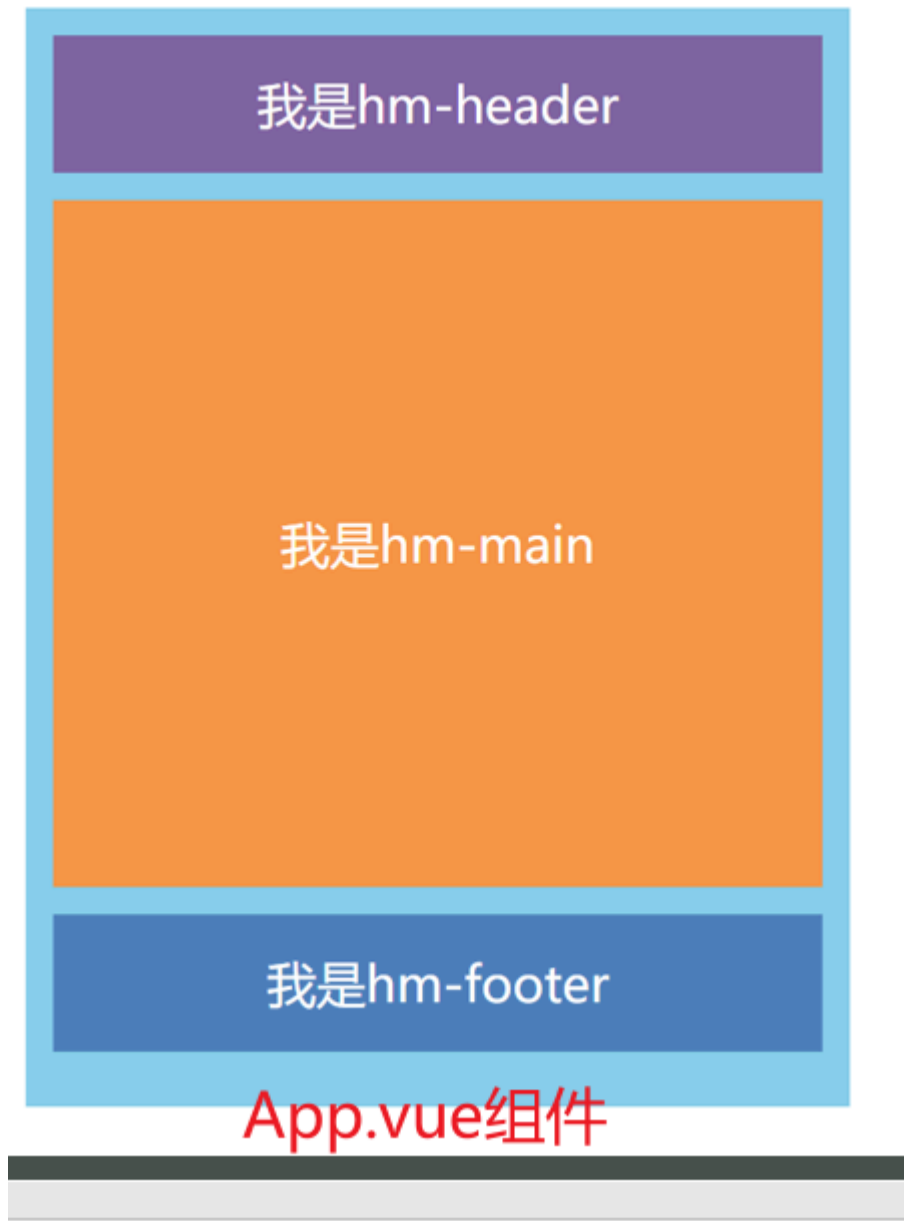


```
// 导入需要注册的组件
import 组件对象 from '.vue文件路径'
import HmHeader from './components/HmHeader'

export default {• // 局部注册
  components: {
    '组件名': 组件对象,
    HmHeader:HmHeaer,
    HmHeader
  }
}
```

6.练习

在App组件中，完成以下练习。在App.vue中使用组件的方式完成下面布局



```
<template>
  <div class="hm-header">
    我是hm-header
  </div>
</template>

<script>
export default {
}
</script>

<style>
.hm-header {
  height: 100px;
  line-height: 100px;
  text-align: center;
  font-size: 30px;
}
```

```
background-color: #8064a2;
color: white;
}
</style>
```

```
<template>
  <div class="hm-main">
    我是hm-main
  </div>
</template>

<script>
export default {

}
</script>

<style>
.hm-main {
  height: 400px;
  line-height: 400px;
  text-align: center;
  font-size: 30px;
  background-color: #f79646;
  color: white;
  margin: 20px 0;
}
</style>
```

```
<template>
  <div class="hm-footer">
    我是hm-footer
  </div>
</template>

<script>
export default {

}
</script>

<style>
.hm-footer {
  height: 100px;
  line-height: 100px;
  text-align: center;
  font-size: 30px;
  background-color: #4f81bd;
  color: white;
}
</style>
```

7.总结

- A组件内部注册的局部组件能在B组件使用吗
- 局部注册组件的步骤是什么
- 使用组件时 应该按照什么命名法

十一、普通组件的注册使用-全局注册

1.特点：

全局注册的组件，在项目的**任何组件**中都能使用

2.步骤

1. 创建.vue组件（三个组成部分）
2. main.js中进行全局注册

3.使用方式

当成HTML标签直接使用

```
<组件名></组件名>
```

4.注意

组件名规范 → 大驼峰命名法，如 HmHeader

5.语法

Vue.component('组件名', 组件对象)

例：

```
// 导入需要全局注册的组件
import HmButton from './components/HmButton'
Vue.component('HmButton', HmButton)
```

6.练习

在以下3个局部组件中展示一个通用按钮



```
<template>
  <button class="hm-button">通用按钮</button>
</template>

<script>
export default {
}
</script>

<style>
.hm-button {
  height: 50px;
  line-height: 50px;
  padding: 0 20px;
  background-color: #3bae56;
  border-radius: 5px;
  color: white;
  border: none;
  vertical-align: middle;
  cursor: pointer;
}
</style>
```

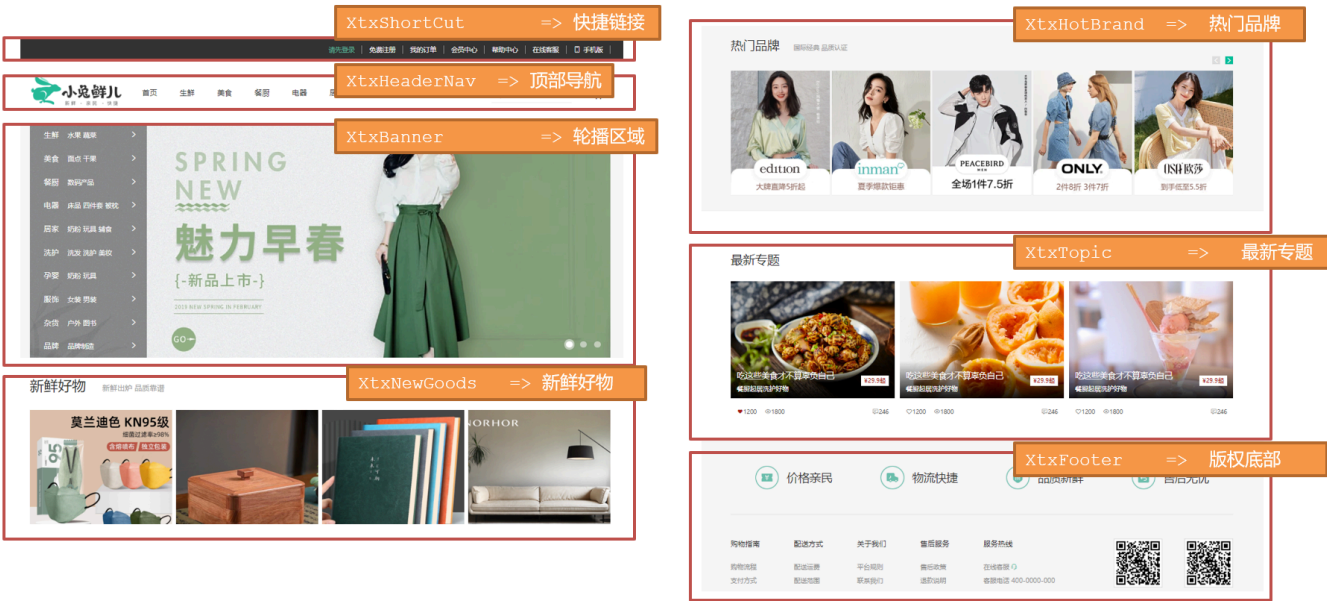
7.总结

- 1.全局注册组件应该在哪个文件中注册以及语法是什么？
- 2.全局组件在项目中的任何一个组件中可不可以使用？

十二、综合案例

1.小兔仙首页启动项目演示

2.小兔仙组件拆分示意图



3.开发思路

- 1. 分析页面，按模块拆分子组件，搭架子 (局部或全局注册)
 - 2. 根据设计图，编写组件 html 结构 css 样式 (已准备好)
 - 3. 拆分封装通用小组件 (局部或全局注册)
- 将来 → 通过 js 动态渲染，实现功能

day04

一、学习目标

1.组件的三大组成部分（结构/样式/逻辑）

scoped解决样式冲突/data是一个函数

2.组件通信

1. 组件通信语法
2. 父传子
3. 子传父
4. 非父子通信（扩展）

3.综合案例：小黑记事本（组件版）

1. 拆分组件
2. 列表渲染
3. 数据添加
4. 数据删除
5. 列表统计
6. 清空
7. 持久化

4.进阶语法

1. v-model原理
2. v-model应用于组件
3. sync修饰符
4. ref和\$refs
5. \$nextTick

二、scoped解决样式冲突

1.默认情况：

写在组件中的样式会 **全局生效** → 因此很容易造成多个组件之间的样式冲突问题。

1. **全局样式**: 默认组件中的样式会作用到全局，任何一个组件中都会受到此样式的影响
2. **局部样式**: 可以给组件加上**scoped** 属性,可以**让样式只作用于当前组件**

2.代码演示

BaseOne.vue

```
<template>
  <div class="base-one">
    BaseOne
  </div>
</template>

<script>
export default {

}
</script>
<style scoped>
</style>
```

BaseTwo.vue

```
<template>
  <div class="base-one">
    BaseTwo
  </div>
</template>

<script>
export default {

}
</script>

<style scoped>
</style>
```

App.vue

```
<template>
  <div id="app">
    <BaseOne></BaseOne>
    <BaseTwo></BaseTwo>
  </div>
</template>
```



```
<script>
import BaseOne from './components/BaseOne'
import BaseTwo from './components/BaseTwo'
export default {
  name: 'App',
  components: {
    BaseOne,
    BaseTwo
  }
}
</script>
```

3.scoped原理

1. 当前组件内标签都被添加data-v-hash值 的属性
2. css选择器都被添加 [data-v-hash值] 的属性选择器

最终效果: 必须是当前组件的元素, 才会有这个自定义属性, 才会被这个样式作用到

```
<div data-v-cbe7c9bc == $0>
  <p data-v-cbe7c9bc>我是hm-header</p>
</div>
<div>我是普通盒子</div>
```

```
div[data-v-cbe7c9bc] {
  border: 1px solid #000;
  margin: 10px 0;
}
```

4.总结

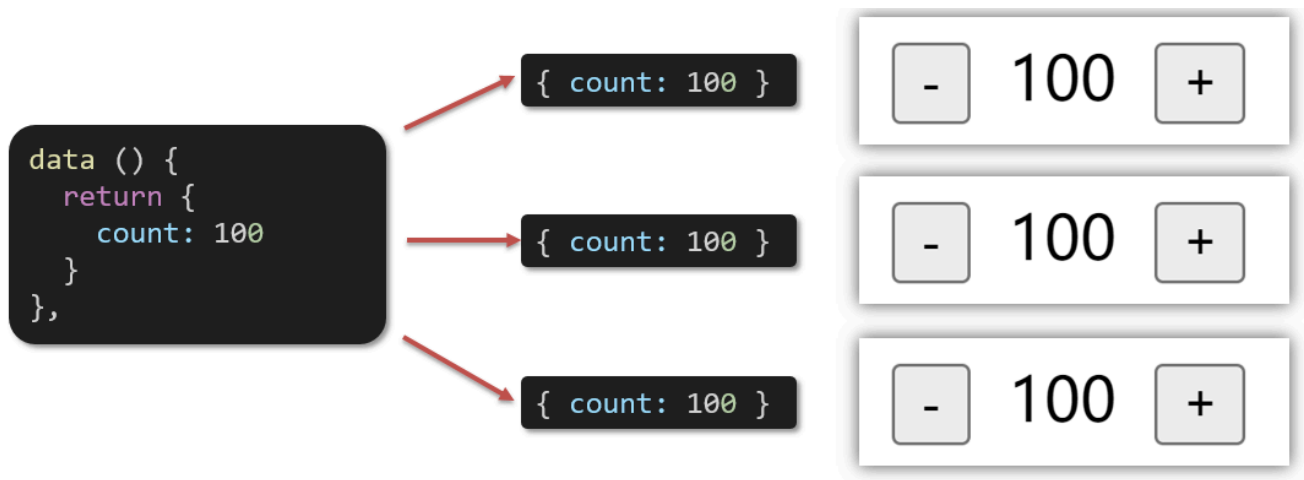
1. style的默认样式是作用到哪儿的?
2. scoped的作用是什么?
3. style中推不推荐加scoped?

三、data必须是一个函数

1、data为什么要写成函数

一个组件的 data 选项必须是一个函数。目的是为了：保证每个组件实例，维护独立的一份数据对象。

每次创建新的组件实例，都会新执行一次data 函数，得到一个新对象。



2. 代码演示

BaseCount.vue

```
<template>  
  <div class="base-count">  
    <button @click="count--">-</button>  
    <span>{{ count }}</span>  
    <button @click="count++">+</button>  
  </div>  
</template>  
  
<script>  
export default {  
  data: function () {  
    return {  
      count: 100,  
    }  
  },  
},  
</script>  
  
<style>  
.base-count {  
  margin: 20px;  
}</style>
```

App.vue

```
<template>  
  <div class="app">  
    <BaseCount></BaseCount>  
  </div>  
</template>  
  
<script>  
import BaseCount from './components/BaseCount'  
export default {
```

```
components: {  
  BaseCount,  
},  
}  
</script>  
  
<style>  
</style>
```

3.总结

data写成函数的目的是什么？

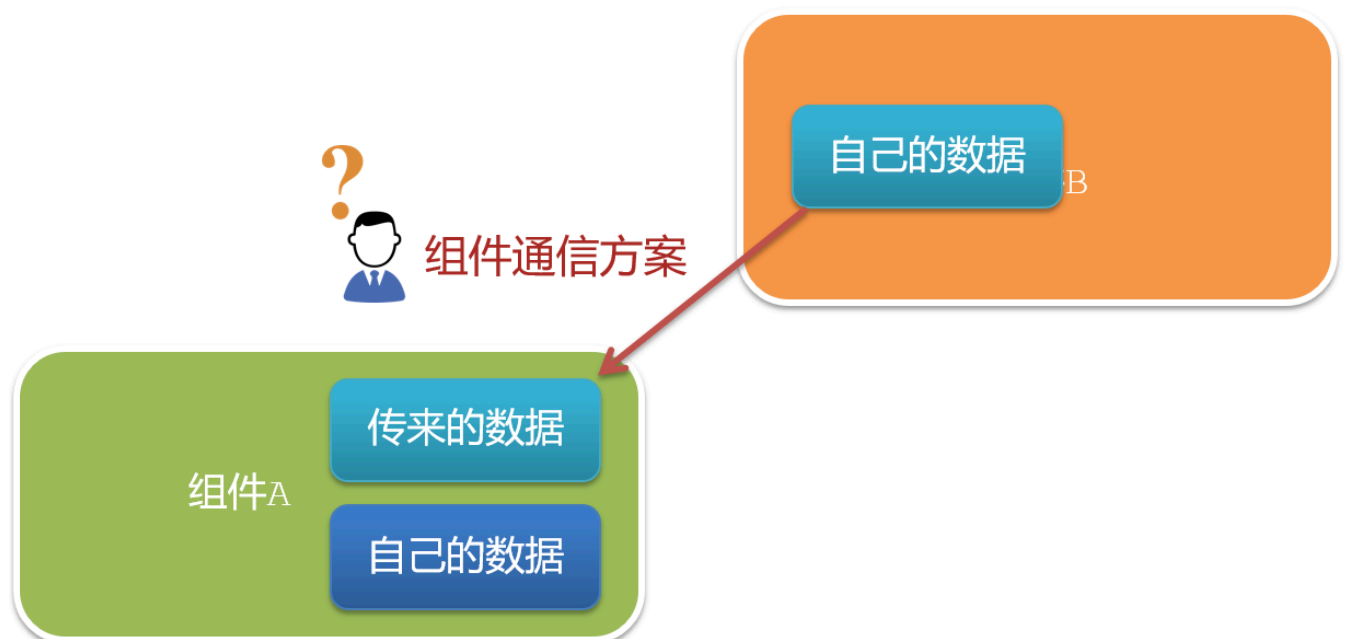
四、组件通信

1.什么是组件通信？

组件通信，就是指**组件与组件之间的数据传递**

- 组件的数据是独立的，无法直接访问其他组件的数据。
- 想使用其他组件的数据，就需要组件通信

2.组件之间如何通信

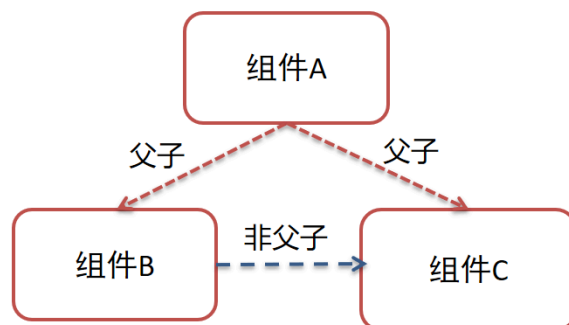
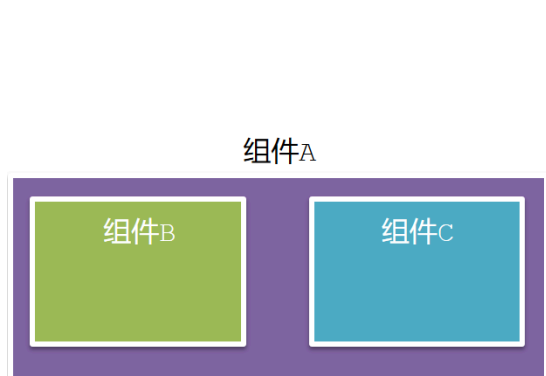


思考：

1. 组件之间有哪些关系？
2. 对应的组件通信方案有哪几类？

3.组件关系分类

1. 父子关系
2. 非父子关系



4.通信解决方案

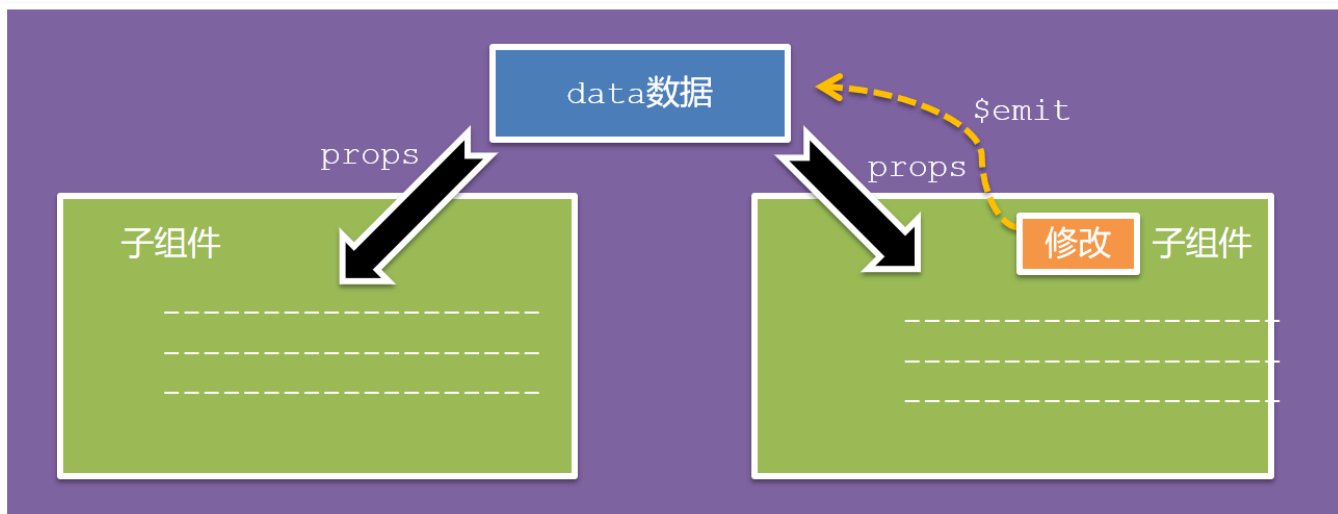


通用解决方案: Vuex (适合复杂业务场景)

5.父子通信流程

1. 父组件通过 `props` 将数据传递给子组件
2. 子组件利用 `$emit` 通知父组件修改更新

父组件



6.父向子通信代码示例

父组件通过props将数据传递给子组件

父组件App.vue

```
<template>
  <div class="app" style="border: 3px solid #000; margin: 10px">
    我是APP组件
    <Son></Son>
  </div>
</template>

<script>
import Son from './components/Son.vue'
export default {
  name: 'App',
  data() {
    return {
      myTitle: '学前端，就来黑马程序员',
    }
  },
  components: {
    Son,
  },
}
</script>

<style>
</style>
```

子组件Son.vue

```
<template>
```

```

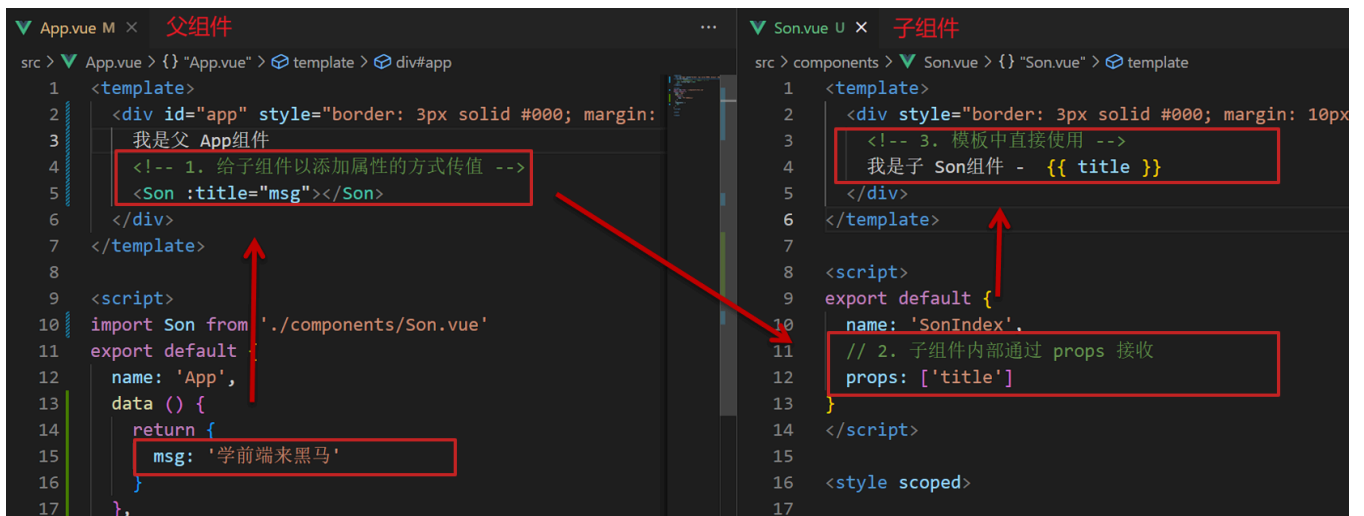
<div class="son" style="border:3px solid #000;margin:10px">
  我是Son组件
</div>
</template>

<script>
export default {
  name: 'Son-Child',
}
</script>

<style>

</style>

```

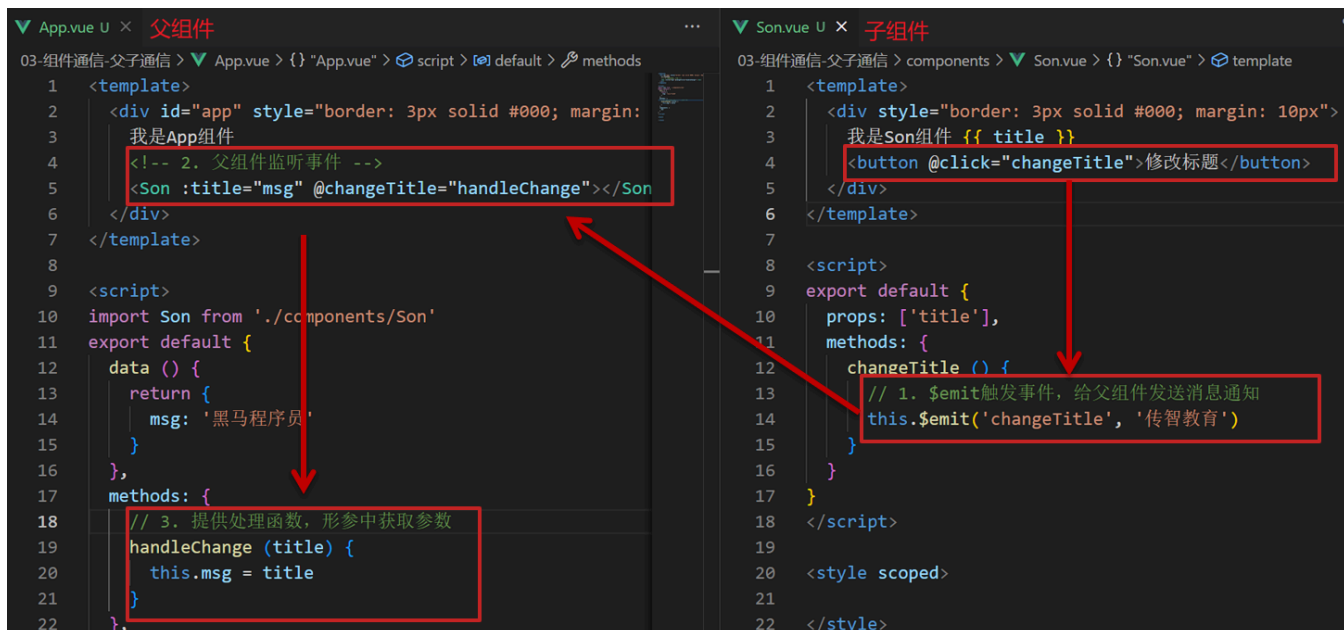


父向子传值步骤

1. 给子组件以添加属性的方式传值
2. 子组件内部通过props接收
3. 模板中直接使用 props接收的值

7.子向父通信代码示例

子组件利用 `$emit` 通知父组件，进行修改更新



子向父传值步骤

1. \$emit触发事件，给父组件发送消息通知
2. 父组件监听\$emit触发的事件
3. 提供处理函数，在函数的形参中获取传过来的参数

8.总结

1. 组件关系分类有哪两种
2. 父子组件通信的流程是什么？
 1. 父向子
 2. 子向父

五、什么是props

1.Props 定义

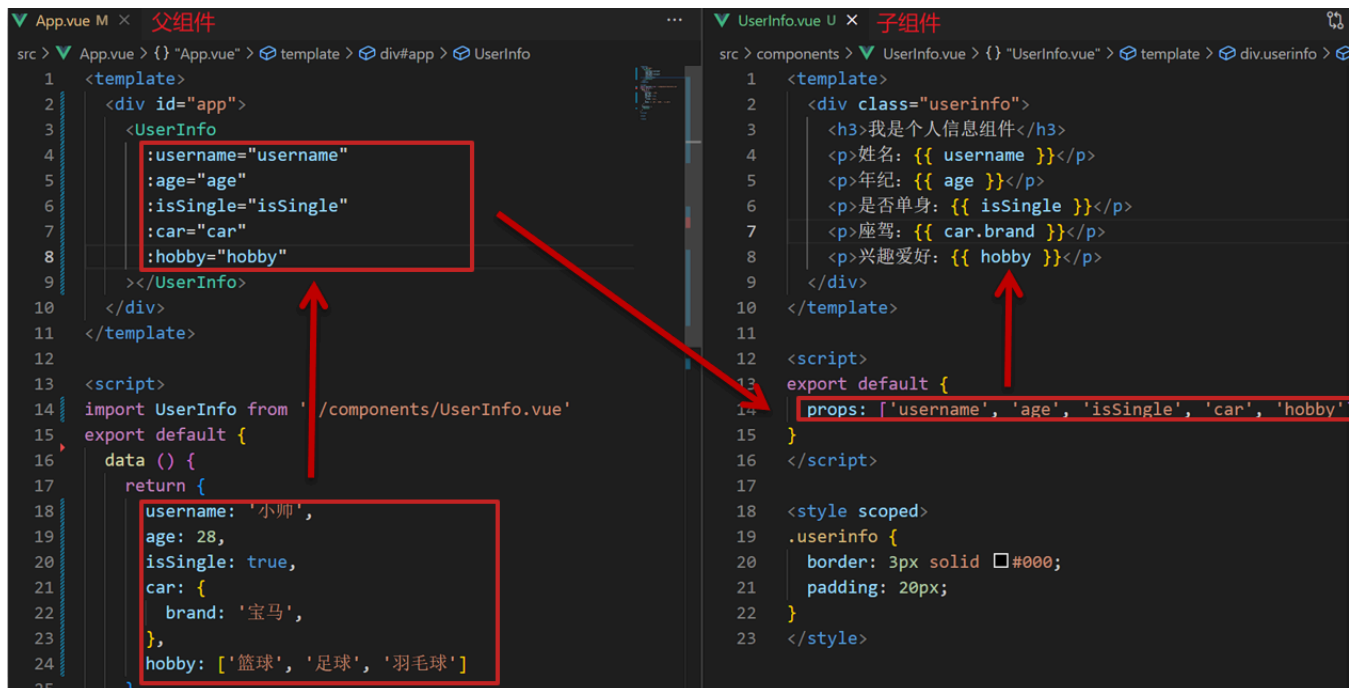
组件上注册的一些 自定义属性

2.Props 作用

向子组件传递数据

3.特点

1. 可以传递 **任意数量** 的prop
2. 可以传递 **任意类型** 的prop



4.代码演示

父组件App.vue

```
<template>
  <div class="app">
    <UserInfo
      :username="username"
      :age="age"
      :isSingle="isSingle"
      :car="car"
      :hobby="hobby"
    ></UserInfo>
  </div>
</template>

<script>
import UserInfo from './components/UserInfo.vue'
export default {
  data() {
    return {
      username: '小帅',
      age: 28,
      isSingle: true,
      car: {
        brand: '宝马',
      },
      hobby: ['篮球', '足球', '羽毛球'],
    }
  },
  components: {
    UserInfo,
  },
}
```



```
</script>
```

```
<style>  
</style>
```

子组件UserInfo.vue

```
<template>  
  <div class="userinfo">  
    <h3>我是个人信息组件</h3>  
    <div>姓名: </div>  
    <div>年龄: </div>  
    <div>是否单身: </div>  
    <div>座驾: </div>  
    <div>兴趣爱好: </div>  
  </div>  
</template>  
  
<script>  
export default {  
  
}  
</script>  
  
<style>  
.userinfo {  
  width: 300px;  
  border: 3px solid #000;  
  padding: 20px;  
}  
.userinfo > div {  
  margin: 20px 10px;  
}  
</style>
```

六、props校验

1.思考

组件的props可以乱传吗

2.作用

为组件的 prop 指定**验证要求**，不符合要求，控制台就会有**错误提示** → 帮助开发者，快速发现错误

3.语法

- 类型校验
- 非空校验
- 默认值
- 自定义校验

```
props: {  
  校验的属性名: 类型 // Number String Boolean ...  
},
```

4.代码演示

App.vue

```
<template>  
  <div class="app">  
    <BaseProgress :w="width"></BaseProgress>  
  </div>  
</template>  
  
<script>  
import BaseProgress from './components/BaseProgress.vue'  
export default {  
  data() {  
    return {  
      width: 30,  
    }  
  },  
  components: {  
    BaseProgress,  
  },  
}  
</script>  
  
<style>  
</style>
```

BaseProgress.vue

```
<template>  
  <div class="base-progress">  
    <div class="inner" :style="{ width: w + '%' }">  
      <span>{{ w }}%</span>  
    </div>  
  </div>  
</template>  
  
<script>  
export default {
```

```
    props: ['w'],
  }
</script>

<style scoped>
.base-progress {
  height: 26px;
  width: 400px;
  border-radius: 15px;
  background-color: #272425;
  border: 3px solid #272425;
  box-sizing: border-box;
  margin-bottom: 30px;
}
.inner {
  position: relative;
  background: #379bff;
  border-radius: 15px;
  height: 25px;
  box-sizing: border-box;
  left: -3px;
  top: -2px;
}
.inner span {
  position: absolute;
  right: 0;
  top: 26px;
}
</style>
```

七、props校验完整写法

1.语法

```
props: {
  校验的属性名: {
    type: 类型, // Number String Boolean ...
    required: true, // 是否必填
    default: 默认值, // 默认值
    validator (value) {
      // 自定义校验逻辑
      return 是否通过校验
    }
  }
},
```

2.代码实例

```
<script>
export default {
  // 完整写法（类型、默认值、非空、自定义校验）
  props: {
    w: {
      type: Number,
      //required: true,
      default: 0,
      validator(val) {
        // console.log(val)
        if (val >= 100 || val <= 0) {
          console.error('传入的范围必须是0-100之间')
          return false
        } else {
          return true
        }
      },
    },
  },
},
</script>
```

3.注意

- 1.default和required一般不同时写（因为当时必填项时，肯定是有值的）
- 2.default后面如果是简单类型的值，可以直接写默认。如果是复杂类型的值，则需要以函数的形式return一个默认值

八、 props&data、 单向数据流

1.共同点

都可以给组件提供数据

2.区别

- data 的数据是**自己的** → 随便改
- prop 的数据是**外部的** → 不能直接改，要遵循 **单向数据流**

3.单向数据流：

父级props 的数据更新，会向下流动，影响子组件。这个数据流动是单向的

4.代码演示

App.vue

```
<template>
  <div class="app">
    <BaseCount></BaseCount>
  </div>
</template>

<script>
import BaseCount from './components/BaseCount.vue'
export default {
  components:{
    BaseCount
  },
  data(){
  },
}
</script>

<style>

</style>
```

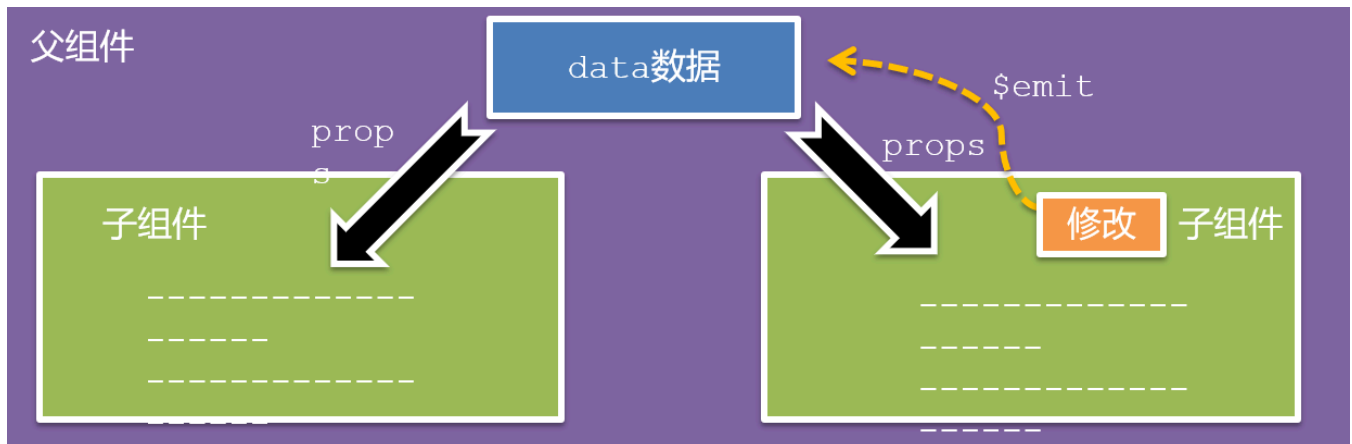
BaseCount.vue

```
<template>
  <div class="base-count">
    <button @click="count--">-</button>
    <span>{{ count }}</span>
    <button @click="count++">+</button>
  </div>
</template>

<script>
export default {
  // 1.自己的数据随便修改 （谁的数据 谁负责）
  data () {
    return {
      count: 100,
    }
  },
  // 2.外部传过来的数据 不能随便修改
  //props: {
  //  count: {
  //    type: Number,
  //  },
  //}
}
</script>

<style>
.base-count {
```

```
margin: 20px;
}
</style>
```



5. 口诀

谁的数据谁负责

九、综合案例-组件拆分

1. 需求说明

- 拆分基础组件
- 渲染待办任务
- 添加任务
- 删除任务
- 底部合计 和 清空功能
- 持久化存储

2. 拆分基础组件

咱们可以把小黑记事本原有的结构拆成三部分内容：头部（TodoHeader）、列表(TodoMain)、底部(TodoFooter)



十、综合案例-列表渲染

思路分析：

1. 提供数据：提供在公共的父组件 App.vue
2. 通过父传子，将数据传递给TodoMain
3. 利用v-for进行渲染

十一、综合案例-添加功能

思路分析：

1. 收集表单数据 v-model
2. 监听时间（回车+点击 都要进行添加）
3. 子传父，将任务名称传递给父组件App.vue
4. 父组件接受到数据后 进行添加 unshift(自己的数据自己负责)

十二、综合案例-删除功能

思路分析：

1. 监听时间（监听删除的点击）携带id
2. 子传父，将删除的id传递给父组件App.vue
3. 进行删除 **filter** (自己的数据自己负责)

十三、综合案例-底部功能及持久化存储

思路分析：

1. 底部合计：父组件传递list到底部组件 →展示合计
2. 清空功能：监听事件 → **子组件**通知父组件 →父组件清空
3. 持久化存储：watch监听数据变化，持久化到本地

十四、非父子通信-event bus 事件总线

1.作用

非父子组件之间，进行简易消息传递。(复杂场景→Vuex)

2.步骤

1. 创建一个都能访问的事件总线（空Vue实例）

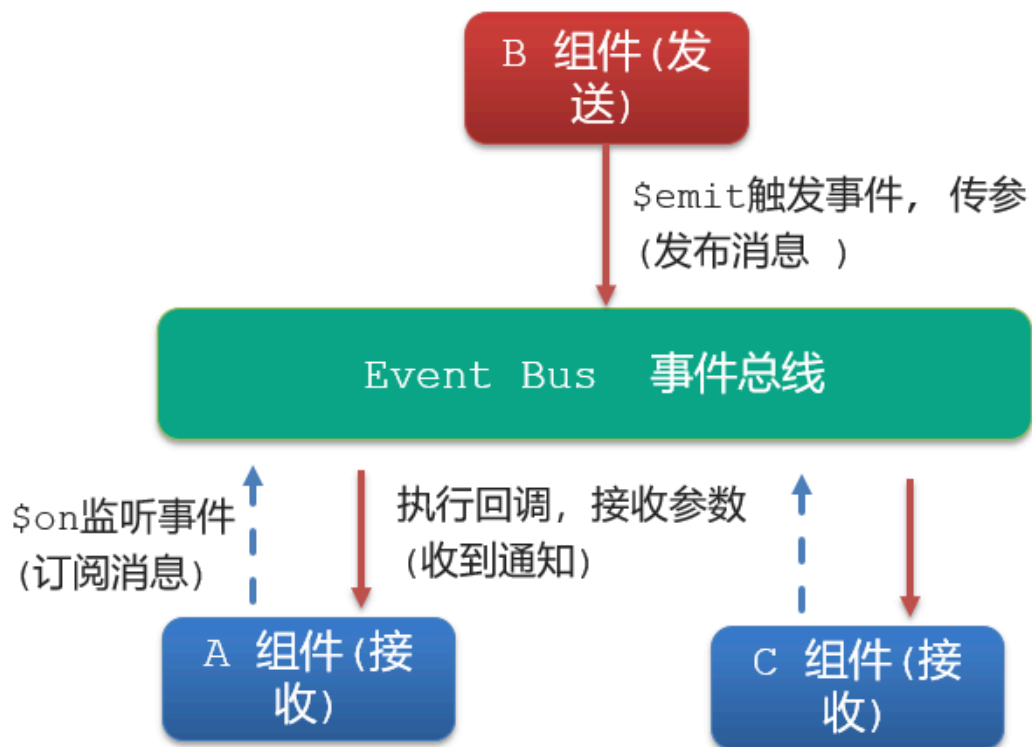
```
import Vue from 'vue'
const Bus = new Vue()
export default Bus
```

2. A组件（接受方），监听Bus的 \$on事件

```
created () {
  Bus.$on('sendMsg', (msg) => {
    this.msg = msg
  })
}
```

3. B组件（发送方），触发Bus的\$emit事件

```
Bus.$emit('sendMsg', '这是一个消息')
```

3.代码示例

EventBus.js

```
import Vue from 'vue'
const Bus = new Vue()
export default Bus
```

BaseA.vue(接受方)

```
<template>
  <div class="base-a">
    我是A组件（接收方）
    <p>{{msg}}</p>
  </div>
</template>

<script>
import Bus from '../utils/EventBus'
export default {
  data() {
    return {
      msg: '',
    },
  },
}
</script>

<style scoped>
```

```
.base-a {  
  width: 200px;  
  height: 200px;  
  border: 3px solid #000;  
  border-radius: 3px;  
  margin: 10px;  
}  
  
</style>
```

BaseB.vue(发送方)

```
<template>  
  <div class="base-b">  
    <div>我是B组件（发布方）</div>  
    <button>发送消息</button>  
  </div>  
</template>  
  
<script>  
import Bus from '../utils/EventBus'  
export default {  
}  
</script>  
  
<style scoped>  
.base-b {  
  width: 200px;  
  height: 200px;  
  border: 3px solid #000;  
  border-radius: 3px;  
  margin: 10px;  
}  
</style>
```

App.vue

```
<template>  
  <div class="app">  
    <BaseA></BaseA>  
    <BaseB></BaseB>  
  </div>  
</template>  
  
<script>  
import BaseA from './components/BaseA.vue'  
import BaseB from './components/BaseB.vue'  
export default {  
  components:{  
    BaseA,  
    BaseB  
  }  
}  
</script>
```

```
<style>  
  
</style>
```

4.总结

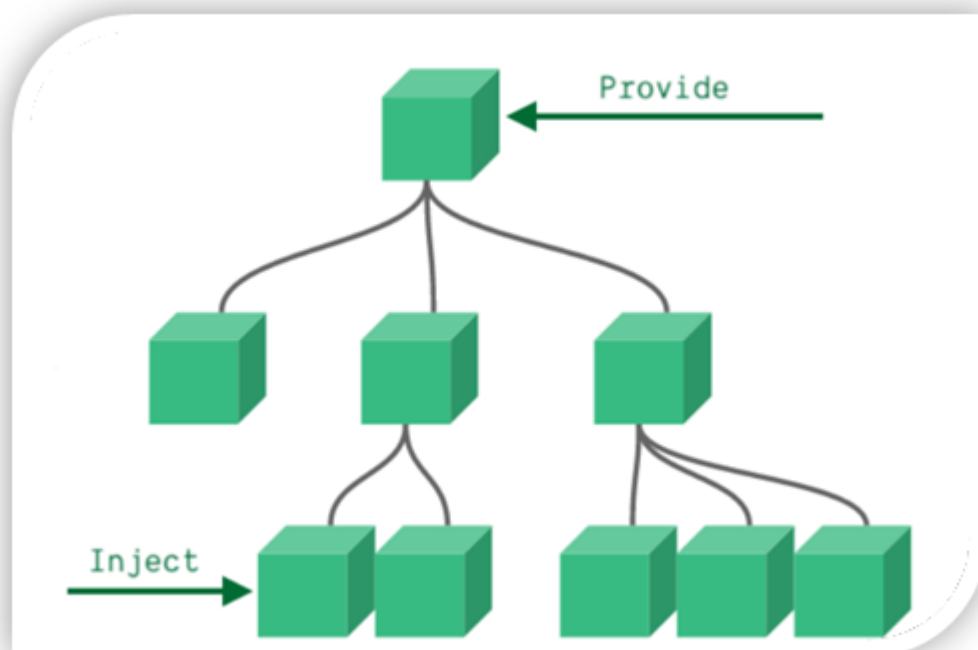
- 1.非父子组件传值借助什么？
- 2.什么是事件总线
- 3.发送方应该调用事件总线的哪个方法
- 4.接收方应该调用事件总线的哪个方法
- 5.一个组件发送数据，可不可以被多个组件接收

十五、非父子通信-provide&inject

1.作用

跨层级共享数据

2.场景



3.语法

1. 父组件 provide提供数据

```
export default {
  provide () {
    return {
      // 普通类型【非响应式】
      color: this.color,
      // 复杂类型【响应式】
      userInfo: this.userInfo,
    }
  }
}
```

2.子/孙组件 inject获取数据

```
export default {
  inject: ['color', 'userInfo'],
  created () {
    console.log(this.color, this.userInfo)
  }
}
```

4.注意

- provide提供的简单类型的数据不是响应式的，复杂类型数据是响应式。（推荐提供复杂类型数据）
- 子/孙组件通过inject获取的数据，不能在自身组件内修改

十六、v-model原理

1.原理：

v-model本质上是一个语法糖。例如应用在输入框上，就是value属性和 input事件 的合写

```
<template>
  <div id="app" >
    <input v-model="msg" type="text">

    <input :value="msg" @input="msg = $event.target.value" type="text">
  </div>
</template>
```

2.作用：

提供数据的双向绑定

- 数据变，视图跟着变 :value
- 视图变，数据跟着变 @input

3.注意

\$event 用于在模板中，获取事件的形参

4.代码示例

```
<template>
  <div class="app">
    <input type="text" />
    <br />
    <input type="text" />
  </div>
</template>

<script>
export default {
  data() {
    return {
      msg1: '',
      msg2: '',
    }
  },
}
</script>
<style>
</style>
```

5.v-model使用在其他表单元素上的原理

不同的表单元素， v-model在底层的处理机制是不一样的。比如给checkbox使用v-model

底层处理的是 checked属性和change事件。

不过咱们只需要掌握应用在文本框上的原理即可

十七、表单类组件封装

1.需求目标

实现子组件和父组件数据的双向绑定（实现App.vue中的selectId和子组件选中的数据进行双向绑定）

2.代码演示

App.vue

```
<template>
  <div class="app">
    <BaseSelect></BaseSelect>
  </div>
</template>

<script>
import BaseSelect from './components/BaseSelect.vue'
export default {
  data() {
    return {
      selectId: '102',
    }
  },
  components: {
    BaseSelect,
  },
}
</script>

<style>
</style>
```

BaseSelect.vue

```
<template>
  <div>
    <select>
      <option value="101">北京</option>
      <option value="102">上海</option>
      <option value="103">武汉</option>
      <option value="104">广州</option>
      <option value="105">深圳</option>
    </select>
  </div>
</template>

<script>
export default {
}
</script>
```

```
<style>
</style>
```

十八、v-model简化代码

1.目标：

父组件通过v-model 简化代码，实现子组件和父组件数据 双向绑定

2.如何简化：

v-model其实就是 :value和@input事件的简写

- 子组件：props通过value接收数据，事件触发 input
- 父组件：v-model直接绑定数据

3.代码示例

子组件

```
<select :value="value" @change="handleChange">...</select>
props: {
  value: String
},
methods: {
  handleChange (e) {
    this.$emit('input', e.target.value)
  }
}
```

父组件

```
<BaseSelect v-model="selectId"></BaseSelect>
```

十九、.sync修饰符

1.作用

可以实现 子组件 与 父组件数据 的 双向绑定，简化代码

简单理解：子组件可以修改父组件传过来的props值

2.场景

封装弹框类的基础组件， visible属性 true显示 false隐藏

3.本质

.sync修饰符 就是 :属性名 和 @update:属性名 合写

4.语法

父组件

```
// .sync写法
<BaseDialog :visible.sync="isShow" />

-----

//完整写法
<BaseDialog
  :visible="isShow"
  @update:visible="isShow = $event"
/>
```

子组件

```
props: {
  visible: Boolean
},

this.$emit('update:visible', false)
```

5.代码示例

App.vue

```
<template>
  <div class="app">
    <button @click="openDialog">退出按钮</button>
    <BaseDialog :isShow="isShow"></BaseDialog>
  </div>
</template>

<script>
import BaseDialog from './components/BaseDialog.vue'
export default {
  data() {
    return {
      isShow: false,
    }
  },
  components: {
    BaseDialog,
  },
}
</script>
```



```
<style>
</style>
```

BaseDialog.vue

```
<template>
  <div class="base-dialog-wrap" v-show="isShow">
    <div class="base-dialog">
      <div class="title">
        <h3>温馨提示: </h3>
        <button class="close">x</button>
      </div>
      <div class="content">
        <p>你确认要退出本系统么? </p>
      </div>
      <div class="footer">
        <button>确认</button>
        <button>取消</button>
      </div>
    </div>
  </div>
</template>
```

```
<script>
export default {
  props: {
    isShow: Boolean,
  }
}
</script>
```

```
<style scoped>
.base-dialog-wrap {
  width: 300px;
  height: 200px;
  box-shadow: 2px 2px 2px 2px #ccc;
  position: fixed;
  left: 50%;
  top: 50%;
  transform: translate(-50%, -50%);
  padding: 0 10px;
}
.base-dialog .title {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 2px solid #000;
}
.base-dialog .content {
  margin-top: 38px;
}
.base-dialog .title .close {
  width: 20px;
```

```
height: 20px;
cursor: pointer;
line-height: 10px;
}
.footer {
  display: flex;
  justify-content: flex-end;
  margin-top: 26px;
}
.footer button {
  width: 80px;
  height: 40px;
}
.footer button:nth-child(1) {
  margin-right: 10px;
  cursor: pointer;
}
</style>
```

6.总结

- 1.父组件如果想让子组件修改传过去的值 必须加什么修饰符?
- 2.子组件要修改父组件的props值 必须使用什么语法?

二十、ref和\$refs

1.作用

利用ref 和 \$refs 可以用于 获取 dom 元素 或 组件实例

2.特点:

查找范围 → 当前组件内(更精确稳定)

3.语法

- 1.给要获取的盒子添加ref属性

```
<div ref="chartRef">我是渲染图表的容器</div>
```

- 2.获取时通过 \$refs获取 this.\$refs.chartRef 获取

```
mounted () {
  console.log(this.$refs.chartRef)
}
```

4.注意

之前只用`document.querySelector('.box')` 获取的是整个页面中的盒子

5.代码示例

App.vue

```
<template>
  <div class="app">
    <BaseChart></BaseChart>
  </div>
</template>

<script>
import BaseChart from './components/BaseChart.vue'
export default {
  components:{
    BaseChart
  }
}
</script>

<style>
</style>
```

BaseChart.vue

```
<template>
  <div class="base-chart-box" ref="baseChartBox">子组件</div>
</template>

<script>
// yarn add echarts 或者 npm i echarts
import * as echarts from 'echarts'

export default {
  mounted() {
    // 基于准备好的dom，初始化echarts实例
    var myChart = echarts.init(document.querySelector('.base-chart-box'))
    // 绘制图表
    myChart.setOption({
      title: {
        text: 'ECharts 入门示例',
      },
      tooltip: {},
      xAxis: {
        data: ['衬衫', '羊毛衫', '雪纺衫', '裤子', '高跟鞋', '袜子'],
      },
      yAxis: {},
      series: [
        {

```

```
      name: '销量',
      type: 'bar',
      data: [5, 20, 36, 10, 10, 20],
    },
  ],
})
},
}
</script>

<style scoped>
.base-chart-box {
  width: 400px;
  height: 300px;
  border: 3px solid #000;
  border-radius: 6px;
}
</style>
```

二十一、异步更新 & \$nextTick

1.需求

编辑标题, 编辑框自动聚焦

1. 点击编辑, 显示编辑框
2. 让编辑框, 立刻获取焦点



2.代码实现

```
<template>
  <div class="app">
    <div v-if="isShowEdit">
      <input type="text" v-model="editValue" ref="inp" />
      <button>确认</button>
    </div>
    <div v-else>
      <span>{{ title }}</span>
      <button @click="editFn">编辑</button>
    </div>
  </div>
</template>

<script>
export default {
  data() {
    return {
      title: '大标题',
      isShowEdit: false,
      editValue: '',
    }
  },
  methods: {
```

```
editFn() {  
  // 显示输入框  
  this.isShowEdit = true  
  // 获取焦点  
  this.$refs.inp.focus()  
} },  
}  
</script>
```

3.问题

"显示之后", 立刻获取焦点是不能成功的!

原因: Vue 是异步更新DOM (提升性能)

4.解决方案

\$nextTick: 等 **DOM更新后**, 才会触发执行此方法里的函数体

语法: this.\$nextTick(函数体)

```
this.$nextTick(() => {  
  this.$refs.inp.focus()  
})
```

注意: \$nextTick 内的函数体 一定是**箭头函数**, 这样才能让函数内部的this指向Vue实例

day05

一、学习目标

1.自定义指令

- 基本语法（全局、局部注册）
- 指令的值
- v-loading的指令封装

2.插槽

- 默认插槽
- 具名插槽
- 作用域插槽

3.综合案例：商品列表

- MyTag组件封装
- MyTable组件封装

4.路由入门

- 单页应用程序
- 路由
- VueRouter的基本使用

二、自定义指令

1.指令介绍

- 内置指令：**v-html**、**v-if**、**v-bind**、**v-on**... 这都是Vue给咱们内置的一些指令，可以直接使用
- 自定义指令：同时Vue也支持让开发者，自己注册一些指令。这些指令被称为 **自定义指令**
每个指令都有自己各自独立的功能

2.自定义指令

概念：自己定义的指令，可以**封装一些DOM操作**，扩展额外的功能

3.自定义指令语法

- 全局注册

```
//在main.js中
Vue.directive('指令名', {
  "inserted" (el) {
    // 可以对 el 标签，扩展额外功能
    el.focus()
  }
})
```

- 局部注册

```
//在Vue组件的配置项中
directives: {
  "指令名": {
    inserted () {
      // 可以对 el 标签，扩展额外功能
      el.focus()
    }
  }
}
```

- 使用指令

注意：在使用指令的时候，一定要**先注册，再使用**，否则会报错

使用指令语法：v-指令名。如：

注册指令时不用加v-前缀，但使用时一定要加v-前缀

4.指令中的配置项介绍

inserted:被绑定元素插入父节点时调用的钩子函数

el：使用指令的那个DOM元素

5.代码示例

需求：当页面加载时，让元素获取焦点（**autofocus在safari浏览器有兼容性**）

App.vue

```
<div>
  <h1>自定义指令</h1>
  <input v-focus ref="inp" type="text">
</div>
```


6.总结

- 1.自定义指令的作用是什么？
- 2.使用自定义指令的步骤是哪两步？

三、自定义指令-指令的值

1.需求

实现一个 color 指令 - 传入不同的颜色, 给标签设置文字颜色

2.语法

- 1.在绑定指令时, 可以通过“等号”的形式为指令 绑定 具体的参数值

```
<div v-color="color">我是内容</div>
```

- 2.通过 binding.value 可以拿到指令值, **指令值修改会 触发 update 函数**

```
directives: {  
  color: {  
    inserted (el, binding) {  
      el.style.color = binding.value  
    },  
    update (el, binding) {  
      el.style.color = binding.value  
    }  
  }  
}
```

3.代码示例

App.vue

```
<template>  
  <div>  
    <!--显示红色-->  
    <h2 v-color="color1">指令的值1测试</h2>  
    <!--显示蓝色-->  
    <h2 v-color="color2">指令的值2测试</h2>  
    <button>  
      改变第一个h1的颜色  
    </button>  
  </div>  
</template>  
  
<script>  
export default {  
  data () {
```

```
    return {  
      color1: 'red',  
      color2: 'blue'  
    }  
  }  
}  
</script>  
  
<style>  
  
</style>
```

四、自定义指令-v-loading指令的封装

1.场景

实际开发过程中，发送请求需要时间，在请求的数据未回来时，页面会处于**空白状态** => 用户体验不好

2.需求

封装一个 v-loading 指令，实现加载中的效果

3.分析

- 1.本质 loading效果就是一个蒙层，盖在了盒子上
- 2.数据请求中，开启loading状态，添加蒙层
- 3.数据请求完毕，关闭loading状态，移除蒙层

4.实现

- 1.准备一个 loading类，通过伪元素定位，设置宽高，实现蒙层
- 2.开启关闭 loading状态（添加移除蒙层），本质只需要添加移除类即可
- 3.结合自定义指令的语法进行封装复用

```
.loading:before {  
  content: "";  
  position: absolute;  
  left: 0;  
  top: 0;  
  width: 100%;  
  height: 100%;  
  background: #fff url("../loading.gif") no-repeat center;  
}
```

5.准备代码

```
<template>
  <div class="main">
    <div class="box">
      <ul>
        <li v-for="item in list" :key="item.id" class="news">
          <div class="left">
            <div class="title">{{ item.title }}</div>
            <div class="info">
              <span>{{ item.source }}</span>
              <span>{{ item.time }}</span>
            </div>
          </div>
          <div class="right">
            
          </div>
        </li>
      </ul>
    </div>
  </div>
</template>

<script>
// 安装axios => yarn add axios || npm i axios
import axios from 'axios'

// 接口地址: http://hmajax.itheima.net/api/news
// 请求方式: get
export default {
  data () {
    return {
      list: [],
      isLoading: false,
      isLoading2: false
    }
  },
  async created () {
    // 1. 发送请求获取数据
    const res = await axios.get('http://hmajax.itheima.net/api/news')

    setTimeout(() => {
      // 2. 更新到 list 中, 用于页面渲染 v-for
      this.list = res.data.data
    }, 2000)
  }
}
</script>

<style>
.loading:before {
  content: '';
  position: absolute;
```

```
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
    background: #fff url('./loading.gif') no-repeat center;
}

.box2 {
    width: 400px;
    height: 400px;
    border: 2px solid #000;
    position: relative;
}

.box {
    width: 800px;
    min-height: 500px;
    border: 3px solid orange;
    border-radius: 5px;
    position: relative;
}

.news {
    display: flex;
    height: 120px;
    width: 600px;
    margin: 0 auto;
    padding: 20px 0;
    cursor: pointer;
}

.news .left {
    flex: 1;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    padding-right: 10px;
}

.news .left .title {
    font-size: 20px;
}

.news .left .info {
    color: #999999;
}

.news .left .info span {
    margin-right: 20px;
}

.news .right {
    width: 160px;
    height: 120px;
}

.news .right img {
    width: 100%;
    height: 100%;
}
```

```
object-fit: cover;
}
</style>
```

五、插槽-默认插槽

1.作用

让组件内部的一些 **结构** 支持 **自定义**



2.需求

将需要多次显示的对话框,封装成一个组件

3.问题

组件的内容部分, **不希望写死**, 希望能使用的时候 **自定义**。怎么办

4.插槽的基本语法

1. 组件内需要定制的结构部分, 改用 占位
2. 使用组件时, 标签内部, 传入结构替换slot
3. 给插槽传入内容时, 可以传入 **纯文本**、**html标签**、**组件**

```
<template>
  <div class="dialog">
    <div class="dialog-header">
      <h3>友情提示</h3>
      <span class="close">✕□</span>
    </div>

    <div class="dialog-content">
      <slot></slot>
    </div>
    <div class="dialog-footer">
      <button>取消</button>
      <button>确认</button>
    </div>
  </div>
</template>
```

```
<MyDialog>
  你确认要退出本系统么?
</MyDialog>
```

5.代码示例

MyDialog.vue

```
<template>
  <div class="dialog">
    <div class="dialog-header">
      <h3>友情提示</h3>
      <span class="close">✕</span>
    </div>

    <div class="dialog-content">
      您确定要进行删除操作吗?
    </div>

    <div class="dialog-footer">
      <button>取消</button>
      <button>确认</button>
    </div>
  </div>
</template>

<script>
export default {
  data () {
    return {

    }
  }
}
</script>

<style scoped>
* {
  margin: 0;
  padding: 0;
}
.dialog {
  width: 470px;
  height: 230px;
  padding: 0 25px;
  background-color: #ffffff;
  margin: 40px auto;
  border-radius: 5px;
}
.dialog-header {
  height: 70px;
  line-height: 70px;
  font-size: 20px;
  border-bottom: 1px solid #ccc;
  position: relative;
}
.dialog-header .close {
  position: absolute;
  right: 0px;
```

```

    top: 0px;
    cursor: pointer;
  }
  .dialog-content {
    height: 80px;
    font-size: 18px;
    padding: 15px 0;
  }
  .dialog-footer {
    display: flex;
    justify-content: flex-end;
  }
  .dialog-footer button {
    width: 65px;
    height: 35px;
    background-color: #ffffff;
    border: 1px solid #e1e3e9;
    cursor: pointer;
    outline: none;
    margin-left: 10px;
    border-radius: 3px;
  }
  .dialog-footer button:last-child {
    background-color: #007acc;
    color: #fff;
  }
</style>

```

App.vue

```

<template>
  <div>
    <MyDialog>
    </MyDialog>
  </div>
</template>

<script>
import MyDialog from './components/MyDialog.vue'
export default {
  data () {
    return {

    }
  },
  components: {
    MyDialog
  }
}
</script>

<style>
body {
  background-color: #b3b3b3;

```

```
}  
</style>
```

6.总结

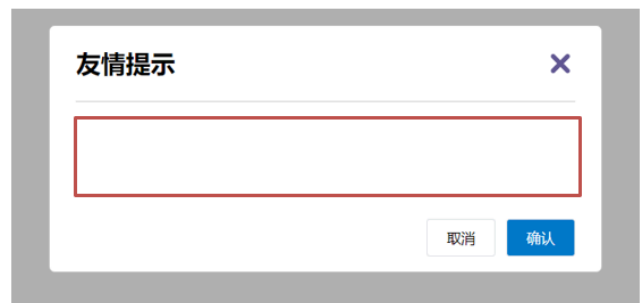
场景：组件内某一部分结构不确定，想要自定义怎么办

使用：插槽的步骤分为哪几步？

六、插槽-后备内容（默认值）

1.问题

通过插槽完成了内容的定制，传什么显示什么, 但是如果不传，则是空白



能否给插槽设置 默认显示内容 呢？

2.插槽的后备内容

封装组件时，可以为预留的 `<slot>` 插槽提供后备内容（默认内容）。

3.语法

在 `template` 标签内，放置内容, 作为默认显示内容


```

<template>
  <div class="dialog">
    <div class="dialog-header">
      <h3>友情提示</h3>
      <span class="close">✖ □</span>
    </div>

    <div class="dialog-content">
      <slot>我是后备内容</slot>
    </div>
    <div class="dialog-footer">
      <button>取消</button>
      <button>确认</button>
    </div>
  </div>
</template>

```

4.效果

- 外部使用组件时，不传东西，则slot会显示后备内容

```
<MyDialog></MyDialog>
```

- 外部使用组件时，传东西了，则slot整体会被换掉

```
<MyDialog>我是内容</MyDialog>
```

5.代码示例

App.vue

```

<template>
  <div>
    <MyDialog></MyDialog>
    <MyDialog>
      你确认要退出么
    </MyDialog>
  </div>
</template>

```

```
<script>
import MyDialog from './components/MyDialog.vue'
export default {
  data () {
    return {

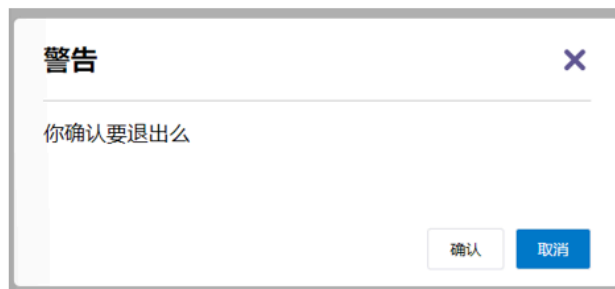
    }
  },
  components: {
    MyDialog
  }
}
</script>

<style>
body {
  background-color: #b3b3b3;
}
</style>
```

七、插槽-具名插槽

1.需求

一个组件内有多处结构，需要外部传入标签，进行定制



上面的弹框中有三处不同，但是默认插槽只能定制一个位置，这时候怎么办呢？

2.具名插槽语法

- 多个slot使用name属性区分名字

```

<div class="dialog-header">
  <slot name="head"></slot>
</div>
<div class="dialog-content">
  <slot name="content"></slot>
</div>
<div class="dialog-footer">
  <slot name="footer"></slot>
</div>

```

- template配合v-slot:名字来分发对应标签

```

<MyDialog>
  <template v-slot:head>
    大标题
  </template>
  <template v-slot:content>
    内容文本
  </template>
  <template v-slot:footer>
    <button>按钮</button>
  </template>
</MyDialog>

```

3.v-slot的简写

v-slot写起来太长，vue给我们提供一个简单写法 v-slot -> #

4.总结

- 组件内 有多处不确定的结构 怎么办?
- 具名插槽的语法是什么?
- v-slot:插槽名可以简化成什么?

八、作用域插槽

1.插槽分类

- 默认插槽
- 具名插槽

插槽只有两种，作用域插槽不属于插槽的一种分类

2.作用

定义slot 插槽的同时, 是可以**传值的**。给 **插槽** 上可以 **绑定数据**，将来 **使用组件时**可以用

3.场景

封装表格组件

序号	姓名	年纪	操作
1	张小花	18	删除
2	孙大明	19	删除
3	刘德忠	17	删除

序号	姓名	年纪	操作
1	赵小云	18	查看
2	刘蓓蓓	19	查看
3	姜肖泰	17	查看

4.使用步骤

1. 给 slot 标签, 以 添加属性的方式传值

```
<slot :id="item.id" msg="测试文本"></slot>
```

2. 所有添加的属性, 都会被收集到一个对象中

```
{ id: 3, msg: '测试文本' }
```

3. 在template中, 通过 `#插槽名= "obj"` 接收, 默认插槽名为 default

```
<MyTable :list="list">
  <template #default="obj">
    <button @click="del(obj.id)">删除</button>
  </template>
</MyTable>
```

5.代码示例

MyTable.vue

```
<template>
  <table class="my-table">
    <thead>
      <tr>
        <th>序号</th>
        <th>姓名</th>
        <th>年纪</th>
        <th>操作</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>1</td>
        <td>赵小云</td>
        <td>19</td>
        <td>
          <button>
            查看
          </button>
        </td>
      </tr>
      <tr>
        <td>1</td>
        <td>张小花</td>
        <td>19</td>
        <td>
          <button>
            查看
          </button>
        </td>
      </tr>
    </tbody>
  </table>
</template>
```

```
<td>1</td>
<td>孙大明</td>
<td>19</td>
<td>
  <button>
    查看
  </button>
</td>
</tr>
</tbody>
</table>
</template>

<script>
export default {
  props: {
    data: Array
  }
}
</script>

<style scoped>
.my-table {
  width: 450px;
  text-align: center;
  border: 1px solid #ccc;
  font-size: 24px;
  margin: 30px auto;
}
.my-table thead {
  background-color: #1f74ff;
  color: #fff;
}
.my-table thead th {
  font-weight: normal;
}
.my-table thead tr {
  line-height: 40px;
}
.my-table th,
.my-table td {
  border-bottom: 1px solid #ccc;
  border-right: 1px solid #ccc;
}
.my-table td:last-child {
  border-right: none;
}
.my-table tr:last-child td {
  border-bottom: none;
}
.my-table button {
  width: 65px;
  height: 35px;
  font-size: 18px;
```

```
border: 1px solid #ccc;
outline: none;
border-radius: 3px;
cursor: pointer;
background-color: #ffffff;
margin-left: 5px;
}
</style>
```

App.vue

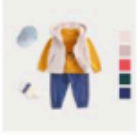
```
<template>
  <div>
    <MyTable :data="list"></MyTable>
    <MyTable :data="list2"></MyTable>
  </div>
</template>

<script>
import MyTable from './components/MyTable.vue'
export default {
  data () {
    return {
      list: [
        { id: 1, name: '张小花', age: 18 },
        { id: 2, name: '孙大明', age: 19 },
        { id: 3, name: '刘德忠', age: 17 },
      ],
      list2: [
        { id: 1, name: '赵小云', age: 18 },
        { id: 2, name: '刘蓓蓓', age: 19 },
        { id: 3, name: '姜肖泰', age: 17 },
      ]
    }
  },
  components: {
    MyTable
  }
}
</script>
```

6.总结

- 1.作用域插槽的作用是什么？
- 2.作用域插槽的使用步骤是什么？

九、综合案例 - 商品列表-MyTag组件抽离

编号	图片	名称	标签
101		梨皮朱泥三绝清代小品壶经典款紫砂壶	茶具
102		全防水HABU旋钮牛皮户外徒步鞋山宁泰抗菌	男鞋
103		毛茸茸小熊出没，儿童羊羔绒背心73-90cm	儿童服饰
104		基础百搭，儿童套头针织毛衣1-9岁	儿童服饰

1.需求说明

- 1. my-tag 标签组件封装
 - (1) 双击显示输入框，输入框获取焦点
 - (2) 失去焦点，隐藏输入框
 - (3) 回显标签信息
 - (4) 内容修改，回车 → 修改标签信息
- 2. my-table 表格组件封装
 - (1) 动态传递表格数据渲染
 - (2) 表头支持用户自定义
 - (3) 主体支持用户自定义

2.代码准备

```
<template>
  <div class="table-case">
    <table class="my-table">
      <thead>
        <tr>
          <th>编号</th>
          <th>名称</th>
          <th>图片</th>
          <th width="100px">标签</th>
        </tr>
      </thead>
```



```

<tbody>
  <tr>
    <td>1</td>
    <td>梨皮朱泥三绝清代小品壶经典款紫砂壶</td>
    <td>
      
    </td>
    <td>
      <div class="my-tag">
        <!-- <input
          class="input"
          type="text"
          placeholder="输入标签"
        /> -->
        <div class="text">
          茶具
        </div>
      </div>
    </td>
  </tr>
  <tr>
    <td>1</td>
    <td>梨皮朱泥三绝清代小品壶经典款紫砂壶</td>
    <td>
      
    </td>
    <td>
      <div class="my-tag">
        <!-- <input
          ref="inp"
          class="input"
          type="text"
          placeholder="输入标签"
        /> -->
        <div class="text">
          男靴
        </div>
      </div>
    </td>
  </tr>
</tbody>
</table>
</div>
</template>

<script>
export default {
  name: 'TableCase',
  components: {},
  data() {
    return {
      goods: [
        {
          id: 101,

```

```

        picture:
            'https://yanxuan-item.nosdn.127.net/f8c37ffa41ab1eb84bffa499e1f6acfc7.jpg',
        name: '梨皮朱泥三绝清代小品壶经典款紫砂壶',
        tag: '茶具',
    },
    {
        id: 102,
        picture:
            'https://yanxuan-item.nosdn.127.net/221317c85274a188174352474b859d7b.jpg',
        name: '全防水HABU旋钮牛皮户外徒步鞋山宁泰抗菌',
        tag: '男鞋',
    },
    {
        id: 103,
        picture:
            'https://yanxuan-item.nosdn.127.net/cd4b840751ef4f7505c85004f0bebc5.png',
        name: '毛茸茸小熊出没, 儿童羊羔绒背心73-90cm',
        tag: '儿童服饰',
    },
    {
        id: 104,
        picture:
            'https://yanxuan-item.nosdn.127.net/56eb25a38d7a630e76a608a9360eec6b.jpg',
        name: '基础百搭, 儿童套头针织毛衣1-9岁',
        tag: '儿童服饰',
    },
    ],
}
},
}
</script>

```

```

<style lang="less" scoped>

```

```

.table-case {
    width: 1000px;
    margin: 50px auto;
    img {
        width: 100px;
        height: 100px;
        object-fit: contain;
        vertical-align: middle;
    }

    .my-table {
        width: 100%;
        border-spacing: 0;
        img {
            width: 100px;
            height: 100px;
            object-fit: contain;
            vertical-align: middle;
        }
        th {
            background: #f5f5f5;

```

```

        border-bottom: 2px solid #069;
    }
    td {
        border-bottom: 1px dashed #ccc;
    }
    td,
    th {
        text-align: center;
        padding: 10px;
        transition: all 0.5s;
        &.red {
            color: red;
        }
    }
}
.none {
    height: 100px;
    line-height: 100px;
    color: #999;
}
}
.my-tag {
    cursor: pointer;
    .input {
        appearance: none;
        outline: none;
        border: 1px solid #ccc;
        width: 100px;
        height: 40px;
        box-sizing: border-box;
        padding: 10px;
        color: #666;
        &::placeholder {
            color: #666;
        }
    }
}
}
</style>

```

3.my-tag组件封装-创建组件

MyTag.vue

```

<template>
  <div class="my-tag">
    <!-- <input
      class="input"
      type="text"
      placeholder="输入标签"
    /> -->
    <div
      class="text">
        茶具

```

```

    </div>
  </div>
</template>

<script>
export default {

}
</script>

<style lang="less" scoped>
.my-tag {
  cursor: pointer;
  .input {
    appearance: none;
    outline: none;
    border: 1px solid #ccc;
    width: 100px;
    height: 40px;
    box-sizing: border-box;
    padding: 10px;
    color: #666;
    &::placeholder {
      color: #666;
    }
  }
}
}
</style>

```

App.vue

```

<template>
  ...
  <tbody>
    <tr>
      ....
      <td>
        <MyTag></MyTag>
      </td>
    </tr>
  </tbody>
  ...
</template>
<script>
import MyTag from './components/MyTag.vue'
export default {
  name: 'TableCase',
  components: {
    MyTag,
  },
  ...
</script>

```

十、综合案例-MyTag组件控制显示隐藏

MyTag.vue

```
<template>
  <div class="my-tag">
    <input
      v-if="isEdit"
      v-focus
      ref="inp"
      class="input"
      type="text"
      placeholder="输入标签"
      @blur="isEdit = false"
    />
    <div
      v-else
      @dblclick="handleClick"
      class="text">
      茶具
    </div>
  </div>
</template>

<script>
export default {
  data () {
    return {
      isEdit: false
    }
  },
  methods: {
    handleClick () {
      this.isEdit = true
    }
  }
}
</script>
```

main.js

```
// 封装全局指令 focus
Vue.directive('focus', {
  // 指令所在的dom元素，被插入到页面中时触发
  inserted (el) {
    el.focus()
  }
})
```

十一、综合案例-MyTag组件进行v-model绑定

App.vue

```
<MyTag v-model="tempText"></MyTag>
<script>
  export default {
    data() {
      tempText: '水杯'
    }
  }
</script>
```

MyTag.vue

```
<template>
  <div class="my-tag">
    <input
      v-if="isEdit"
      v-focus
      ref="inp"
      class="input"
      type="text"
      placeholder="输入标签"
      :value="value"
      @blur="isEdit = false"
      @keyup.enter="handleEnter"
    />
    <div
      v-else
      @dblclick="handleClick"
      class="text">
        {{ value }}
      </div>
    </div>
  </div>
</template>

<script>
export default {
  props: {
    value: String
  },
  data () {
    return {
      isEdit: false
    }
  },
  methods: {
    handleClick () {
      this.isEdit = true
    },
    handleEnter (e) {
      // 非空处理
```

```

        if (e.target.value.trim() === '') return alert('标签内容不能为空')
        this.$emit('input', e.target.value)
        // 提交完成，关闭输入状态
        this.isEdit = false
    }
}
}
</script>

```

十二、综合案例-封装MyTable组件-动态渲染数据

App.vue

```

<template>
  <div class="table-case">
    <MyTable :data="goods"></MyTable>
  </div>
</template>

<script>
import MyTable from './components/MyTable.vue'
export default {
  name: 'TableCase',
  components: {
    MyTable
  },
  data(){
    return {
      ....
    }
  },
}
</script>

```

MyTable.vue

```

<template>
  <table class="my-table">
    <thead>
      <tr>
        <th>编号</th>
        <th>名称</th>
        <th>图片</th>
        <th width="100px">标签</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="(item, index) in data" :key="item.id">
        <td>{{ index + 1 }}</td>
        <td>{{ item.name }}</td>

```

```

        <td>
          
        </td>
        <td>
          标签内容
          <!-- <MyTag v-model="item.tag"></MyTag> -->
        </td>
      </tr>
    </tbody>
  </table>
</template>

```

```

<script>
export default {
  props: {
    data: {
      type: Array,
      required: true
    }
  }
};
</script>

```

```

<style lang="less" scoped>

.my-table {
  width: 100%;
  border-spacing: 0;
  img {
    width: 100px;
    height: 100px;
    object-fit: contain;
    vertical-align: middle;
  }
  th {
    background: #f5f5f5;
    border-bottom: 2px solid #069;
  }
  td {
    border-bottom: 1px dashed #ccc;
  }
  td,
  th {
    text-align: center;
    padding: 10px;
    transition: all .5s;
    &.red {
      color: red;
    }
  }
}

.none {
  height: 100px;
}

```



```
    line-height: 100px;
    color: #999;
  }
}

</style>
```

十三、综合案例-封装MyTable组件-自定义结构

App.vue

```
<template>
  <div class="table-case">
    <MyTable :data="goods">
      <template #head>
        <th>编号</th>
        <th>名称</th>
        <th>图片</th>
        <th width="100px">标签</th>
      </template>

      <template #body="{ item, index }">
        <td>{{ index + 1 }}</td>
        <td>{{ item.name }}</td>
        <td>
          
        </td>
        <td>
          <MyTag v-model="item.tag"></MyTag>
        </td>
      </template>
    </MyTable>
  </div>
</template>

<script>
import MyTag from './components/MyTag.vue'
import MyTable from './components/MyTable.vue'
export default {
  name: 'TableCase',
  components: {
    MyTag,
    MyTable
  },
  data () {
    return {
      ...
    }
  }
}
```

```
}  
</script>
```

MyTable.vue

```
<template>  
  <table class="my-table">  
    <thead>  
      <tr>  
        <slot name="head"></slot>  
      </tr>  
    </thead>  
    <tbody>  
      <tr v-for="(item, index) in data" :key="item.id">  
        <slot name="body" :item="item" :index="index" ></slot>  
      </tr>  
    </tbody>  
  </table>  
</template>  
  
<script>  
export default {  
  props: {  
    data: {  
      type: Array,  
      required: true  
    }  
  }  
};  
</script>
```

十四、单页应用程序介绍

1.概念

单页应用程序：SPA【Single Page Application】是指所有的功能都在一个html页面上实现

2.具体示例

单页应用网站： 网易云音乐 <https://music.163.com/>

多页应用网站： 京东 <https://jd.com/>

3.单页应用 VS 多页面应用

开发分类	实现方式	页面性能	开发效率	用户体验	学习成本	首屏加载	SEO
单页	一个html页面	按需更新 性能高	高	非常好	高	慢	差
多页	多个html页面	整页更新 性能低	中等	一般	中等	快	优

单页应用类网站：系统类网站 / 内部网站 / 文档类网站 / 移动端站点

多页应用类网站：公司官网 / 电商类网站

4.总结

- 1.什么是单页面应用程序？
- 2.单页面应用优缺点？
- 3.单页应用场景？

十五、路由介绍

1.思考

单页面应用程序，之所以开发效率高，性能好，用户体验好

最大的原因就是： 页面按需更新



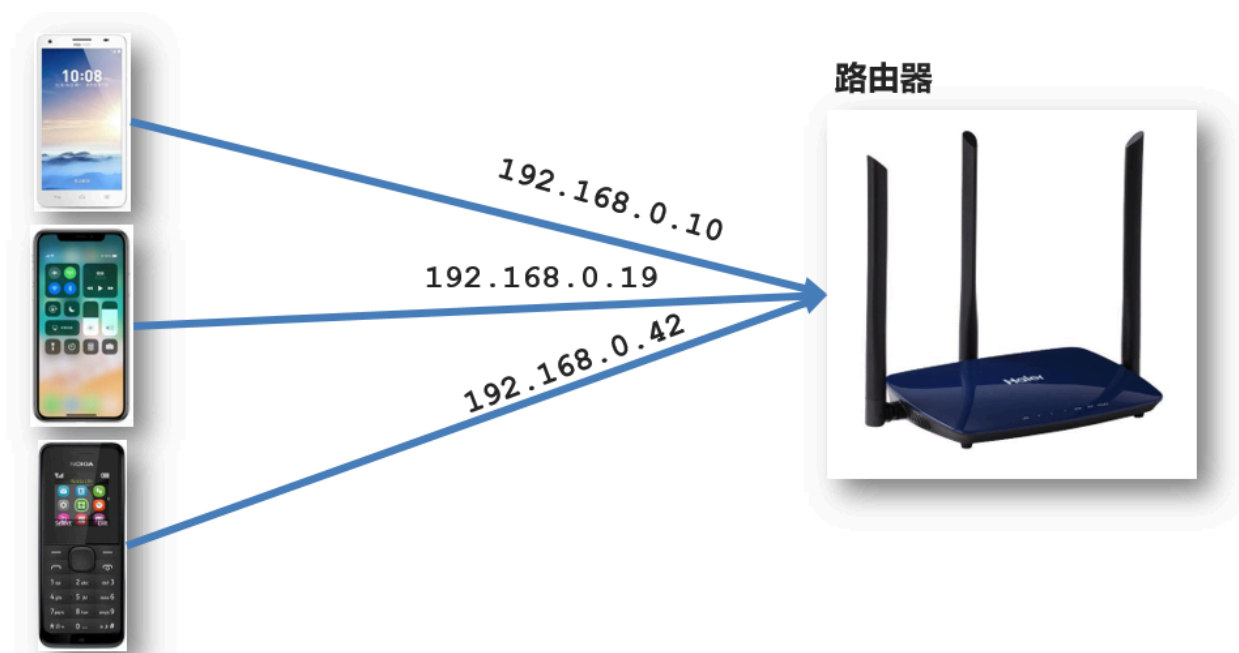
比如当点击【发现音乐】和【关注】时，只是更新下面部分内容，对于头部是不更新的

要按需更新，首先就需要明确： 访问路径和 组件的对应关系！

访问路径 和 组件的对应关系如何确定呢？ 路由

2.路由的介绍

生活中的路由：设备和ip的映射关系

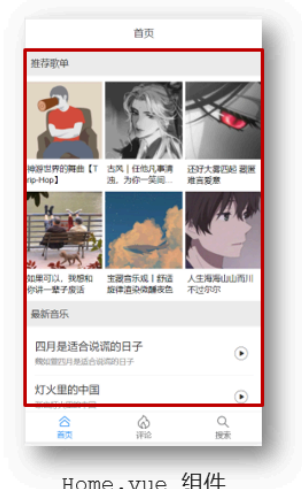


Vue中的路由：路径和组件的映射关系

`http://localhost:8080/#/home`

`http://localhost:8080/#/comment`

`http://localhost:8080/#/search`



3.总结

- 什么是路由
- Vue中的路由是什么

十六、路由的基本使用

1.目标

认识插件 VueRouter，掌握 VueRouter 的基本使用步骤

2.作用

修改地址栏路径时，切换显示匹配的组件

3.说明

Vue 官方的一个路由插件，是一个第三方包

4.官网

<https://v3.router.vuejs.org/zh/>

5.VueRouter的使用（5+2）

固定5个固定的步骤（不用死背，熟能生巧）

1. 下载 VueRouter 模块到当前工程，版本3.6.5

```
yarn add vue-router@3.6.5
```

2. main.js中引入VueRouter

```
import VueRouter from 'vue-router'
```

3. 安装注册

```
Vue.use(VueRouter)
```

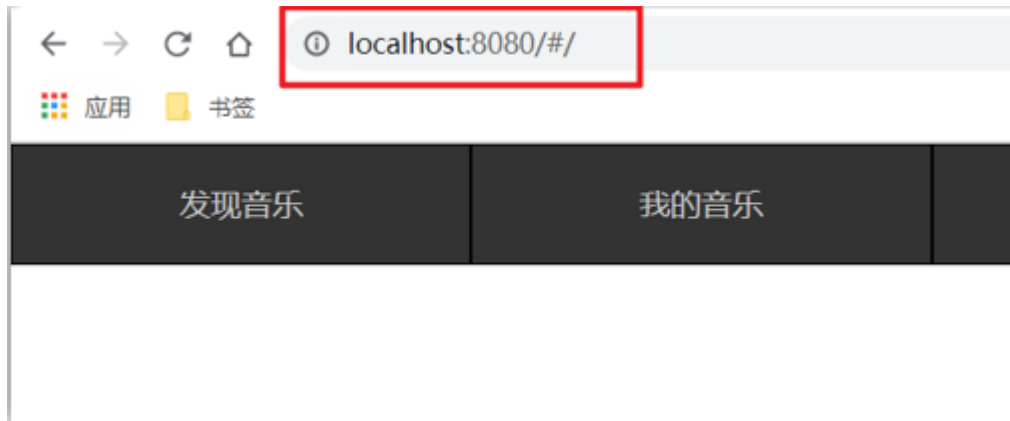
4. 创建路由对象

```
const router = new VueRouter()
```

5. 注入，将路由对象注入到new Vue实例中，建立关联

```
new Vue({  
  render: h => h(App),  
  router:router  
}).$mount('#app')
```

当我们配置完以上5步之后 就可以看到浏览器地址栏中的路由 变成了 /#/的形式。表示项目的路由已经被Vue-Router管理了



6.代码示例

main.js

```
// 路由的使用步骤 5 + 2
// 5个基础步骤
// 1. 下载 v3.6.5
// yarn add vue-router@3.6.5
// 2. 引入
// 3. 安装注册 Vue.use(Vue插件)
// 4. 创建路由对象
// 5. 注入到new Vue中, 建立关联

import VueRouter from 'vue-router'
Vue.use(VueRouter) // VueRouter插件初始化

const router = new VueRouter()

new Vue({
  render: h => h(App),
  router
}).$mount('#app')
```

7.两个核心步骤

1. 创建需要的组件 (views目录), 配置路由规则

Find.vue

My.vue

Friend.vue

```
import Find from './views/Find.vue'
import My from './views/My.vue'
import Friend from './views/Friend.vue'

const router = new VueRouter({
  routes: [
    { path: '/find', component: Find },
    { path: '/my', component: My },
    { path: '/friend', component: Friend },
  ]
})
```

地址栏路径

组件

2. 配置导航，配置路由出口(路径匹配的组件显示的位置)

App.vue

```
<div class="footer_wrap">
  <a href="#/find">发现音乐</a>
  <a href="#/my">我的音乐</a>
  <a href="#/friend">朋友</a>
</div>
<div class="top">
  <router-view></router-view>
</div>
```

8.总结

1. 如何实现 路径改变，对应组件 切换,应该使用哪个插件?
2. Vue-Router的使用步骤是什么(5+2)?

十七、组件的存放目录问题

注意：.vue文件 本质无区别

1.组件分类

.vue文件分为2类，都是 .vue文件（本质无区别）

- 页面组件（配置路由规则时使用的组件）
- 复用组件（多个组件中都使用到的组件）



2.存放目录

分类开来的目的就是为了 更易维护

1. src/views文件夹
页面组件 - 页面展示 - 配合路由用
2. src/components文件夹
复用组件 - 展示数据 - 常用于复用

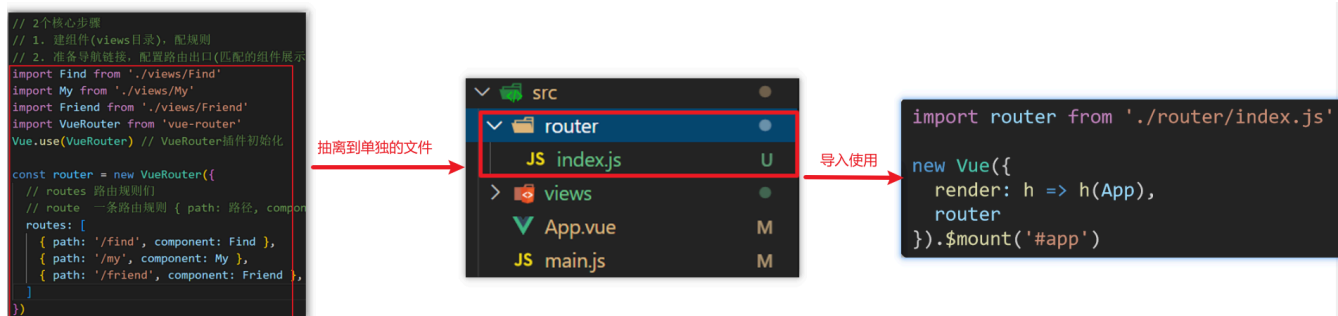
3.总结

- 组件分类有哪两类？分类的目的？
- 不同分类的组件应该放在什么文件夹？作用分别是什么？

十八、路由的封装抽离

问题：所有的路由配置都在main.js中合适吗？

目标：将路由模块抽离出来。 好处：拆分模块，利于维护



路径简写：

脚手架环境下 @指代src目录，可以用于快速引入组件

总结：

- 路由模块的封装抽离的好处是什么？
- 以后如何快速引入组件？

day06

一、声明式导航-导航链接

1.需求

实现导航高亮效果



如果使用a标签进行跳转的话，需要给当前跳转的导航加样式，同时要移除上一个a标签的样式，太麻烦！！

2.解决方案

vue-router 提供了一个全局组件 router-link (取代 a 标签)

- 能跳转，配置 to 属性指定路径(必须)。本质还是 a 标签，to 无需 #
- 能高亮，默认就会提供高亮类名，可以直接设置高亮样式

语法：发现音乐

```
<div>
  <div class="footer_wrap">
    <router-link to="/find">发现音乐</router-link>
    <router-link to="/my">我的音乐</router-link>
    <router-link to="/friend">朋友</router-link>
  </div>
  <div class="top">
    <!-- 路由出口 -> 匹配的组件所展示的位置 -->
    <router-view></router-view>
  </div>
</div>
```

3.通过router-link自带的两个样式进行高亮

使用router-link跳转后，我们发现。当前点击的链接默认加了两个class的值 `router-link-exact-active` 和 `router-link-active`

我们可以给任意一个class属性添加高亮样式即可实现功能

4.总结

- router-link是什么？
- router-link怎么用？
- router-link的好处是什么？

二、声明式导航-两个类名

当我们使用跳转时，自动给当前导航加了两个类名

```
<div class="footer_wrap">
  <a href="#/find" class>发现音乐</a>
  <a href="#/my" aria-current="page" class="router-link-exact-active router-link-active">我的音乐</a> == $0
  <a href="#/friend" class>朋友</a>
</div>
```

1.router-link-active

模糊匹配（用的多）

to="/my" 可以匹配 /my /my/a /my/b

只要是以/my开头的路径 都可以和 to="/my"匹配到

2.router-link-exact-active

精确匹配

to="/my" 仅可以匹配 /my

3.在地址栏中输入二级路由查看类名的添加

4.总结

- router-link 会自动给当前导航添加两个类名，有什么区别呢？

三、声明式导航-自定义类名（了解）

1.问题

router-link的两个高亮类名 太长了，我们希望能定制怎么办

```
<div class="footer_wrap">
  <a href="#/find" class>发现音乐</a>
  <a href="#/my" aria-current="page" class="router-link-exact-active router-link-active">我的音乐</a> == $0
  <a href="#/friend" class>朋友</a>
</div>
```

2.解决方案

我们可以在创建路由对象时，额外配置两个配置项即可。 `linkActiveClass` 和 `linkExactActiveClass`

```
const router = new VueRouter({
  routes: [...],
  linkActiveClass: "类名1",
  linkExactActiveClass: "类名2"
})

<div class="footer_wrap"> == $0
  <a href="#/find" class>发现音乐</a>
  <a href="#/my" aria-current="page" class="类名2 类名1">我的音乐</a>
  <a href="#/friend" class>朋友</a>
</div>
```

3.代码演示

```
// 创建了一个路由对象
const router = new VueRouter({
  routes: [
    ...
  ],
  linkActiveClass: 'active', // 配置模糊匹配的类名
  linkExactActiveClass: 'exact-active' // 配置精确匹配的类名
})
```

4.总结

如何自定义router-link的两个高亮类名

四、声明式导航-查询参数传参

1.目标

在跳转路由时，进行传参



比如：现在我们在搜索页点击了热门搜索链接，跳转到详情页，需要把点击的内容带到详情页，改怎么办呢？

2.跳转传参

我们可以通过两种方式，在跳转的时候把所需要的参数传到其他页面中

- 查询参数传参
- 动态路由传参

3.查询参数传参

- 如何传参？
- 如何接受参数

固定用法：\$router.query.参数名

4.代码演示

App.vue

```
<template>
  <div id="app">
    <div class="link">
      <router-link to="/home">首页</router-link>
      <router-link to="/search">搜索页</router-link>
    </div>
  </div>
</template>
```

```

    </div>

    <router-view></router-view>
  </div>
</template>

<script>
export default {};
</script>

<style scoped>
.link {
  height: 50px;
  line-height: 50px;
  background-color: #495150;
  display: flex;
  margin: -8px -8px 0 -8px;
  margin-bottom: 50px;
}
.link a {
  display: block;
  text-decoration: none;
  background-color: #ad2a26;
  width: 100px;
  text-align: center;
  margin-right: 5px;
  color: #fff;
  border-radius: 5px;
}
</style>

```

Home.vue

```

<template>
  <div class="home">
    <div class="logo-box"></div>
    <div class="search-box">
      <input type="text">
      <button>搜索一下</button>
    </div>
    <div class="hot-link">
      热门搜索：
      <router-link to="">黑马程序员</router-link>
      <router-link to="">前端培训</router-link>
      <router-link to="">如何成为前端大牛</router-link>
    </div>
  </div>
</template>

<script>
export default {
  name: 'FindMusic'
}
</script>

```

```

<style>
.logo-box {
  height: 150px;
  background: url('@/assets/logo.jpeg') no-repeat center;
}
.search-box {
  display: flex;
  justify-content: center;
}
.search-box input {
  width: 400px;
  height: 30px;
  line-height: 30px;
  border: 2px solid #c4c7ce;
  border-radius: 4px 0 0 4px;
  outline: none;
}
.search-box input:focus {
  border: 2px solid #ad2a26;
}
.search-box button {
  width: 100px;
  height: 36px;
  border: none;
  background-color: #ad2a26;
  color: #fff;
  position: relative;
  left: -2px;
  border-radius: 0 4px 4px 0;
}
.hot-link {
  width: 508px;
  height: 60px;
  line-height: 60px;
  margin: 0 auto;
}
.hot-link a {
  margin: 0 5px;
}
</style>

```

Search.vue

```

<template>
  <div class="search">
    <p>搜索关键字：黑马程序员</p>
    <p>搜索结果： </p>
    <ul>
      <li>.....</li>
      <li>.....</li>
      <li>.....</li>
      <li>.....</li>
    </ul>
  </div>
</template>

```

```

    </div>
  </template>

  <script>
  export default {
    name: 'MyFriend',
    created () {
      // 在created中，获取路由参数
    }
  }
  </script>

  <style>
  .search {
    width: 400px;
    height: 240px;
    padding: 0 20px;
    margin: 0 auto;
    border: 2px solid #c4c7ce;
    border-radius: 5px;
  }
  </style>

```

router/index.js

```

import Home from '@views/Home'
import Search from '@views/Search'
import Vue from 'vue'
import VueRouter from 'vue-router'
Vue.use(VueRouter) // VueRouter插件初始化

// 创建了一个路由对象
const router = new VueRouter({
  routes: [
    { path: '/home', component: Home },
    { path: '/search', component: Search }
  ]
})

export default router

```

main.js

```

...
import router from './router/index'
...
new Vue({
  render: h => h(App),
  router
}).$mount('#app')

```


五、声明式导航-动态路由传参

1. 动态路由传参方式

- 配置动态路由

动态路由后面的参数可以随便起名，但要有语义

```
const router = new VueRouter({
  routes: [
    ...,
    {
      path: '/search/:words',
      component: Search
    }
  ]
})
```

- 配置导航链接

to="/path/参数值"

- 对应页面组件接受参数

\$route.params.参数名

params后面的参数名要和动态路由配置的参数保持一致

2. 查询参数传参 VS 动态路由传参

- 查询参数传参 (比较适合传多个参数)

1. 跳转: to="/path?参数名=值&参数名2=值"
2. 获取: \$route.query.参数名

2. 动态路由传参 (优雅简洁, 传单个参数比较方便)

1. 配置动态路由: path: "/path/:参数名"
2. 跳转: to="/path/参数值"
3. 获取: \$route.params.参数名

注意: 动态路由也可以传多个参数, 但一般只传一个

3. 总结

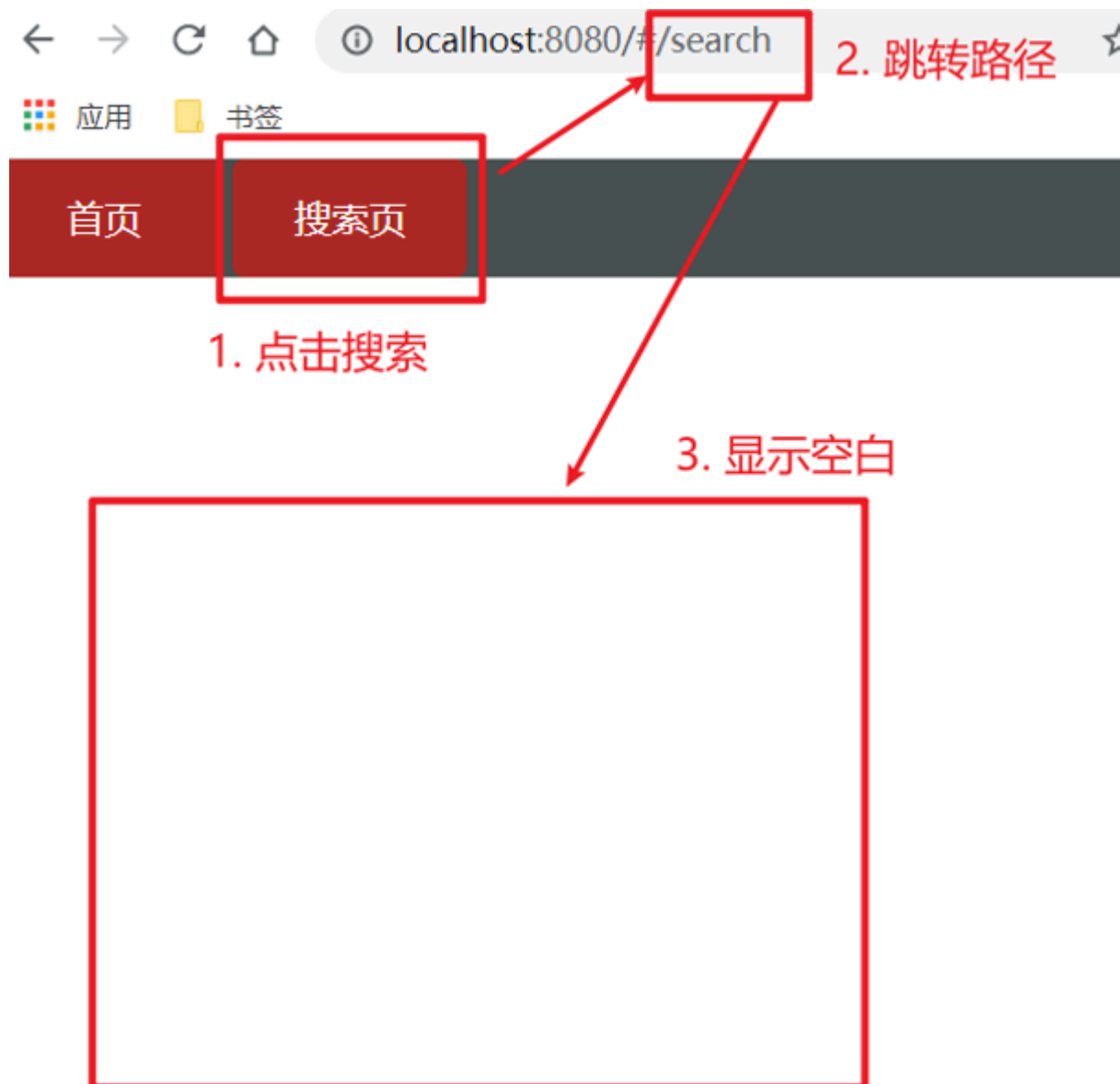
声明式导航跳转时, 有几种方式传值给路由页面?

- 查询参数传参 (多个参数)
- 动态路由传参 (一个参数, 优雅简洁)

六、动态路由参数的可选符(了解)

1.问题

配了路由 `path: "/search/:words"` 为什么按下面步骤操作，会未匹配到组件，显示空白？



2.原因

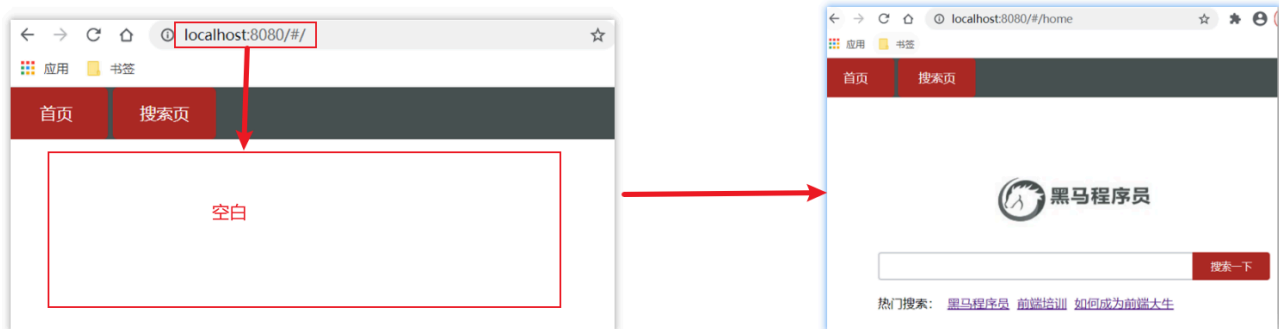
`/search/:words` 表示，**必须要传参数**。如果不传参数，也希望匹配，可以加个可选符"?"

```
const router = new VueRouter({
  routes: [
    ...
    { path: '/search/:words?', component: Search }
  ]
})
```

七、Vue路由-重定向

1.问题

网页打开时，url 默认是 / 路径，未匹配到组件时，会出现空白



2.解决方案

重定向 → 匹配 / 后, 强制跳转 /home 路径

3.语法

```
{ path: 匹配路径, redirect: 重定向到的路径 },  
比如:  
{ path: '/', redirect: '/home' }
```

4.代码演示

```
const router = new VueRouter({  
  routes: [  
    { path: '/', redirect: '/home' },  
    ...  
  ]  
})
```

八、Vue路由-404

1.作用

当路径找不到匹配时，给个提示页面

2.位置

404的路由，虽然配置在任何一个位置都可以，但一般都**配置在其他路由规则的最后面**

3.语法

path: "*" (任意路径) – 前面不匹配就命中最后这个

```
import NotFound from '@/views/NotFound'

const router = new VueRouter({
  routes: [
    ...
    { path: '*', component: NotFound } //最后一个
  ]
})
```

4.代码示例

NotFound.vue

```
<template>
  <div>
    <h1>404 Not Found</h1>
  </div>
</template>

<script>
export default {
}
</script>

<style>

</style>
```

router/index.js

```
...
import NotFound from '@views/NotFound'
...

// 创建了一个路由对象
const router = new VueRouter({
  routes: [
    ...
    { path: '*', component: NotFound }
  ]
})

export default router
```

九、Vue路由-模式设置

1.问题

路由的路径看起来不自然, 有#, 能否切成真正路径形式?

- hash路由(默认) 例如: <http://localhost:8080/#/home>
- history路由(常用) 例如: <http://localhost:8080/home> (以后上线需要服务器端支持, 开发环境webpack给规避掉了history模式的问题)

2.语法

```
const router = new VueRouter({
  mode: 'history', //默认是hash
  routes: []
})
```

十、程式化导航-两种路由跳转方式

1.问题

点击按钮跳转如何实现?



黑马程序员

搜索一下

热门搜索: [黑马程序员](#) [前端培训](#) [如何成为前端大生](#)

2.方案

程式导航：用JS代码来进行跳转

3.语法

两种语法：

- path 路径跳转（简易方便）
- name 命名路由跳转 (适合 path 路径长的场景)

4.path路径跳转语法

特点：简易方便

```
//简单写法
this.$router.push('路由路径')

//完整写法
this.$router.push({
  path: '路由路径'
})
```

5.代码演示 path跳转方式

6.name命名路由跳转

特点：适合 path 路径长的场景

语法：

- 路由规则，必须配置name配置项

```
{ name: '路由名', path: '/path/xxx', component: XXX },
```

- 通过name来进行跳转

```
this.$router.push({  
  name: '路由名'  
})
```

7.代码演示通过name命名路由跳转

8.总结

编程式导航有几种跳转方式？

十一、编程式导航-path路径跳转传参

1.问题

点击搜索按钮，跳转需要把文本框中输入的内容传到下一个页面如何实现？



2.两种传参方式

1.查询参数

2.动态路由传参

3.传参

两种跳转方式，对于两种传参方式都支持：

① path 路径跳转传参

② name 命名路由跳转传参

4.path路径跳转传参（query传参）

```
//简单写法
this.$router.push('/路径?参数名1=参数值1&参数2=参数值2')
//完整写法
this.$router.push({
  path: '/路径',
  query: {
    参数名1: '参数值1',
    参数名2: '参数值2'
  }
})
```

接受参数的方式依然是：\$route.query.参数名

5.path路径跳转传参（动态路由传参）

```
//简单写法
this.$router.push('/路径/参数值')
//完整写法
this.$router.push({
  path: '/路径/参数值'
})
```

接受参数的方式依然是：\$route.params.参数值

注意： path不能配合params使用

十二、程式导航-name命名路由传参

1.name 命名路由跳转传参 (query传参)

```
this.$router.push({
  name: '路由名字',
  query: {
    参数名1: '参数值1',
    参数名2: '参数值2'
  }
})
```


2.name 命名路由跳转传参 (动态路由传参)

```
this.$router.push({
  name: '路由名字',
  params: {
    参数名: '参数值',
  }
})
```

3.总结

编程式导航，如何跳转传参？

1.path路径跳转

- query传参

```
this.$router.push('/路径?参数名1=参数值1&参数2=参数值2')
this.$router.push({
  path: '/路径',
  query: {
    参数名1: '参数值1',
    参数名2: '参数值2'
  }
})
```

- 动态路由传参

```
this.$router.push('/路径/参数值')
this.$router.push({
  path: '/路径/参数值'
})
```

2.name命名路由跳转

- query传参

```
this.$router.push({
  name: '路由名字',
  query: {
    参数名1: '参数值1',
    参数名2: '参数值2'
  }
})
```

- 动态路由传参 (需要配动态路由)

```
this.$router.push({
  name: '路由名字',
  params: {
    参数名: '参数值',
  }
})
```

十三、面经基础版-案例效果分析

1.面经效果演示

2.功能分析

- 通过演示效果发现，主要的功能页面有两个，一个是**列表页**，一个是**详情页**，并且在列表页点击时可以跳转到详情页
- 底部导航可以来回切换，并且切换时，只有上面的主题内容在动态渲染



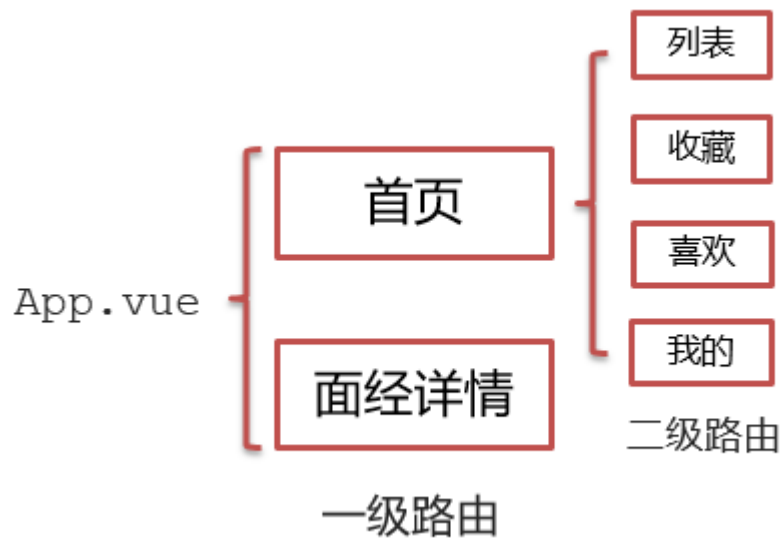
3.实现思路分析：配置路由+功能实现

1.配置路由

- 首页和面经详情页，两个一级路由
- 首页内嵌套4个可切换的页面（嵌套二级路由）

2.实现功能

- 首页请求渲染
- 跳转传参到详情页，详情页动态渲染
- 组件缓存，性能优化



十四、面经基础版-一级路由配置

- 1.把文档中准备的素材拷贝到项目中
- 2.针对router/index.js文件 进行一级路由配置

```
...
import Layout from '@views/Layout.vue'
import ArticleDetail from '@views/ArticleDetail.vue'
...

const router = new VueRouter({
  routes: [
    {
      path: '/',
      component: Layout
    },
    {
      path: '/detail',
      component: ArticleDetail
    }
  ]
})
```

十五、面经基础版-二级路由配置

二级路由也叫嵌套路由，当然也可以嵌套三级、四级...

1.使用场景

当在页面中点击链接跳转，只是部分内容切换时，我们可以使用嵌套路由

2.语法

- 在一级路由下，配置children属性即可
- 配置二级路由的出口

1.在一级路由下，配置children属性

注意:一级的路由path 需要加 `/` 二级路由的path不需要加 `/`

```
const router = new VueRouter({
  routes: [
    {
      path: '/',
      component: Layout,
      children:[
        //children中的配置项 跟一级路由中的配置项一模一样
        {path:'xxx',component:xxx.vue},
        {path:'xxx',component:xxx.vue},
      ]
    }
  ]
})
```

技巧：二级路由应该配置到哪个一级路由下呢？

这些二级路由对应的组件渲染到哪个一级路由下，children就配置到哪个路由下边

2.配置二级路由的出口

注意：配置了嵌套路由，一定配置对应的路由出口，否则不会渲染出对应的组件

Layout.vue

```
<template>
  <div class="h5-wrapper">
    <div class="content">
      <router-view></router-view>
    </div>
    ....
  </div>
</template>
```

3.代码实现

router/index.js

```
...
import Article from '@views/Article.vue'
import Collect from '@views/Collect.vue'
import Like from '@views/Like.vue'
import User from '@views/User.vue'
...

const router = new VueRouter({
  routes: [
    {
      path: '/',
      component: Layout,
      redirect: '/article',
      children:[
        {
          path:'/article',
          component:Article
        },
        {
          path:'/collect',
          component:Collect
        },
        {
          path:'/like',
          component:Like
        },
        {
          path:'/user',
          component:User
        }
      ]
    },
    ....
  ]
})
```

Layout.vue

```
<template>
  <div class="h5-wrapper">
    <div class="content">
      <!-- 内容部分 -->
      <router-view></router-view>
    </div>
    <nav class="tabbar">
      <a href="#/article">面经</a>
      <a href="#/collect">收藏</a>
      <a href="#/like">喜欢</a>
      <a href="#/user">我的</a>
    </nav>
  </div>
```

```
    </nav>
  </div>
</template>
```

十六、面经基础版-二级导航高亮

1.实现思路

- 将a标签替换成 组件，配置to属性，不用加 #
- 结合高亮类名实现高亮效果 (推荐模糊匹配：router-link-active)

2.代码实现

Layout.vue

```
....
<nav class="tabbar">
  <router-link to="/article">面经</router-link>
  <router-link to="/collect">收藏</router-link>
  <router-link to="/like">喜欢</router-link>
  <router-link to="/user">我的</router-link>
</nav>

<style>
  a.router-link-active {
    color: orange;
  }
</style>
```

十七、面经基础版-首页请求渲染

1.步骤分析

- 1.安装axios
- 2.看接口文档，确认请求方式，请求地址，请求参数
- 3.created中发送请求，获取数据，存储到data中
- 4.页面动态渲染

2.代码实现

1.安装axios

```
yarn add axios  npm i axios
```

2.接口文档

请求地址：<https://mock.boxuegu.com/mock/3083/articles>

请求方式：get

3.created中发送请求，获取数据，存储到data中

```
data() {  
  return {  
    articellList: [],  
  }  
},  
async created() {  
  const { data: { result: { rows } } } = await  
    axios.get('https://mock.boxuegu.com/mock/3083/articles')  
  this.articellList = rows  
},
```

4.页面动态渲染

```
<template>  
  <div class="article-page">  
    <div class="article-item" v-for="item in articellList" :key="item.id">  
      <div class="head">  
          
        <div class="con">  
          <p class="title">{{ item.stem }}</p>  
          <p class="other">{{ item.creatorName }} | {{ item.createdAt }}</p>  
        </div>  
      </div>  
      <div class="body">  
        {{item.content}}  
      </div>  
      <div class="foot">点赞 {{item.likeCount}} | 浏览 {{item.views}}</div>  
    </div>  
  </div>  
</template>
```

十八、面经基础版-查询参数传参

1.说明

跳转详情页需要把当前点击的文章id传给详情页，获取数据

- 查询参数传参 `this.$router.push('/detail?参数1=参数值&参数2=参数值')`
- 动态路由传参 先改造路由 在传参 `this.$router.push('/detail/参数值')`

2.查询参数传参实现

Article.vue

```
<template>
  <div class="article-page">
    <div class="article-item"
      v-for="item in articellist" :key="item.id"
      @click="$router.push(`/detail?id=${item.id}`)">
      ...
    </div>
  </div>
</template>
```

ArticleDetail.vue

```
created(){
  console.log(this.$route.query.id)
}
```

十九、面经基础版-动态路由传参

1.实现步骤

- 改造路由
- 动态传参
- 在详情页获取参数

2.代码实现

改造路由

router/index.js


```
...
{
  path: '/detail/:id',
  component: ArticleDetail
}
```

Article.vue

```
<div class="article-item"
  v-for="item in articellList" :key="item.id"
  @click="$router.push(`/detail/${item.id}`)">
  ....
</div>
```

ArticleDetail.vue

```
created(){
  console.log(this.$route.params.id)
}
```

3.额外优化功能点-点击回退跳转到上一页

ArticleDetail.vue

```
<template>
  <div class="article-detail-page">
    <nav class="nav"><span class="back" @click="$router.back()">&lt;</span> 面经详情</nav>
    ....
  </div>
</template>
```

二十、面经基础版-详情页渲染

1.实现步骤分析

- 导入axios
- 查看接口文档
- 在created中发送请求
- 页面动态渲染

2.代码实现

接口文档

请求地址: <https://mock.boxuegu.com/mock/3083/articles/:id>

请求方式: get

在created中发送请求

```
data() {
  return {
    articleDetail:{}
  }
},
async created() {
  const id = this.$route.params.id
  const {data:{result}} = await axios.get(
    `https://mock.boxuegu.com/mock/3083/articles/${id}`
  )
  this.articleDetail = result
},
```

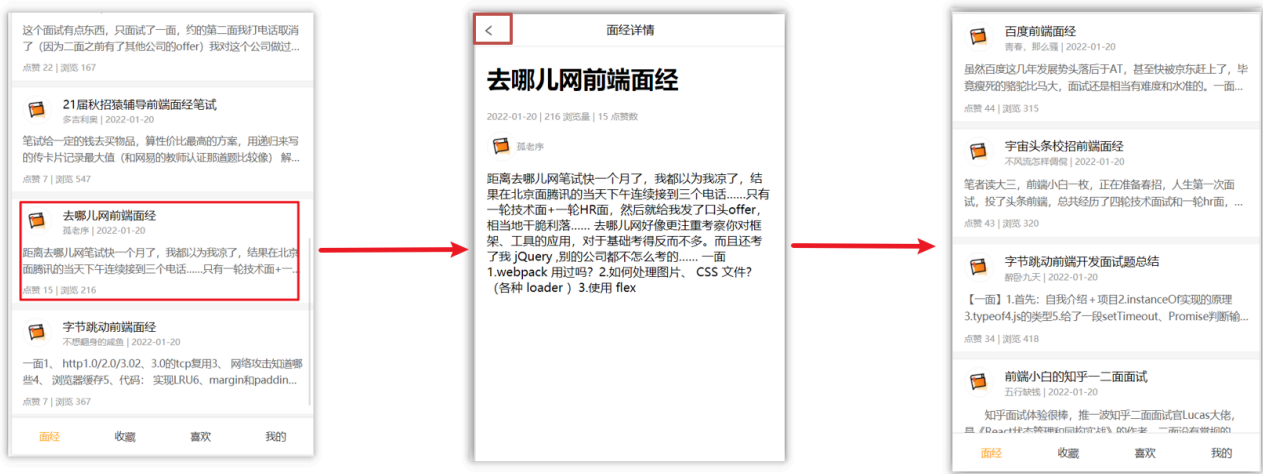
页面动态渲染

```
<template>
  <div class="article-detail-page">
    <nav class="nav">
      <span class="back" @click="$router.back()">&lt;</span> 面经详情
    </nav>
    <header class="header">
      <h1>{{articleDetail.stem}}</h1>
      <p>{{articleDetail.createAt}} | {{articleDetail.views}} 浏览量 | {{articleDetail.likeCount}}
      点赞数</p>
      <p>
        
        <span>{{articleDetail.creatorName}}</span>
      </p>
    </header>
    <main class="body">
      {{articleDetail.content}}
    </main>
  </div>
</template>
```

二十一、面经基础版-缓存组件

1.问题

从面经列表 点到 详情页，又点返回，数据重新加载了 → 希望回到原来的位置



2.原因

当路由被跳转后，原来所看到的组件就被销毁了（会执行组件内的beforeDestroy和destroyed生命周期钩子），重新返回后组件又被重新创建了（会执行组件内的beforeCreate,created,beforeMount,Mounted生命周期钩子），所以数据被加载了

3.解决方案

利用keep-alive把原来的组件给缓存下来

4.什么是keep-alive

keep-alive 是 Vue 的内置组件，当它包裹动态组件时，会缓存不活动的组件实例，而不是销毁它们。

keep-alive 是一个抽象组件：它自身不会渲染成一个 DOM 元素，也不会出现在父组件中。

优点：

在组件切换过程中把切换出去的组件保留在内存中，防止重复渲染DOM，

减少加载时间及性能消耗，提高用户体验性。

App.vue

```
<template>
  <div class="h5-wrapper">
    <keep-alive>
      <router-view></router-view>
    </keep-alive>
  </div>
</template>
```

问题：

缓存了所有被切换的组件

5.keep-alive的三个属性

- ① include ： 组件名数组，只有匹配的组件**会被缓存**
- ② exclude ： 组件名数组，任何匹配的组件都**不会被缓存**
- ③ max ： 最多可以**缓存多少**组件实例

App.vue

```
<template>
  <div class="h5-wrapper">
    <keep-alive :include="['LayoutPage']">
      <router-view></router-view>
    </keep-alive>
  </div>
</template>
```

6.额外的两个生命周期钩子

keep-alive的使用会触发两个生命周期函数

activated 当组件被激活（使用）的时候触发 → 进入这个页面的时候触发

deactivated 当组件不被使用的时候触发 → 离开这个页面的时候触发

组件**缓存后**就**不会执行**组件的**created, mounted, destroyed** 等钩子了

所以其提供了**activated** 和**deactivated**钩子，帮我们实现业务需求。

7.总结

- 1.keep-alive是什么
- 2.keep-alive的优点
- 3.keep-alive的三个属性 (了解)
- 4.keep-alive的使用会触发两个生命周期函数(了解)

二十二、VueCli 自定义创建项目

- 1.安装脚手架 (已安装)

```
npm i @vue/cli -g
```

- 2.创建项目

```
vue create hm-exp-mobile
```

- 选项

```
Vue CLI v5.0.8
? Please pick a preset:
  Default ([Vue 3] babel, eslint)
  Default ([Vue 2] babel, eslint)
> Manually select features    选自定义
```

- 手动选择功能

```
? Check the features needed for your project: (Press <space> to select, <a> to toggle all, <i> to
and <enter> to proceed)
(*) Babel → es6 转 es3
( ) TypeScript
( ) Progressive Web App (PWA) Support
(*) Router → 路由
( ) Vuex
> (*) CSS Pre-processors → less/scss
(*) Linter / Formatter → eslint 校验代码格式
( ) Unit Testing
( ) E2E Testing
```

- 选择vue的版本

```
3.x
> 2.x
```

- 是否使用history模式

```
? Use history mode for router? (Requires proper server setup for index fallback in production) (Y/n) n
```

是否使用history模式，选择n，，代表使用hash模式

- 选择css预处理

```
? Pick a CSS pre-processor (PostCSS, Autoprefixer and CSS Modules are supported)
> Sass/SCSS (with dart-sass)
Less
Stylus
```

- 选择eslint的风格（eslint 代码规范的检验工具，检验代码是否符合规范）
- 比如：const age = 18; => 报错！多加了分号！后面有工具，一保存，全部格式化成最规范的样子

```
? Pick a linter / formatter config:
ESLint with error prevention only
ESLint + Airbnb config
> ESLint + Standard config
ESLint + Prettier
```

标准风格

- 选择校验的时机（直接回车）

```
? Pick additional lint features: (Press <space> to select, <a> to toggle all, <enter> to proceed)
> (*) Lint on save
( ) Lint and fix on commit
```

正常开启，保存校验即可

- 选择配置文件的生成方式（直接回车）

```
? Where do you prefer placing config for Babel, ESLint, etc.?
> In dedicated config files
In package.json
```

把配置文件生成到单独的文件中

- 是否保存预设，下次直接使用？ => 不保存，输入 N

```
Vue CLI v5.0.4
? Please pick a preset: Manually select features
? Check the features needed for your project: Babel, Router, CSS Pre-processors, Linter
? Choose a version of Vue.js that you want to start the project with 2.x
? Use history mode for router? (Requires proper server setup for index fallback in production) No
? Pick a CSS pre-processor (PostCSS, Autoprefixer and CSS Modules are supported by default): Less
? Pick a linter / formatter config: Standard
? Pick additional lint features: Lint on save
? Where do you prefer placing config for Babel, ESLint, etc.? In dedicated config files
? Save this as a preset for future projects? (y/N) N
```

是否保存此预设，下次直接用上面的选项 => N（不保存，我们每次自定义，会有变化）

- 等待安装，项目初始化完成

```
success Saved lockfile.
Done in 6.28s.
❑ Running completion hooks...

❑ Generating README.md...

❑ Successfully created project hm-vant-h5.
❑ Get started with the following commands:

$ cd hm-vant-h5
$ yarn serve
```

- 启动项目

```
npm run serve
```

二十三、ESlint代码规范及手动修复

代码规范：一套写代码的约定规则。例如：赋值符号的左右是否需要空格？一句结束是否是要加;? ...

没有规矩不成方圆

ESLint:是一个代码检查工具，用来检查你的代码是否符合指定的规则(你和你的团队可以自行约定一套规则)。在创建项目时，我们使用的是 **JavaScript Standard Style** 代码风格的规则。

1.JavaScript Standard Style 规范说明

建议把：<https://standardjs.com/rules-zhcn.html> 看一遍，然后在写的时候，遇到错误就查询解决。

下面是这份规则中的一小部分：

- 字符串使用单引号 - 需要转义的地方除外
- 无分号 - 这没什么不好。不骗你!
- 关键字后加空格 `if (condition) { ... }`
- 函数名后加空格 `function name (arg) { ... }`
- 坚持使用全等 `===` 摒弃 `==` 一但在需要检查 `null || undefined` 时可以使用 `obj == null`
-

2.代码规范错误

如果你的代码不符合standard的要求，eslint会跳出来刀子嘴，豆腐心地提示你。

下面我们在main.js中随意做一些改动：添加一些空行，空格。

```
import Vue from 'vue'
import App from './App.vue'

import './styles/index.less'
import router from './router'
Vue.config.productionTip = false

new Vue ( {
  render: h => h(App),
  router
}).$mount('#app')
```

按下保存代码之后：

你将会看在控制台中输出如下错误：

D:\demo\demo_toutiao_pc\src\main.js ➡ 1. 有代码格式错误的文件

8:5	error	Unexpected newline between function name and paren	func-call-spacing
8:10	error	There should be no space after this paren	space-in-parens
12:1	error	Too many blank lines at the end of file. Max of 0 allowed	no-multiple-empty-lines

➡ 2. 具体错误的位置·行:列·

➡ 3. 错误具体描述

➡ 4. 错误代码

✖ 3 problems (3 errors, 0 warnings)
3 errors and 0 warnings potentially fixable with the `--fix` option.

eslint 是来帮助你的。心态要好，有错，就改。

3.手动修正

根据错误提示来一项一项手动修正。

如果你不认识命令行中的语法报错是什么意思，你可以根据错误代码（func-call-spacing, space-in-parens,.....）去 ESLint 规则列表中查找其具体含义。

打开 [ESLint 规则表](#)，使用页面搜索（Ctrl + F）这个代码，查找对该规则的一个释义。

semi

Require or disallow semicolons instead of ASI



semi-spacing

Enforce consistent spacing before and after semicolons



semi-style

Enforce location of semicolons



space-before-blocks

Enforce consistent spacing before blocks



space-before-function-paren

Enforce consistent spacing before `function` definition opening parenthesis



space-in-parens

Enforce consistent spacing inside parentheses



space-infix-ops

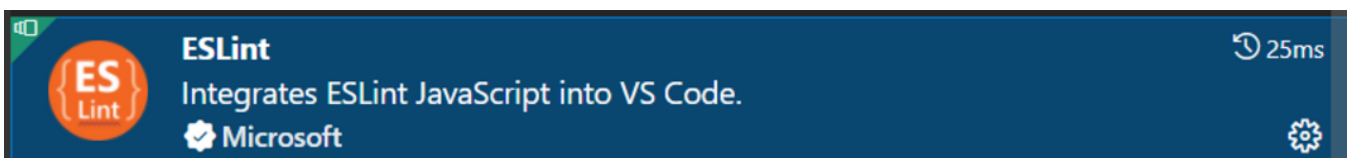
Require spacing around infix operators



二十四、通过eslint插件来实现自动修正

1. eslint会自动高亮错误显示
2. 通过配置，eslint会自动帮助我们修复错误

- 如何安装



- 如何配置

```
// 当保存的时候，eslint自动帮我们修复错误
"editor.codeActionsOnSave": {
  "source.fixAll": true
},
// 保存代码，不自动格式化
"editor.formatOnSave": false
```

- 注意：eslint的配置文件必须在根目录下，这个插件才能生效。打开项目必须以根目录打开，一次打开一个项目

- 注意：使用了eslint校验之后，把vscode带的那些格式化工具全禁用了 Beatify

settings.json 参考

```
{
  "window.zoomLevel": 2,
  "workbench.iconTheme": "vscode-icons",
  "editor.tabSize": 2,
  "emmet.triggerExpansionOnTab": true,
  // 当保存的时候，eslint自动帮我们修复错误
  "editor.codeActionsOnSave": {
    "source.fixAll": true
  },
  // 保存代码，不自动格式化
  "editor.formatOnSave": false
}
```

一、Vuex 概述

目标：明确 **Vuex** 是什么，应用场景以及优势

1.是什么

Vuex 是一个 Vue 的 状态管理工具，状态就是数据。

大白话：Vuex 是一个插件，可以帮我们管理 Vue 通用的数据 (多组件共享的数据)。例如：购物车数据 个人信息数

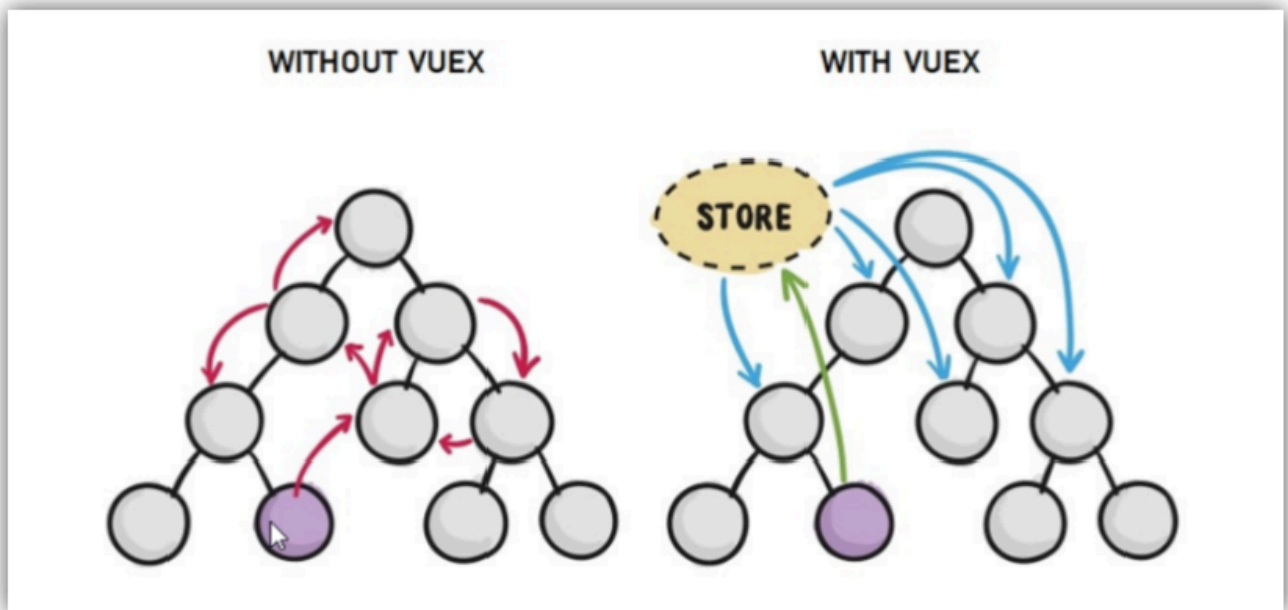
2.使用场景

- 某个状态 在 很多个组件 来使用 (个人信息)
- 多个组件 共同维护 一份数据 (购物车)



3.优势

- 共同维护一份数据，**数据集中化管理**
- **响应式变化**
- 操作简洁 (vuex提供了一些辅助函数)



4.注意：

官方原文：

- 不是所有的场景都适用于vuex，只有在必要的时候才使用vuex
- 使用了vuex之后，会附加更多的框架中的概念进来，增加了项目的复杂度（数据的操作更便捷，数据的流动更清晰）

Vuex就像《近视眼镜》，你自然会知道什么时候需要用它~

二、需求: 多组件共享数据

目标：基于脚手架创建项目，构建 vuex 多组件数据共享环境

根组件 100

Son1 子组件

从vuex中获取的值: 100

值 + 1

Son2 子组件

从vuex中获取的值:100

值 - 1

效果是三个组件共享一份数据:

- 任意一个组件都可以修改数据
- 三个组件的数据是同步的

1.创建项目

```
vue create vuex-demo
```

2.创建三个组件, 目录如下

```
| -components  
| --Son1.vue  
| --Son2.vue  
| -App.vue
```

3.源代码如下

App.vue 在入口组件中引入 Son1 和 Son2 这两个子组件

```
<template>  
  <div id="app">  
    <h1>根组件</h1>  
    <input type="text">  
    <Son1></Son1>  
    <hr>  
    <Son2></Son2>
```

```

    </div>
  </template>

  <script>
  import Son1 from './components/Son1.vue'
  import Son2 from './components/Son2.vue'

  export default {
    name: 'app',
    data: function () {
      return {

      }
    },
    components: {
      Son1,
      Son2
    }
  }
  </script>

  <style>
  #app {
    width: 600px;
    margin: 20px auto;
    border: 3px solid #ccc;
    border-radius: 3px;
    padding: 10px;
  }
  </style>

```

main.js

```

import Vue from 'vue'
import App from './App.vue'

Vue.config.productionTip = false

new Vue({
  render: h => h(App)
}).$mount('#app')

```

Son1.vue

```

<template>
  <div class="box">
    <h2>Son1 子组件</h2>
    从vuex中获取的值: <label></label>
    <br>
    <button>值 + 1</button>
  </div>
</template>

```

```

<script>
export default {
  name: 'Son1Com'
}
</script>

<style lang="css" scoped>
.box{
  border: 3px solid #ccc;
  width: 400px;
  padding: 10px;
  margin: 20px;
}
h2 {
  margin-top: 10px;
}
</style>

```

Son2.vue

```

<template>
  <div class="box">
    <h2>Son2 子组件</h2>
    从vuex中获取的值:<label></label>
    <br />
    <button>值 - 1</button>
  </div>
</template>

<script>
export default {
  name: 'Son2Com'
}
</script>

<style lang="css" scoped>
.box {
  border: 3px solid #ccc;
  width: 400px;
  padding: 10px;
  margin: 20px;
}
h2 {
  margin-top: 10px;
}
</style>

```

三、vuex 的使用 - 创建仓库



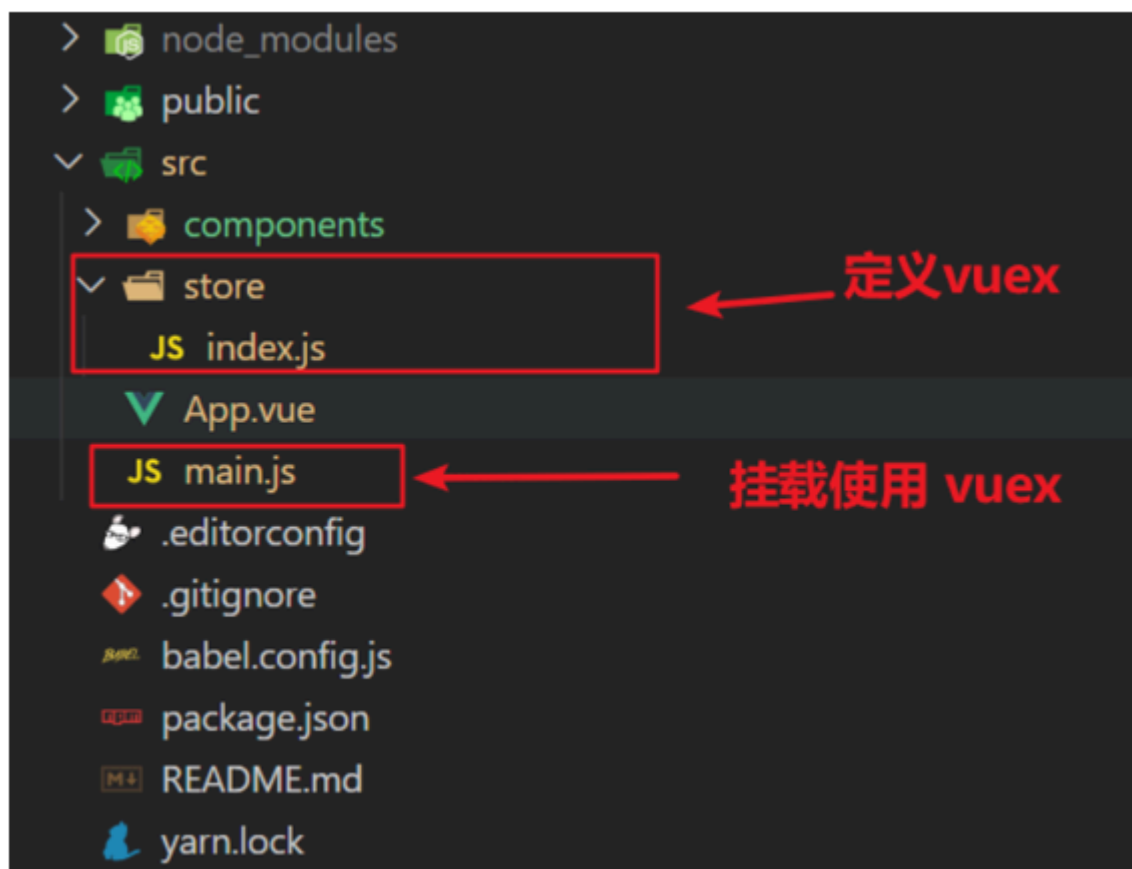
1.安装 vuex

安装vuex与vue-router类似，vuex是一个独立存在的插件，如果脚手架初始化没有选 vuex，就需要额外安装。

```
yarn add vuex@3 或者 npm i vuex@3
```

2.新建 `store/index.js` 专门存放 vuex

为了维护项目目录的整洁，在src目录下新建一个store目录其下放置一个index.js文件。(和 `router/index.js` 类似)



3.创建仓库 `store/index.js`

```
// 导入 vue
import Vue from 'vue'
// 导入 vuex
import Vuex from 'vuex'
// vuex也是vue的插件，需要use一下，进行插件的安装初始化
Vue.use(Vuex)

// 创建仓库 store
const store = new Vuex.Store()

// 导出仓库
export default store
```

4 在 main.js 中导入挂载到 Vue 实例上

```
import Vue from 'vue'
import App from './App.vue'
import store from './store'

Vue.config.productionTip = false

new Vue({
  render: h => h(App),
  store
}).$mount('#app')
```

此刻起, 就成功创建了一个 空仓库!!

5.测试打印Vuex

App.vue

```
created(){
  console.log(this.$store)
}
```

四、核心概念 - state 状态

1.目标

明确如何给仓库 提供 数据，如何 使用 仓库的数据

2.提供数据

State提供唯一的公共数据源，所有共享的数据都要统一放到Store中的State中存储。

打开项目中的store.js文件，在state对象中可以添加我们要共享的数据。

```
// 创建仓库 store
const store = new Vuex.Store({
  // state 状态，即数据，类似于vue组件中的data，
  // 区别：
  // 1.data 是组件自己的数据，
  // 2.state 中的数据整个vue项目的组件都能访问到
  state: {
    count: 101
  }
})
```

3.访问Vuex中的数据

问题: 如何在组件中获取count?

1. 通过\$store直接访问 → {{ \$store.state.count }}
2. 通过辅助函数mapState 映射计算属性 → {{ count }}

4.通过\$store访问的语法

获取 store:

- 1.Vue模板中获取 `this.$store`
- 2.js文件中获取 `import 导入 store`

模板中: {{ \$store.state.xxx }}

组件逻辑中: `this.$store.state.xxx`

JS模块中: `store.state.xxx`

5.代码实现

5.1模板中使用

组件中可以使用 `$store` 获取到vuex中的store对象实例，可通过state属性属性获取count，如下

```
<h1>state的数据 - {{ $store.state.count }}</h1>
```

5.2组件逻辑中使用

将state属性定义在计算属性中 <https://vuex.vuejs.org/zh/guide/state.html>

```
<h1>state的数据 - {{ count }}</h1>

// 把state中数据，定义在组件内的计算属性中
computed: {
  count () {
    return this.$store.state.count
  }
}
```

5.3 js文件中使用

```
//main.js

import store from "@store"

console.log(store.state.count)
```

每次都像这样一个一个的提供计算属性, 太麻烦了, 我们有没有简单的语法帮我们获取state中的值呢?

五、通过辅助函数 - mapState获取 state中的数据

mapState是辅助函数，帮助我们z把store中的数据映射到 组件的计算属性中, 它属于一种方便的用法

用法：



1. 第一步：导入mapState (mapState是vuex中的一个函数)

```
import { mapState } from 'vuex'
```

2. 第二步：采用数组形式引入state属性

```
mapState(['count'])
```

上面代码的最终得到的是 类似于

```
count () {
  return this.$store.state.count
}
```

3.第三步：利用展开运算符将导出的状态映射给计算属性

```
computed: {  
  ...mapState(['count'])  
}
```

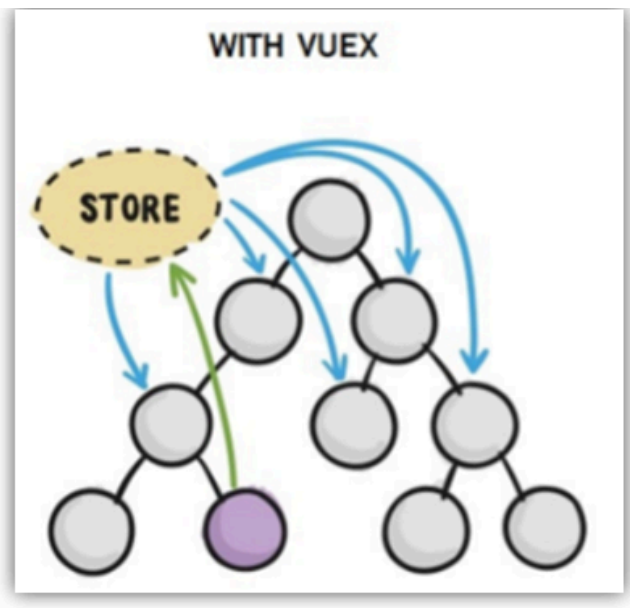
```
<div> state的数据: {{ count }}</div>
```

六、开启严格模式及Vuex的单项数据流

1.目标

明确 vuex 同样遵循单向数据流，组件中不能直接修改仓库的数据

2.直接在组件中修改Vuex中state的值



Son1.vue

```
button @click="handleAdd">值 + 1</button>  
  
methods:{  
  handleAdd (n) {  
    // 错误代码(vue默认不会监测，监测需要成本)  
    this.$store.state.count++  
    // console.log(this.$store.state.count)  
  },  
}
```

3.开启严格模式

通过 `strict: true` 可以开启严格模式,开启严格模式后,直接修改state中的值会报错

state数据的修改只能通过mutations, 并且mutations必须是同步的

```
// 创建仓库
const store = new Vuex.Store({
  // 开启严格模式
  strict: true,
  // 通过 state 可以提供数据
  state: {
    title: '仓库大标题',
    count: 100
  }
})
```

七、核心概念-mutations

1.定义mutations

```
const store = new Vuex.Store({
  state: {
    count: 0
  },
  // 定义mutations
  mutations: {

  }
})
```

2.格式说明

mutations是一个对象,对象中存放修改state的方法

```
mutations: {
  // 方法里参数 第一个参数是当前store的state属性
  // payload 载荷 运输参数 调用mutations的时候 可以传递参数 传递载荷
  addCount (state) {
    state.count += 1
  },
}
```

3.组件中提交 mutations

```
this.$store.commit('addCount')
```

4.练习

- 1.在mutations中定义个点击按钮进行 +5 的方法
- 2.在mutations中定义个点击按钮进行 改变title 的方法
- 3.在组件中调用mutations修改state中的值

5.总结

通过mutations修改state的步骤

- 1.定义 mutations 对象，对象中存放修改 state 的方法
- 2.组件中提交调用 mutations(通过\$store.commit('mutations的方法名'))

八、带参数的 mutations

1.目标：

掌握 mutations 传参语法

2.语法

看下面这个案例，每次点击不同的按钮，加的值都不同，每次都要定义不同的mutations处理吗？



提交 mutation 是可以传递参数的 `this.$store.commit('xxx', 参数)`

2.1 提供mutation函数（带参数）

```
mutations: {  
  ...  
  addCount (state, count) {  
    state.count = count  
  }  
},
```

2.2 提交mutation

```
handle ( ) {  
  this.$store.commit('addCount', 10)  
}
```

小tips: 提交的参数只能是一个, 如果有多个参数要传, 可以传递一个对象

```
this.$store.commit('addCount', {  
  count: 10  
})
```

九、练习-mutations的减法功能

Son2 子组件

从vuex中获取的值:100

值 - 1

值 - 5

值 - 10

1.步骤

封装mutation 函数



页面中commit调用

2.代码实现

Son2.vue

```

<button @click="subCount(1)">值 - 1</button>
<button @click="subCount(5)">值 - 5</button>
<button @click="subCount(10)">值 - 10</button>

```

```

export default {
  methods: {
    subCount (n) {
      this.$store.commit('addCount', n)
    },
  }
}

```

store/index.js

```

mutations: {
  subCount (state, n) {
    state.count -= n
  },
}

```

十、练习-Vuex中的值和组件中的input双向绑定

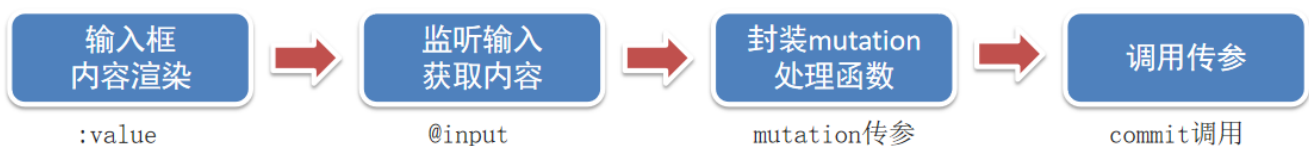
1.目标

实时输入，实时更新，巩固 mutations 传参语法

根组件 - 仓库大标题 - 102



2.实现步骤



3.代码实现

App.vue

```
<input :value="count" @input="handleInput" type="text">

export default {
  methods: {
    handleInput (e) {
      // 1. 实时获取输入框的值
      const num = +e.target.value
      // 2. 提交mutation, 调用mutation函数
      this.$store.commit('changeCount', num)
    }
  }
}
```

store/index.js

```
mutations: {
  changeCount (state, newCount) {
    state.count = newCount
  }
},
```

十一、辅助函数- mapMutations

mapMutations和mapState很像，它把位于mutations中的方法提取了出来，我们可以将它导入

```
import { mapMutations } from 'vuex'
methods: {
  ...mapMutations(['addCount'])
}
```

上面代码的含义是将mutations的方法导入了methods中，等价于

```
methods: {
  // commit(方法名, 载荷参数)
  addCount () {
    this.$store.commit('addCount')
  }
}
```

此时，就可以直接通过this.addCount调用了

```
<button @click="addCount">值+1</button>
```

但是请注意：Vuex中mutations中要求不能写异步代码，如果有异步的ajax请求，应该放置在actions中

十二、核心概念 - actions

state是存放数据的，mutations是同步更新数据 (便于监测数据的变化, 更新视图等, 方便于调试工具查看变化)，actions则负责进行异步操作

说明：mutations必须是同步的

需求：一秒钟之后, 要给一个数 去修改state



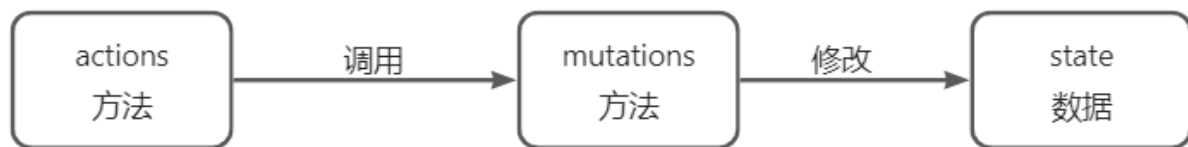
1.定义actions

```
mutations: {
  changeCount (state, newCount) {
    state.count = newCount
  }
}

actions: {
  setAsyncCount (context, num) {
    // 一秒钟，给一个数，去修改 num
    setTimeout(() => {
      context.commit('changeCount', num)
    }, 1000)
  }
},
```

2.组件中通过dispatch调用

```
setAsyncCount () {
  this.$store.dispatch('setAsyncCount', 666)
}
```



1. actions中的方法，调用mutations中的方法
2. mutations中的方法，修改state中的数据

十三、辅助函数 -mapActions

- 1.目标：掌握辅助函数 mapActions，映射方法

mapActions 是把位于 actions中的方法提取了出来，映射到组件methods中

Son2.vue

```
import { mapActions } from 'vuex'
methods: {
  ...mapActions(['changeCountAction'])
}

//mapActions映射的代码 本质上是以下代码的写法
//methods: {
//  changeCountAction (n) {
//    this.$store.dispatch('changeCountAction', n)
//  },
//}
```

直接通过 this.方法 就可以调用

```
<button @click="changeCountAction(200)">异步</button>
```

十四、核心概念 - getters

除了state之外，有时我们还需要从state中筛选出符合条件的一些数据，这些数据是依赖state的，此时会用到 getters

例如，state中定义了list，为1-10的数组，

```
state: {
  list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
}
```

组件中，需要显示所有大于5的数据，正常的方式，是需要list在组件中进行再一步的处理，但是getters可以帮助我们实现它

1.定义getters

```
getters: {  
  // getters函数的第一个参数是 state  
  // 必须要有返回值  
  filterList: state => state.list.filter(item => item > 5)  
}
```

2.使用getters

2.1原始方式-\$store

```
<div>{{ $store.getters.filterList }}</div>
```

2.2辅助函数 - mapGetters

```
computed: {  
  ...mapGetters(['filterList'])  
}
```

```
<div>{{ filterList }}</div>
```

十五、使用小结

state 作用：存数据 定义： state: { age: 20, username: 'zhangsan' } 组件中直接使用： <p>{{ \$store.state.age }}</p> 组件中借助于辅助方法使用： <p>{{ age }}</p> import { mapState } from 'vuex' computed: { ...mapState(['age']) }	getters 作用：计算属性 定义： getters: { abc(state) { return '计算的结果' } } 组件中直接使用： <p>{{ \$store.getters.abc }}</p> 组件中借助于辅助方法使用： <p>{{ abc }}</p> import { mapGetters } from 'vuex' computed: { ...mapGetters(['abc']) }	mutations 作用：更新state数据的唯一途径 定义： mutations: { updateAge(state, val) { state.age = val } } 组件中直接使用： <button @click="\$store.commit('updateAge', 30)">xxx</button> 组件中借助于辅助方法使用： <button @click="updateAge(40)">xxx</button> import { mapMutations } from 'vuex' methods: { ...mapMutations(['updateAge']) }	actions 作用：放异步方法 定义： actions: { abc(store, val) { setTimeout(() => { store.commit('updateAge', val) }, 2000) } } 组件中直接使用： <button @click="\$store.dispatch('abc', 30)">xxx</button> 组件中借助于辅助方法使用： <button @click="abc(40)">xxx</button> import { mapActions } from 'vuex' methods: { ...mapActions(['abc']) }
---	--	---	--

十六、核心概念 - module

1.目标

掌握核心概念 module 模块的创建

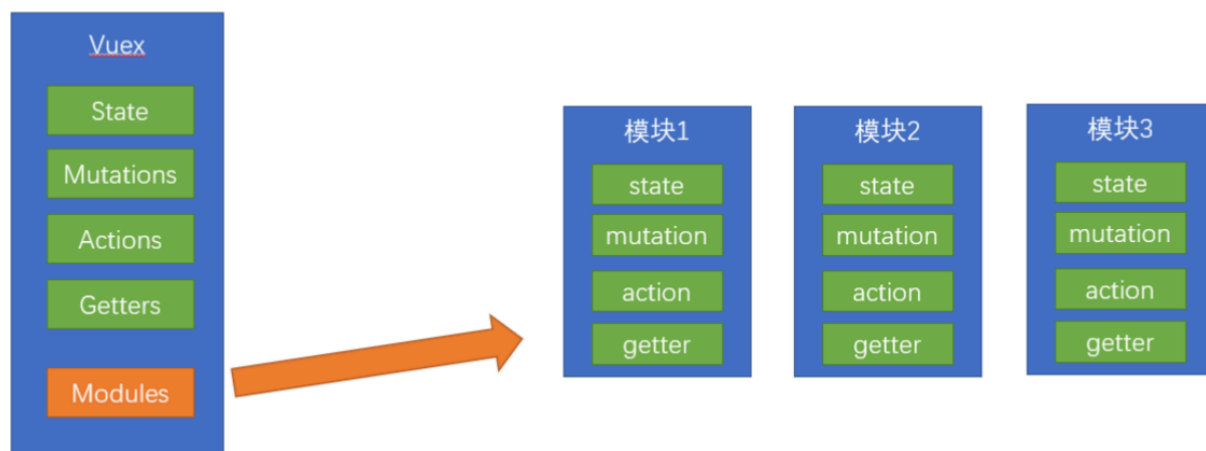
2.问题

由于使用**单一状态树**，应用的所有状态**会集中到一个比较大的对象**。当应用变得非常复杂时，store 对象就有可能变得相当臃肿。

这句话的意思是，如果把所有的状态都放在state中，当项目变得越来越大时，Vuex会变得越来越难以维护

由此，才有了Vuex的模块化

所有的数据，更新，操作都在一起，项目越大，越难维护



3.模块定义 - 准备 state

定义两个模块 `user` 和 `setting`

user中管理用户的信息状态 `userInfo` `modules/user.js`

```
const state = {
  userInfo: {
    name: 'zs',
    age: 18
  }
}

const mutations = {}

const actions = {}

const getters = {}

export default {
  state,
  mutations,
  actions,
  getters
}
```

setting中管理项目应用的 主题色 theme，描述 desc， `modules/setting.js`

```
const state = {
  theme: 'dark'
  desc: '描述真呀真不错'
}

const mutations = {}

const actions = {}

const getters = {}

export default {
  state,
  mutations,
  actions,
  getters
}
```

在 `store/index.js` 文件中的modules配置项中，注册这两个模块

```
import user from './modules/user'
import setting from './modules/setting'

const store = new Vuex.Store({
  modules:{
    user,
    setting
  }
})
```

使用模块中的数据，可以直接通过模块名访问 `$store.state.模块名.xxx` => `$store.state.setting.desc`

也可以通过 `mapState` 映射

十七、获取模块内的state数据

1.目标：

掌握模块中 state 的访问语法

尽管已经分模块了，但其实子模块的状态，还是会挂到根级别的 state 中，属性名就是模块名



2.使用模块中的数据

1. 直接通过模块名访问 `$store.state.模块名.xxx`
2. 通过 `mapState` 映射：
 1. 默认根级别的映射 `mapState(['xxx'])`
 2. 子模块的映射：`mapState('模块名', ['xxx'])` - 需要开启命名空间 `namespaced:true`

`modules/user.js`

```
const state = {
  userInfo: {
    name: 'zs',
    age: 18
  },
  myMsg: '我的数据'
}

const mutations = {
  updateMsg (state, msg) {
    state.myMsg = msg
  }
}

const actions = {}

const getters = {}

export default {
  namespaced: true,
  state,
  mutations,
  actions,
  getters
}
```

3.代码示例

\$store直接访问

```
$store.state.user.userInfo.name
```

mapState辅助函数访问

```
...mapState('user', ['userInfo']),  
...mapState('setting', ['theme', 'desc']),
```

十八、获取模块内的getters数据

1.目标：

掌握模块中 getters 的访问语

2.语法：

使用模块中 getters 中的数据：

1. 直接通过模块名访问 `$store.getters['模块名/xxx ']`
2. 通过 mapGetters 映射
 1. 默认根级别的映射 `mapGetters(['xxx' ])`
 2. 子模块的映射 `mapGetters('模块名', ['xxx'])` - 需要开启命名空间

3.代码演示

modules/user.js

```
const getters = {  
  // 分模块后，state指代子模块的state  
  UpperCaseName (state) {  
    return state.userInfo.name.toUpperCase()  
  }  
}
```

Son1.vue 直接访问getters

```
<!-- 测试访问模块中的getters - 原生 -->  
<div>{{ $store.getters['user/UpperCaseName'] }}</div>
```

Son2.vue 通过命名空间访问

```
computed:{  
  ...mapGetters('user', ['UpperCaseName'])  
}
```


十九、获取模块内的mutations方法

1.目标：

掌握模块中 mutation 的调用语法

2.注意：

默认模块中的 mutation 和 actions 会被挂载到全局，**需要开启命名空间**，才会挂载到子模块。

3.调用方式：

1. 直接通过 store 调用 `$store.commit('模块名/xxx', 额外参数)`
2. 通过 `mapMutations` 映射
 1. 默认根级别的映射 `mapMutations([ 'xxx' ])`
 2. 子模块的映射 `mapMutations('模块名', ['xxx'])` - 需要开启命名空间

4.代码实现

`modules/user.js`

```
const mutations = {
  setUser (state, newUserInfo) {
    state.userInfo = newUserInfo
  }
}
```

`modules/setting.js`

```
const mutations = {
  setTheme (state, newTheme) {
    state.theme = newTheme
  }
}
```

`Son1.vue`

```
<button @click="updateUser">更新个人信息</button>
<button @click="updateTheme">更新主题色</button>

export default {
  methods: {
    updateUser () {
      // $store.commit('模块名/mutation名', 额外传参)
      this.$store.commit('user/setUser', {
        name: 'xiaowang',
        age: 25
      })
    },
    updateTheme () {
```

```
    this.$store.commit('setting/setTheme', 'pink')
  }
}
}
```

Son2.vue

```
<button @click="setUser({ name: 'xiaoli', age: 80 })">更新个人信息</button>
<button @click="setTheme('skyblue')">更新主题</button>

methods:{
  // 分模块的映射
  ...mapMutations('setting', ['setTheme']),
  ...mapMutations('user', ['setUser']),
}
```

二十、获取模块内的actions方法

1.目标:

掌握模块中 action 的调用语法 (同理 - 直接类比 mutation 即可)

2.注意:

默认模块中的 mutation 和 actions 会被挂载到全局，**需要开启命名空间**，才会挂载到子模块。

3.调用语法:

1. 直接通过 store 调用 \$store.dispatch('模块名/xxx', 额外参数)
2. 通过 mapActions 映射
 1. 默认根级别的映射 mapActions(['xxx'])
 2. 子模块的映射 mapActions('模块名', ['xxx']) - 需要开启命名空间

4.代码实现

需求:



modules/user.js

```
const actions = {
  setUserSecond (context, newUserInfo) {
    // 将异步在action中进行封装
    setTimeout(() => {
      // 调用mutation context上下文，默认提交的就是自己模块的action和mutation
      context.commit('setUser', newUserInfo)
    }, 1000)
  }
}
```

Son1.vue 直接通过store调用

```
<button @click="updateUser2">一秒后更新信息</button>

methods:{
  updateUser2 () {
    // 调用action dispatch
    this.$store.dispatch('user/setUserSecond', {
      name: 'xiaohong',
      age: 28
    })
  },
}
```

Son2.vue mapActions映射

```
<button @click="setUserSecond({ name: 'xiaoli', age: 80 })">一秒后更新信息</button>

methods:{
  ...mapActions('user', ['setUserSecond'])
}
```

二十一、Vuex模块化的使用小结

1.直接使用

1. state --> \$store.state. **模块名**.数据项名
2. getters --> \$store.getters['**模块名**/属性名']
3. mutations --> \$store.commit(' **模块名**/方法名', 其他参数)
4. actions --> \$store.dispatch(' **模块名**/方法名', 其他参数)

2.借助辅助方法使用

1.import { mapXxxx, mapXxx } from 'vuex'

computed、methods: {

// ...mapState、...mapGetters放computed中；

// ...mapMutations、...mapActions放methods中；

...mapXxxx('模块名', ['数据项|方法']),

...mapXxxx('模块名', { 新的名字: 原来的名字 }),

}

2.组件中直接使用 属性 `{{ age }}` 或 方法 `@click="updateAge(2)"`

二十二、综合案例 - 创建项目

1. 脚手架新建项目 (注意：勾选vuex)

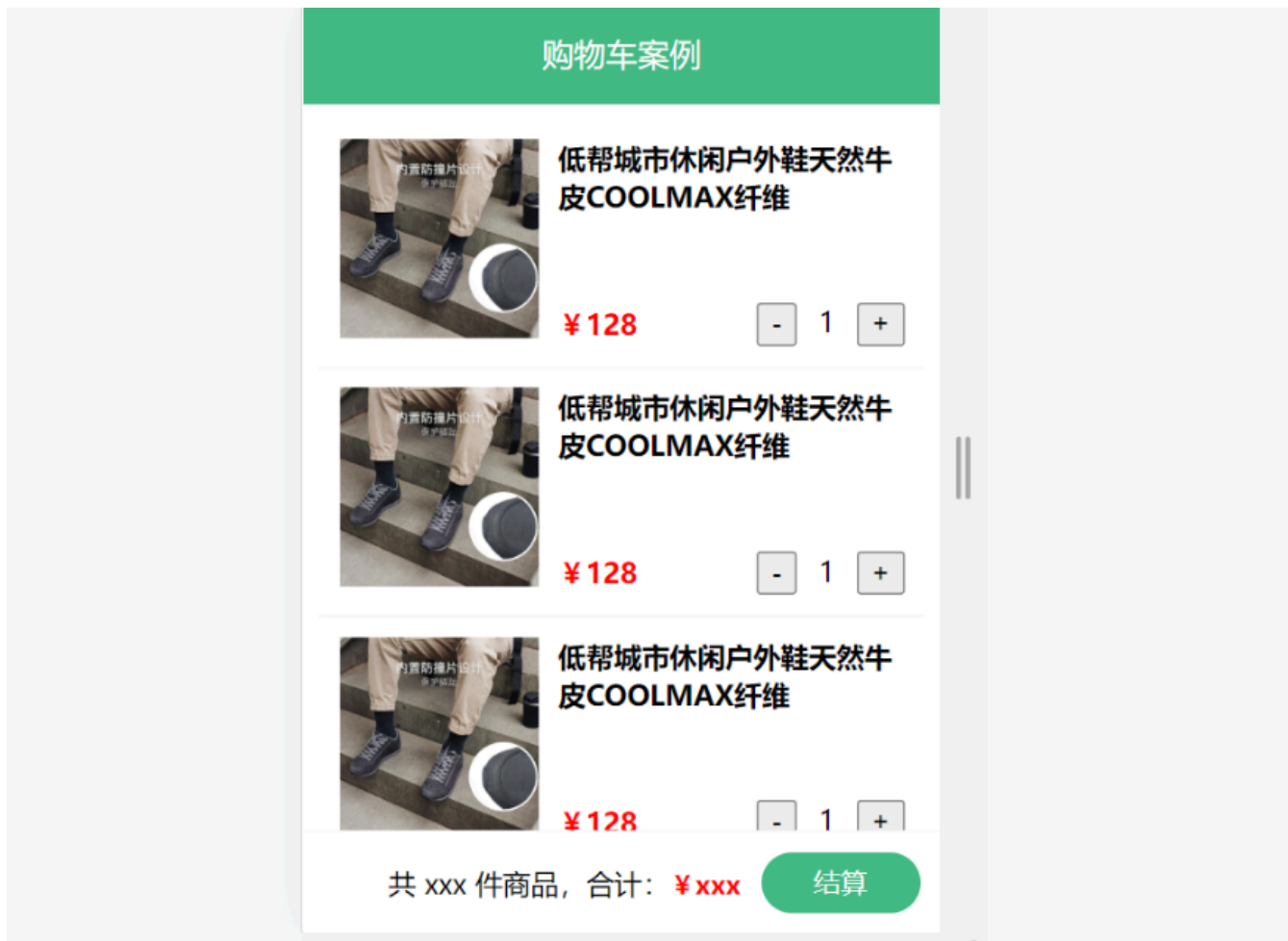
版本说明：

vue2 vue-router3 vuex3

vue3 vue-router4 vuex4/pinia

```
vue create vue-cart-demo
```

2. 将原本src内容清空，替换成教学资料的《vuex-cart-准备代码》



需求：

1. 发请求动态渲染购物车，数据存vuex（存cart模块，将来还会有user模块，article模块...）
2. 数字框可以修改数据
3. 动态计算总价和总数量

二十三、综合案例-构建vuex-cart模块

1. 新建 `store/modules/cart.js`

```
export default {
  namespaced: true,
  state () {
    return {
      list: []
    }
  },
}
```

2. 挂载到 vuex 仓库上 `store/cart.js`

```
import Vuex from 'vuex'
import Vue from 'vue'
```

```
import cart from './modules/cart'

Vue.use(Vuex)

const store = new Vuex.Store({
  modules: {
    cart
  }
})

export default store
```

二十四、综合案例-准备后端接口服务环境(了解)

1. 安装全局工具 json-server （全局工具仅需要安装一次）

```
yarn global add json-server 或 npm i json-server -g
```

2. 代码根目录新建一个 db 目录
3. 将资料 index.json 移入 db 目录
4. 进入 db 目录，执行命令，启动后端接口服务 (使用--watch 参数 可以实时监听 json 文件的修改)

```
json-server --watch index.json
```

二十五、综合案例-请求动态渲染数据

1.目标

请求获取数据存入 vuex, 映射渲染



1. 安装 axios

```
yarn add axios
```

2. 准备actions 和 mutations

```
import axios from 'axios'

export default {
```

```

namespaced: true,
state () {
  return {
    list: []
  }
},
mutations: {
  updateList (state, payload) {
    state.list = payload
  }
},
actions: {
  async getList (ctx) {
    const res = await axios.get('http://localhost:3000/cart')
    ctx.commit('updateList', res.data)
  }
}
}

```

3. App.vue 页面中调用 action, 获取数据

```

import { mapState } from 'vuex'

export default {
  name: 'App',
  components: {
    CartHeader,
    CartFooter,
    CartItem
  },
  created () {
    this.$store.dispatch('cart/getList')
  },
  computed: {
    ...mapState('cart', ['list'])
  }
}

```

4. 动态渲染

```

<!-- 商品 Item 项组件 -->
<cart-item v-for="item in list" :key="item.id" :item="item"></cart-item>

```

cart-item.vue

```

<template>
  <div class="goods-container">
    <!-- 左侧图片区域 -->
    <div class="left">
      
    </div>
    <!-- 右侧商品区域 -->
    <div class="right">
      <!-- 标题 -->

```

```

<div class="title">{{item.name}}</div>
<div class="info">
  <!-- 单价 -->
  <span class="price">¥{{item.price}}</span>
  <div class="btns">
    <!-- 按钮区域 -->
    <button class="btn btn-light">--</button>
    <span class="count">{{item.count}}</span>
    <button class="btn btn-light">+</button>
  </div>
</div>
</div>
</div>
</template>

<script>
export default {
  name: 'CartItem',
  props: {
    item: Object
  },
  methods: {

  }
}
</script>

```

二十六、综合案例-修改数量

注意：前端 vuex 数据，后端数据库数据都要更新



1. 注册点击事件

```

<!-- 按钮区域 -->
<button class="btn btn-light" @click="onBtnClick(-1)">--</button>
<span class="count">{{item.count}}</span>
<button class="btn btn-light" @click="onBtnClick(1)">+</button>

```

2. 页面中dispatch action


```

onBtnClick (step) {
  const newCount = this.item.count + step
  if (newCount < 1) return

  // 发送修改数量请求
  this.$store.dispatch('cart/updateCount', {
    id: this.item.id,
    count: newCount
  })
}

```

3. 提供action函数

```

async updateCount (ctx, payload) {
  await axios.patch('http://localhost:3000/cart/' + payload.id, {
    count: payload.count
  })
  ctx.commit('updateCount', payload)
}

```

4. 提供mutation处理函数

```

mutations: {
  ...,
  updateCount (state, payload) {
    const goods = state.list.find((item) => item.id === payload.id)
    goods.count = payload.count
  },
}

```

二十七、综合案例-底部总价展示

1. 提供getters

```

getters: {
  total(state) {
    return state.list.reduce((p, c) => p + c.count, 0);
  },
  totalPrice (state) {
    return state.list.reduce((p, c) => p + c.count * c.price, 0);
  },
},

```

2. 动态渲染

```

<template>
  <div class="footer-container">
    <!-- 中间的合计 -->
    <div>
      <span>共 {{total}} 件商品, 合计: </span>
    
```

```
      <span class="price">¥{{totalPrice}}</span>
    </div>
    <!-- 右侧结算按钮 -->
    <button class="btn btn-success btn-settle">结算</button>
  </div>
</template>

<script>
import { mapGetters } from 'vuex'
export default {
  name: 'CartFooter',
  computed: {
    ...mapGetters('cart', ['total', 'totalPrice'])
  }
}
</script>
```

认识Vue3

1. Vue2 选项式 API vs Vue3 组合式API

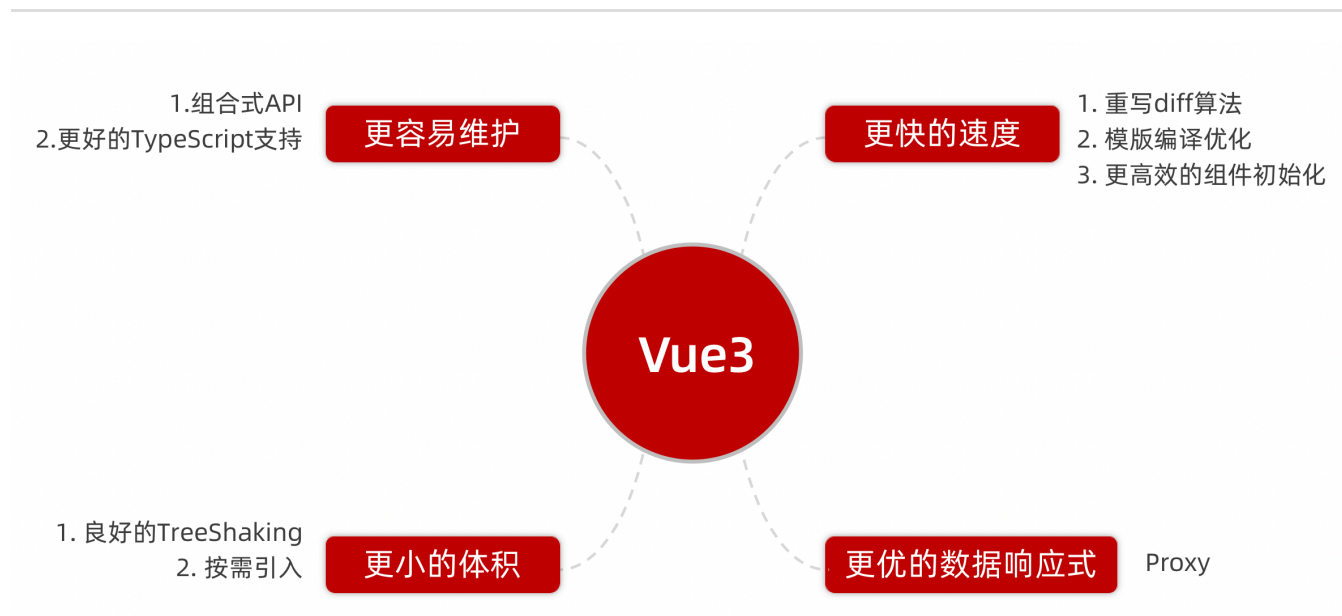
```
<script>
export default {
  data(){
    return {
      count:0
    }
  },
  methods:{
    addCount(){
      this.count++
    }
  }
}
</script>
```

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
const addCount = () => count.value++
</script>
```

特点：

1. 代码量变少
2. 分散式维护变成集中式维护

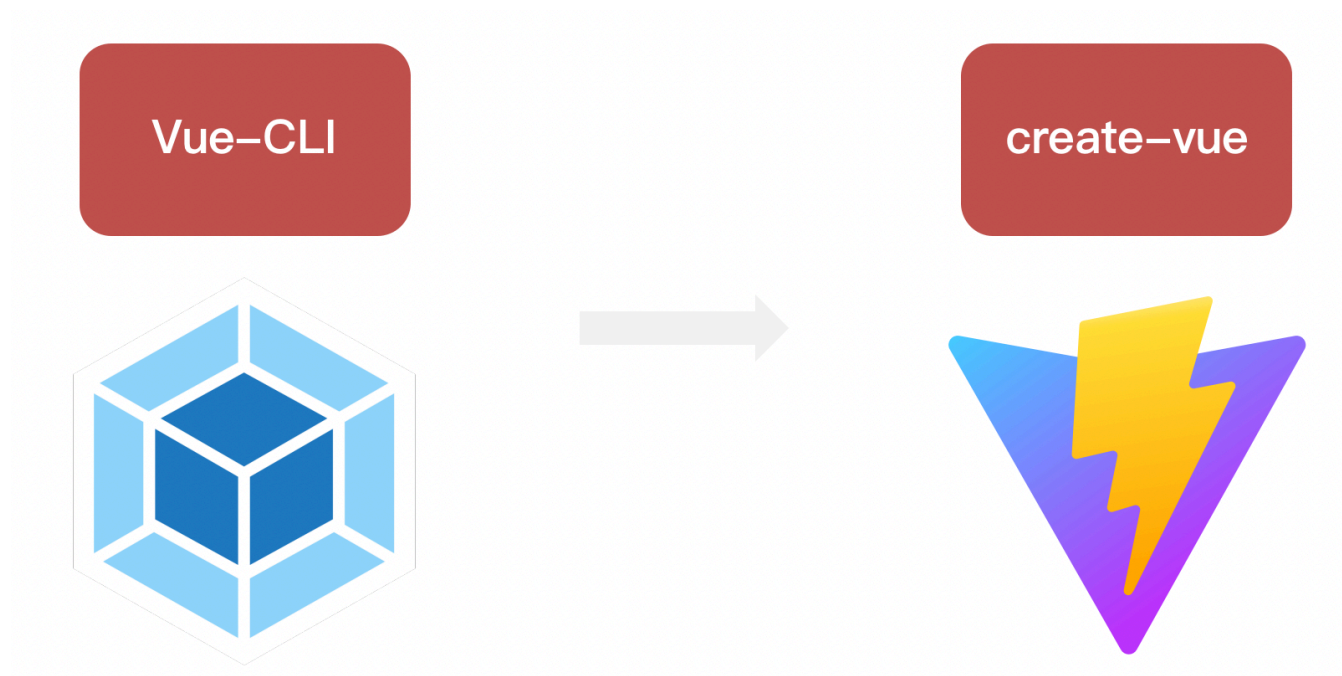
2. Vue3的优势



使用create-vue搭建Vue3项目

1. 认识create-vue

create-vue是Vue官方新的脚手架工具，底层切换到了 vite （下一代前端工具链），为开发提供极速响应



2. 使用create-vue创建项目

前置条件 - 已安装16.0或更高版本的Node.js

执行如下命令，这一指令将会安装并执行 create-vue

```
npm init vue@latest
```

```
chaipeng@localhost pro % npm init vue@latest

Vue.js - The Progressive JavaScript Framework

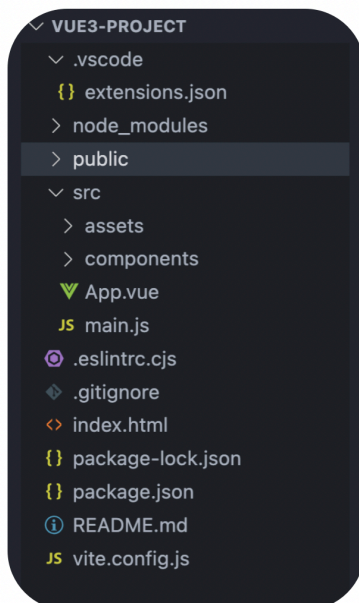
✓ Project name: ... vue3-project
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit Testing? ... No / Yes
✓ Add Cypress for both Unit and End-to-End testing? ... No / Yes
✓ Add ESLint for code quality? ... No / Yes
✓ Add Prettier for code formatting? ... No / Yes

Scaffolding project in /Users/chaipeng/Desktop/小兔鲜新版/pro/vue3-project...

Done. Now run:

cd vue3-project
npm install
npm run dev
```

熟悉项目和关键文件



关键文件：

1. vite.config.js - 项目的配置文件 基于vite的配置
2. package.json - 项目包文件 核心依赖项变成了 Vue3.x 和 vite
3. main.js - 入口文件 createApp函数创建应用实例
4. app.vue - 根组件 SFC单文件组件 script - template - style
 - 变化一：脚本script和模板template顺序调整
 - 变化二：模板template不再要求唯一根元素
 - 变化三：脚本script添加setup标识支持组合式API
5. index.html - 单页入口 提供id为app的挂载点

组合式API - setup选项

1. setup选项的写法和执行时机

写法

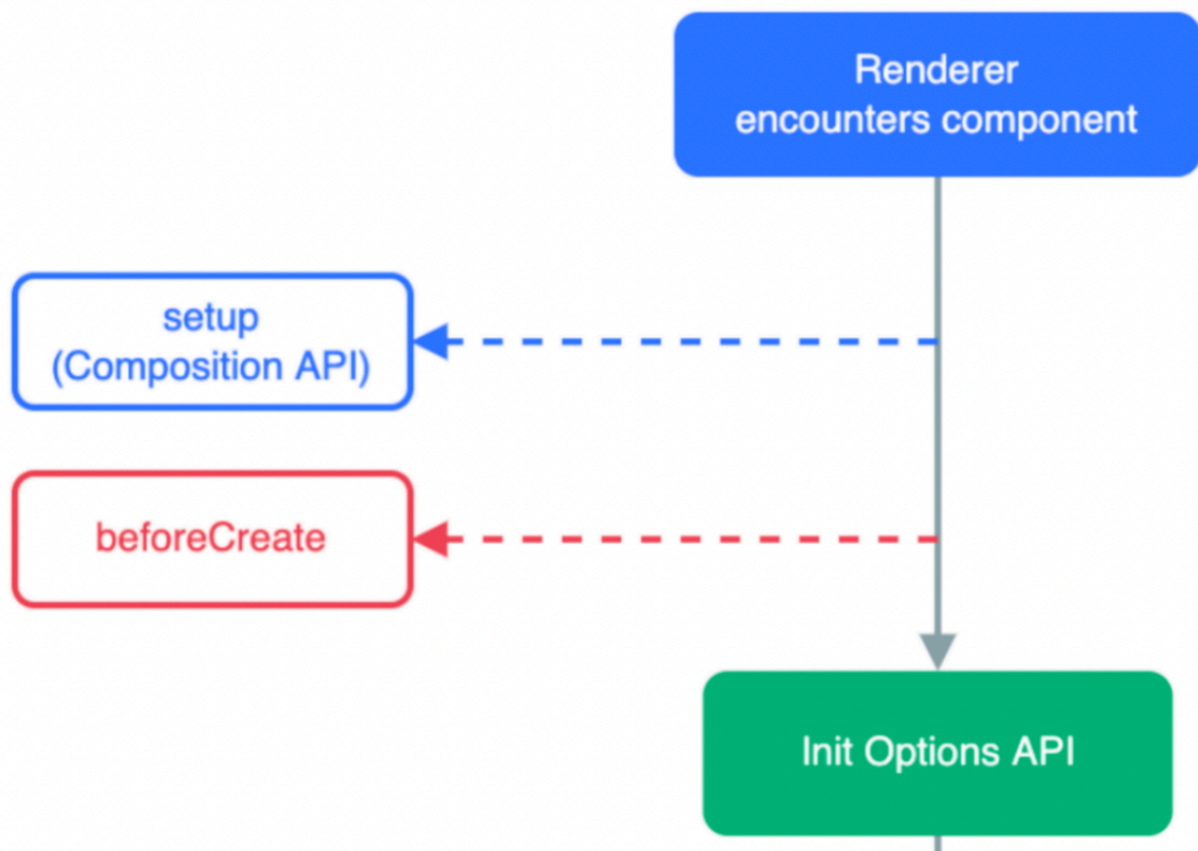
```
<script>
  export default {
    setup(){

    },
    beforeCreate(){

    }
  }
</script>
```

执行时机

在beforeCreate钩子之前执行



2. setup中写代码的特点

在setup函数中写的数据和方法需要在末尾以对象的方式return，才能给模版使用

```
<script>
  export default {
    setup(){
      const message = 'this is message'
      const logMessage = ()=>{
        console.log(message)
      }
      // 必须return才可以
      return {
        message,
        logMessage
      }
    }
  }
</script>
```

3. 语法糖

script标签添加 setup标记，不需要再写导出语句，默认会添加导出语句

```
<script setup>
  const message = 'this is message'
  const logMessage = ()=>{
    console.log(message)
  }
</script>
```

组合式API - reactive和ref函数

1. reactive

接受对象类型数据的参数传入并返回一个响应式的对象

```
<script setup>
  // 导入
  import { reactive } from 'vue'
  // 执行函数 传入参数 变量接收
  const state = reactive({
    msg: 'this is msg'
  })
```

```
const setSate = ()=>{
  // 修改数据更新视图
  state.msg = 'this is new msg'
}
</script>

<template>
  {{ state.msg }}
  <button @click="setSate">change msg</button>
</template>
```

2. ref

接收简单类型或者对象类型的数据传入并返回一个响应式的对象

```
<script setup>
  // 导入
  import { ref } from 'vue'
  // 执行函数 传入参数 变量接收
  const count = ref(0)
  const setCount = ()=>{
    // 修改数据更新视图必须加上.value
    count.value++
  }
</script>

<template>
  <button @click="setCount">{{count}}</button>
</template>
```

3. reactive 对比 ref

1. 都是用来生成响应式数据
2. 不同点
 1. reactive不能处理简单类型的数据
 2. ref参数类型支持更好，但是必须通过.value做访问修改
 3. ref函数内部的实现依赖于reactive函数
3. 在实际工作中的推荐
 1. 推荐使用ref函数，减少记忆负担，小兔鲜项目都使用ref

组合式API - computed

计算属性基本思想和Vue2保持一致，组合式API下的计算属性只是修改了API写法

```
<script setup>
// 导入
import {ref, computed} from 'vue'
// 原始数据
const count = ref(0)
// 计算属性
const doubleCount = computed(() => count.value * 2)

// 原始数据
const list = ref([1, 2, 3, 4, 5, 6, 7, 8])
// 计算属性list
const filterList = computed(item => item > 2)
</script>
```

组合式API - watch

侦听一个或者多个数据的变化，数据变化时执行回调函数，两个额外参数 immediate控制立刻执行，deep开启深度侦听

1. 侦听单个数据

```
<script setup>
// 1. 导入watch
import {ref, watch} from 'vue'
const count = ref(0)
// 2. 调用watch 侦听变化
watch(count, (newValue, oldValue) => {
  console.log(`count发生了变化, 老值为${oldValue}, 新值为${newValue}`)
})
</script>
```

2. 侦听多个数据

侦听多个数据，第一个参数可以改写成数组的写法

```

<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const count = ref(0)
  const name = ref('cp')
  // 2. 调用watch 侦听变化
  watch([count, name], ([newCount, newName],[oldCount,oldName])=>{
    console.log(`count或者name变化了, [newCount, newName],[oldCount,oldName]`)
  })
</script>

```

3. immediate

在侦听器创建时立即出发回调，响应式数据变化之后继续执行回调

```

<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const count = ref(0)
  // 2. 调用watch 侦听变化
  watch(count, (newValue, oldValue)>=>{
    console.log(`count发生了变化, 老值为${oldValue}, 新值为${newValue}`)
  }, {
    immediate: true
  })
</script>

```

4. deep

通过watch监听的ref对象默认是浅层侦听的，直接修改嵌套的对象属性不会触发回调执行，需要开启deep

```

<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const state = ref({ count: 0 })
  // 2. 监听对象state
  watch(state, ()=>{
    console.log('数据变化了')
  })
  const changeStateByCount = ()=>{
    // 直接修改不会引发回调执行
    state.value.count++
  }
</script>

<script setup>
  // 1. 导入watch

```

```
import { ref, watch } from 'vue'
const state = ref({ count: 0 })
// 2. 监听对象state 并开启deep
watch(state, ()=>{
  console.log('数据变化了')
},{deep:true})
const changeStateByCount = ()=>{
  // 此时修改可以触发回调
  state.value.count++
}
</script>
```

组合式API - 生命周期函数

1. 选项式对比组合式

选项式API	组合式API
beforeCreate/created	setup
beforeMount	onBeforeMount
mounted	onMounted
beforeUpdate	onBeforeUpdate
updated	onUpdated
beforeUnmount	onBeforeUnmount
unmounted	onUnmounted

2. 生命周期函数基本使用

- 1. 导入生命周期函数
- 2. 执行生命周期函数，传入回调

```
<script setup>
import { onMounted } from 'vue'
onMounted(()=>{
  // 自定义逻辑
})
</script>
```

3. 执行多次

生命周期函数执行多次的时候，会按照顺序依次执行

```
<script setup>
import { onMounted } from 'vue'
onMounted(()=>{
  // 自定义逻辑
})

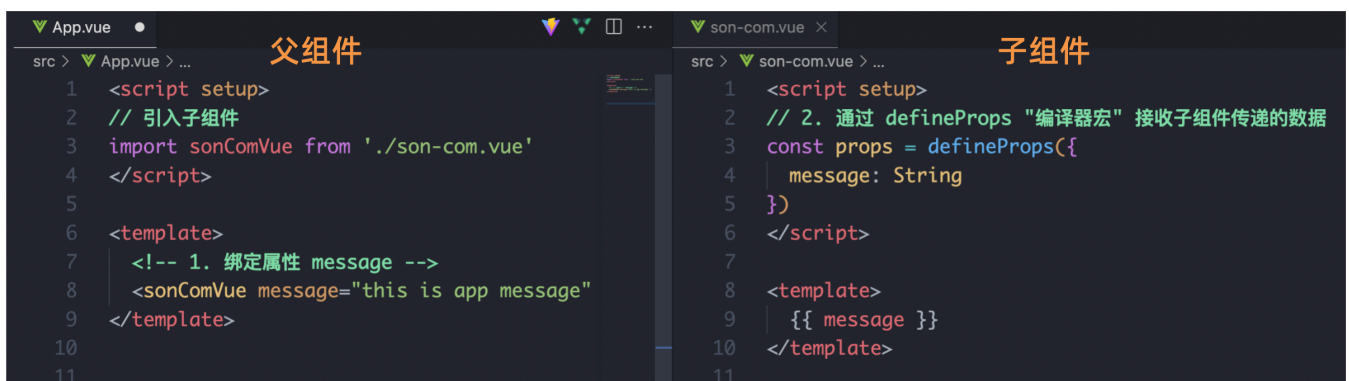
onMounted(()=>{
  // 自定义逻辑
})
</script>
```

组合式API - 父子通信

1. 父传子

基本思想

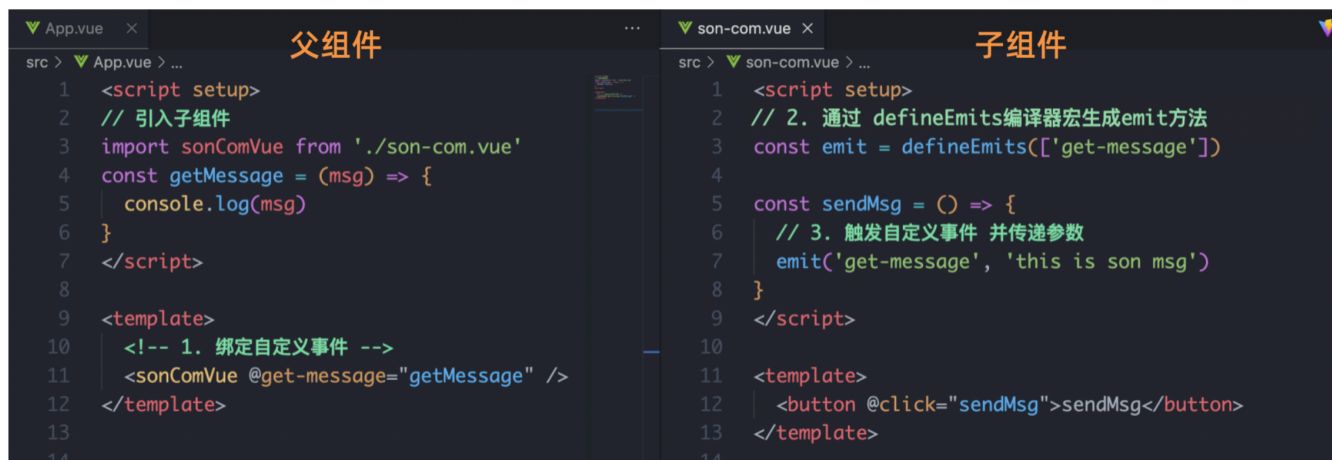
1. 父组件中给子组件绑定属性
2. 子组件内部通过props选项接收数据



2. 子传父

基本思想

1. 父组件中给子组件标签通过@绑定事件
2. 子组件内部通过 emit 方法触发事件



```
src > ▼ App.vue > ... 父组件
1 <script setup>
2 // 引入子组件
3 import sonComVue from './son-com.vue'
4 const getMessage = (msg) => {
5   console.log(msg)
6 }
7 </script>
8
9 <template>
10 <!-- 1. 绑定自定义事件 -->
11 <sonComVue @get-message="getMessage" />
12 </template>
13
14

src > ▼ son-com.vue > ... 子组件
1 <script setup>
2 // 2. 通过 defineEmits编译器宏生成emit方法
3 const emit = defineEmits(['get-message'])
4
5 const sendMsg = () => {
6   // 3. 触发自定义事件 并传递参数
7   emit('get-message', 'this is son msg')
8 }
9 </script>
10
11 <template>
12 <button @click="sendMsg">sendMsg</button>
13 </template>
14
```

组合式API - 模版引用

概念：通过 ref标识 获取真实的 dom对象或者组件实例对象

1. 基本使用

实现步骤：

1. 调用ref函数生成一个ref对象
2. 通过ref标识绑定ref对象到标签



```
1  <script setup>
2  import { ref } from 'vue'
3  // 1. 调用ref函数得到ref对象
4  const h1Ref = ref(null)
5  </script>
6
7  <template>
8    <!-- 2. 通过ref标识绑定ref对象 -->
9    <h1 ref="h1Ref">我是dom标签h1</h1>
10 </template>
11
```

2. defineExpose

默认情况下在 语法糖下组件内部的属性和方法是不开放给父组件访问的，可以通过defineExpose编译宏指定哪些属性和方法容许访问

说明：指定testMessage属性可以被访问到



```
1  <script setup>
2  import { ref } from 'vue'
3  const testMessage = ref('this is test msg')
4  </script>
5
```

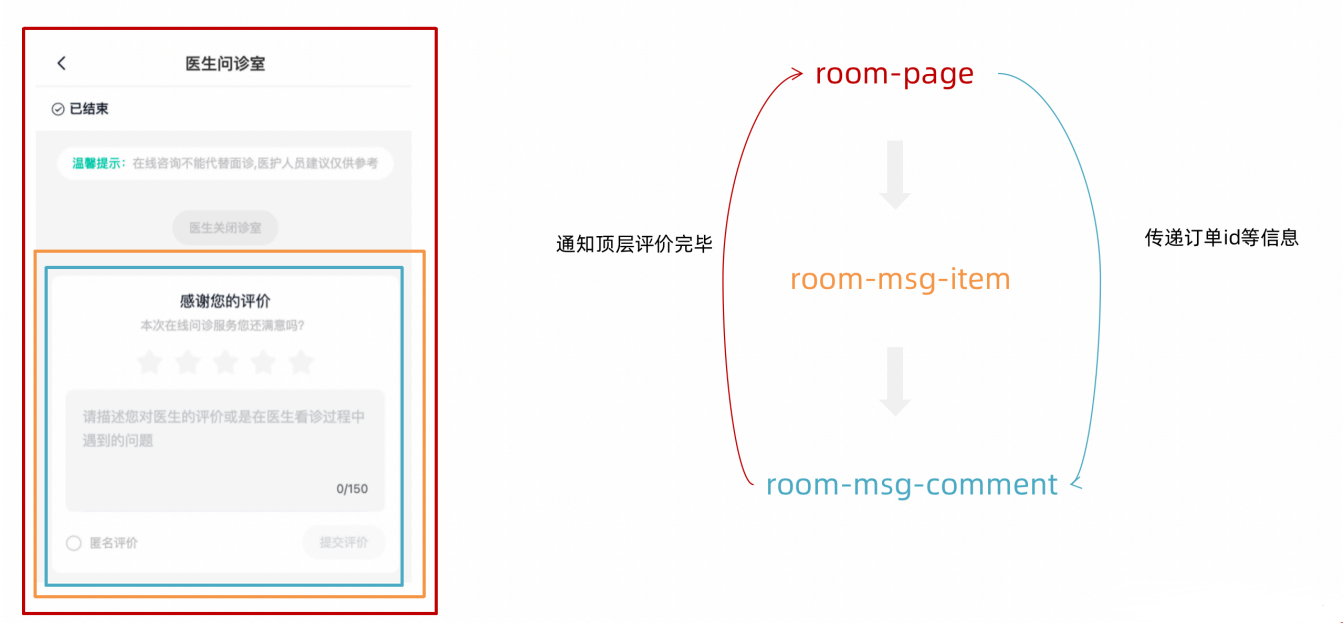


```
1  <script setup>
2  import { ref } from 'vue'
3  const testMessage = ref('this is test msg')
4  defineExpose({
5    testMessage
6  })
7  </script>
```

组合式API - provide和inject

1. 作用和场景

顶层组件向任意的底层组件传递数据和方法，实现跨层组件通信



2. 跨层传递普通数据

- 实现步骤
1. 顶层组件通过 `provide` 函数提供数据
 2. 底层组件通过 `inject` 函数提供数据

顶层组件

```
1 provide('key', 顶层组件中的数据)
```

底层组件

```
1 const message = inject('key')
```

3. 跨层传递响应式数据

在调用provide函数时，第二个参数设置为ref对象

顶层组件

```
1 provide('app-key', ref对象)
```

底层组件

```
1 const message = inject('app-key')
```

4. 跨层传递方法

顶层组件可以向底层组件传递方法，底层组件调用方法修改顶层组件的数据


```
1 const setCount = () => {
2   count.value++
3 }
4
5 provide('setCount-key', setCount)
```

顶层组件

```
1 const setCount = inject('setCount-key')
```

底层组件

Vue3.3 新特性-defineOptions

背景说明：

有 之前，如果要定义 props, emits 可以轻而易举地添加一个与 setup 平级的属性。

但是用了 后，就没法这么干了 setup 属性已经没有了，自然无法添加与其平级的属性。

为了解决这一问题，引入了 defineProps 与 defineEmits 这两个宏。但这只解决了 props 与 emits 这两个属性。

如果我们要定义组件的 name 或其他自定义的属性，还是得回到最原始的用法——再添加一个普通的 标签。

这样就会存在两个 标签。让人无法接受。

所以在 Vue 3.3 中新引入了 defineOptions 宏。顾名思义，主要是用来定义 Options API 的选项。可以用 defineOptions 定义任意的选项， props, emits, expose, slots 除外（因为这些可以使用 defineXXX 来做到）

```
1 <script setup>
2   defineOptions({
3     name: 'Foo',
4     inheritAttrs: false,
5     // ... 更多自定义属性
6   })
7 </script>
```

vue 复制代码

Vue3.3新特性-defineModel

在Vue3中，自定义组件上使用v-model, 相当于传递一个modelValue属性，同时触发 update:modelValue 事件

```
1 <Child v-model="isVisible">
2 // 相当于
3 <Child :modelValue="isVisible" @update:modelValue="isVisible=$event">
```

我们需要先定义 props，再定义 emits。其中有许多重复的代码。如果需要修改此值，还需要手动调用 emit 函数。
于是乎 defineModel 诞生了。

```
vue 复制代码
1 <script setup>
2 const modelValue = defineModel()
3 modelValue.value++
4 </script>
```

生效需要配置 vite.config.js

```
import { fileURLToPath, URL } from 'node:url'

import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [
    vue({
      script: {
        defineModel: true
      }
    })
  ],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url))
    }
  }
})
```

Vue3 状态管理 - Pinia

1. 什么是Pinia

Pinia 是 Vue 的专属的最新状态管理库，是 Vuex 状态管理工具的替代品



1. 提供更加简单的API（去掉了 mutation）
2. 提供符合组合式风格的API（和 Vue3 新语法统一）
3. 去掉了 modules 的概念，每一个 store 都是一个独立的模块
4. 配合 TypeScript 更加友好，提供可靠的类型推断

2. 手动添加Pinia到Vue项目

后面在实际开发项目的时候，Pinia可以在项目创建时自动添加，现在我们初次学习，从零开始：

1. 使用 Vite 创建一个空的 Vue3项目

```
npm init vite@latest
```

2. 按照官方文档安装 pinia 到项目中

3. Pinia基础使用

1. 定义store
2. 组件使用store

定义Store (state + action)

```

1 import { defineStore } from 'pinia'
2 import { ref } from 'vue'
3 export const useCounterStore = defineStore('counter', () => {
4   // 数据 (state)
5   const count = ref(0)
6
7   // 修改数据的方法 (action)
8   const increment = () => {
9     count.value++
10  }
11
12  // 以对象形式返回
13  return { count, increment }
14 })

```



组件使用Store

```

1 <script setup>
2 // 1. 导入 `useCounterStore` 方法
3 import { useCounterStore } from '@/stores/counter'
4 // 2. 执行方法得到counterStore对象
5 const counterStore = useCounterStore()
6 </script>
7
8 <template>
9   <button @click="counterStore.increment">
10     {{ counterStore.count }}
11   </button>
12 </template>
13

```

4. getters实现

Pinia中的 getters 直接使用 computed函数 进行模拟, 组件中需要使用需要把 getters return出去

```

1 // 数据 (state)
2 const count = ref(0)
3
4 // getter
5 const doubleCount = computed(() => count.value * 2)

```

5. action异步实现

方式：异步action函数的写法和组件中获取异步数据的写法完全一致

- 接口地址： http://geek.itheima.net/v1_0/channels
- 请求方式：get
- 请求参数：无



```
1 // 异步action
2 const getList = async ()=>{
3   const res = await axios.request<接口数据类型>({})
4 }
```

需求：在Pinia中获取频道列表数据并把数据渲染App组件的模板中

object {2}

▼ data

object {1}

▼ channels

array[object] {2}

id

integer

name

string

message

string

6. storeToRefs工具函数

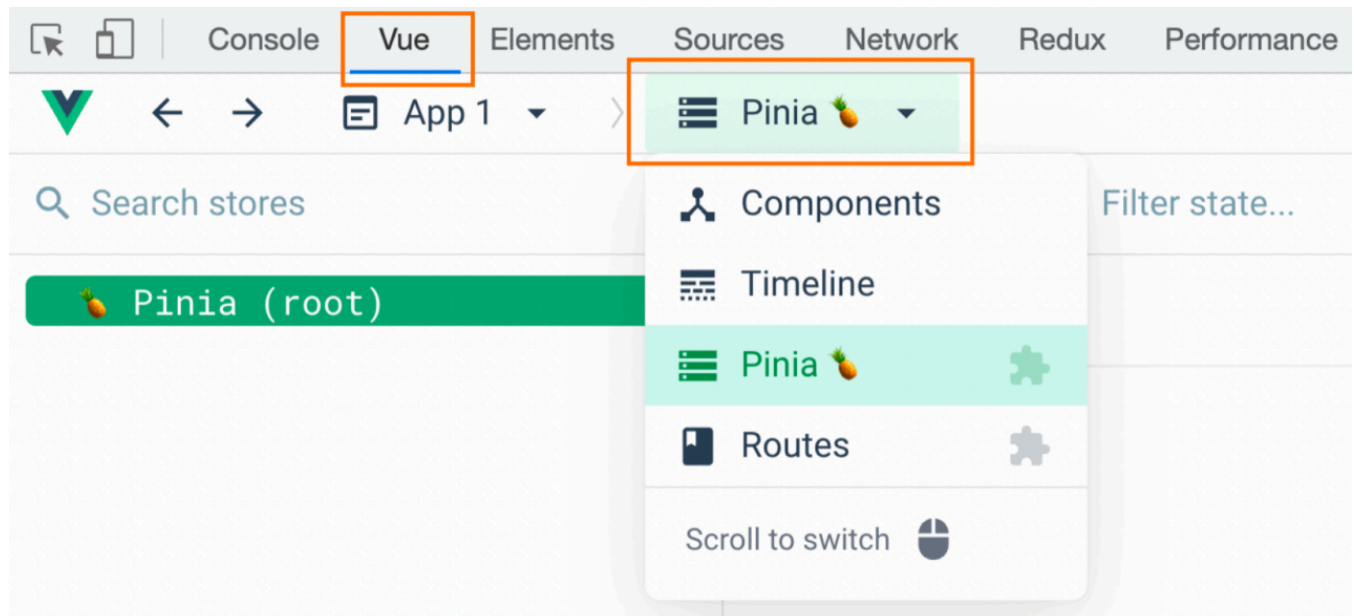
使用storeToRefs函数可以辅助保持数据（state + getter）的响应式解构

```
1 // 响应式丢失 视图不再更新
2 const { count, doubleCount } = counterStore
```

```
1 // 保持数据响应式
2 const { count, doubleCount } = storeToRefs(counterStore)
```

7. Pinia的调试

Vue官方的 dev-tools 调试工具 对 Pinia直接支持，可以直接进行调试



8. Pinia持久化插件

官方文档: <https://prazdevs.github.io/pinia-plugin-persistedstate/zh/>

1. 安装插件 pinia-plugin-persistedstate

```
npm i pinia-plugin-persistedstate
```

2. 使用 main.js

```
import persist from 'pinia-plugin-persistedstate'  
...  
app.use(createPinia()).use(persist)
```

3. 配置 store/counter.js

```
import { defineStore } from 'pinia'  
import { computed, ref } from 'vue'  
  
export const useCounterStore = defineStore('counter', () => {  
  ...  
  return {  
    count,  
    doubleCount,  
    increment  
  }  
}, {  
  persist: true  
})
```

4. 其他配置, 看官网文档即可